

НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ
імені ІГОРЯ СІКОРСЬКОГО»
Факультет інформатики та обчислювальної техніки

(повне найменування інституту, факультету)

Інформатики та програмної інженерії

(повна назва кафедри)

«До захисту допущено»

Максим ГОЛОВЧЕНКО
(підпис) (ініціали, прізвище)
“ ” 2024 р.

Курсова робота

з дисципліни Компоненти програмної інженерії
за освітньо-професійною програмою «Програмне забезпечення інформаційних
управляючих систем та технологій»
спеціальності «121 Інженерія програмного забезпечення»

на тему Гра для мобільного пристрою «Монополія»

Виконав: студент III курсу, групи ПІ-14 Бабіч Денис Володимирович
(прізвище, ім'я, по батькові) (підпис)

Керівник ст. вик. Головченко М.М.
посада, науковий ступінь, вчене звання, прізвище, і ім'я, по батькові (підпис)

Члени комісії ст. вик. Головченко М.М.
посада, науковий ступінь, вчене звання, прізвище, і ім'я, по батькові (підпис)

посада, науковий ступінь, вчене звання, прізвище, і ім'я, по батькові (підпис)

Засвідчую, що у цій курсовій роботі немає
запозичень з праць інших авторів без
відповідних посилань.

Студент (підпис)

Національний технічний університет України
“Київський політехнічний інститут імені Ігоря Сікорського”

Факультет (інститут) Інформатики та обчислювальної техніки
(повна назва)

Кафедра Інформатики та програмної інженерії
(повна назва)

Рівень вищої освіти – перший (бакалаврський)

Спеціальність – *121 Інженерія програмного забезпечення*

Освітньо-професійна програма – *Програмне забезпечення інформаційних
управляючих систем та технологій*

ЗАТВЕРДЖУЮ

Максим ГОЛОВЧЕНКО
(підпис) (ініціали, прізвище)
“ ” 2024 р.

ЗАВДАННЯ
НА КУРСОВУ РОБОТУ СТУДЕНТУ

Бабіча Дениса Володимировича
(прізвище, ім'я, по батькові)

1. Тема роботи «Гра для мобільного пристрою Монополія»

керівник роботи Головченко Максим Миколайович, ст. викладач
(прізвище, ім'я, по батькові, науковий ступінь, вчене звання)

2. Термін подання студентом роботи «16» січня 2024 року

3. Вихідні дані до роботи

*Технічне завдання, керівництво користувача, методика тестування, текст
програми*

4. Зміст пояснювальної записки

*1) Аналіз вимог до програмного забезпечення: вивчення вимог до програмного
забезпечення, аналіз середовища предметної області, аналіз існуючих рішень,
формулювання функціональних та нефункціональних вимог*

2) Моделювання та конструювання програмного забезпечення: моделювання та

аналіз програмного забезпечення за допомогою BPMN-діаграм, аналіз засобів розробки, технічні рішення, визначення архітектури програмного забезпечення

3) Аналіз якості та тестування програмного забезпечення: оцінка якості якості написаного коду завдяки відповідним метриками, тестування програмного забезпечення використовуючи мануальне тестування, опис контрольного прикладу.

4) Впровадження та супровід програмного забезпечення: опис процесу отримання користувачем поточної версії продукту та отримання оновлень

5. Перелік графічного матеріалу

1) Діаграма компонентів ігрового мобільного застосунку

2) Діаграма варіантів використання

3) Діаграми моделювання бізнес-процесів

6. Дата видачі завдання «12» жовтня 2022 року

КАЛЕНДАРНИЙ ПЛАН

№ з/п	Назва етапів виконання курсової роботи	Термін виконання етапів роботи	Примітка
1.	Вивчення рекомендованої літератури	5.09.2023 - 30.09.2023	
2.	Аналіз існуючих методів розв'язання задачі	5.10.2023 - 10.10.2023	
3.	Постановка та формалізація задачі	13.10.2023 - 14.10.2023	
4.	Аналіз вимог до програмного забезпечення	14.10.2023- 15.10.2023	
5.	Алгоритмізація задачі	17.10.2023- 18.10.2023	
6.	Моделювання програмного забезпечення	19.10.2023- 20.10.2023	
7.	Обґрунтування використовуваних технічних засобів	21.10.2023- 22.10.2023	
8.	Розробка архітектури програмного забезпечення	23.10.2023- 24.10.2023	
9.	Розробка програмного забезпечення	25.11.2023- 26.11.2023	
10.	Налагодження програми	27.11.2023- 29.11.2023	
11.	Виконання графічних документів	30.11.2023- 05.12.2023	
12.	Оформлення пояснювальної записки	05.12.2023- 14.12.2023	
13.	Подання КР на перевірку	16.01.2024	
14.	Захист КР	22.01.2024	

Студент

(підпис)

Денис БАБІЧ

Керівник

(підпис)

Максим ГОЛОВЧЕНКО

[illegible]

Факультет інформатики та обчислювальної техніки

Кафедра інформатики та програмної інженерії

“ЗАТВЕРДЖЕНО”

Керівник роботи

_____ Максим ГОЛОВЧЕНКО

“ ” _____ 2024 р.

ГРА ДЛЯ МОБІЛЬНОГО ПРИСТРОЮ “МОНОПОЛІЯ”

Технічне завдання

КП.ІП-1402.045490.01.91

“ПОГОДЖЕНО”

Керівник роботи:

_____ Максим ГОЛОВЧЕНКО

Консультант:

_____ Максим ГОЛОВЧЕНКО

Виконавець:

_____ Денис БАБІЧ

ЗМІСТ

1 НАЙМЕНУВАННЯ ТА ГАЛУЗЬ ЗАСТОСУВАННЯ.....	4
2 ПІДСТАВА ДЛЯ РОЗРОБКИ.....	5
3 ПРИЗНАЧЕННЯ РОЗРОБКИ.....	6
4 ВИМОГИ ДО ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ.....	7
4.1 Вимоги до функціональних характеристик.....	7
4.1.1 Ігрових механік.....	7
4.1.1 Користувацького інтерфейсу.....	7
4.1.2 Для користувача:.....	14
4.1.3 Для адміністратора системи:.....	14
4.1.4 Додаткові вимоги:.....	14
4.2 Вимоги до надійності.....	14
4.3 Умови експлуатації.....	14
4.3.1 Вид обслуговування.....	15
4.3.2 Обслуговуючий персонал.....	15
4.4 Вимоги до складу і параметрів технічних засобів.....	15
4.5 Вимоги до інформаційної та програмної сумісності.....	15
4.5.1 Вимоги до вхідних даних.....	15
4.5.2 Вимоги до вихідних даних.....	15
4.5.3 Вимоги до мови розробки.....	15
4.5.4 Вимоги до середовища розробки.....	16
4.5.5 Вимоги до представлення вихідних кодів.....	16
4.6 Вимоги до маркування та пакування.....	16
4.7 Вимоги до транспортування та зберігання.....	16
4.8 Спеціальні вимоги.....	16
5 ВИМОГИ ДО ПРОГРАМНОЇ ДОКУМЕНТАЦІЇ.....	17
5.1 Попередній склад програмної документації.....	17
5.2 Спеціальні вимоги до програмної документації.....	17

6 СТАДІЇ І ЕТАПИ РОЗРОБКИ.....	18
7 ПОРЯДОК КОНТРОЛЮ ТА ПРИЙМАННЯ.....	19

1 НАЙМЕНУВАННЯ ТА ГАЛУЗЬ ЗАСТОСУВАННЯ

Назва розробки: Гра для мобільного пристрою “Монополія”.

Галузь застосування: Наведене технічне завдання поширюється на розробку гри для мобільного пристрою під назвою “Монополія”, котре використовується для розважальних цілей, розвитку стратегічного мислення та призначена для широко спектру користувачів, людей різного віку, які мають інтерес до настільних ігор, покрокових стратегій та змагального ігрового процесу.

2 ПІДСТАВА ДЛЯ РОЗРОБКИ

Підставою для розробки гри для мобільного пристрою “Монополія” є завдання на курсову роботу, затверджене кафедрою інформатики та програмної інженерії Національного технічного університету України «Київський політехнічний інститут імені Ігоря Сікорського».

3 ПРИЗНАЧЕННЯ РОЗРОБКИ

Розробка призначена для відтворення класичної настільної гри під назвою “Монополія” у 2Д на мобільних пристроях з багатокористувацьким режимом до 5 людей, що реалізований через P2P-з’єднання.

Метою розробки є покращення ігрового досвіду користувачів шляхом підвищення стабільності, продуктивності додатку, покращеного користувацького графічного інтерфейсу та змінених ігрових правил, які забезпечують більш швидкий темп гри у порівнянні з класичною версією, що у результаті створює покращений ігровий досвід для користувачів.

4 ВИМОГИ ДО ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

4.1 Вимоги до функціональних характеристик

Програмне забезпечення повинно забезпечувати виконання наступних основних функцій:

4.1.1 Ігрових механік

- Кидання кубика. Коли настає черга конкретного гравця здійснити свій хід, то надається можливість кинути кубики, відповідного до виданого значення яких і відбудеться пересування ігровим полем;
- Взаємодія з полями. У випадку зібрання повної монополії, гравцю перед кожним ходом надається можливість покращити поля;
- Можливість закласти (у випадку, якщо монополія не розбудована), або зняти покращення з поля (у випадку, коли монополія вже зібрана та має покращення);
- Система обмінів. Можливість обміну між гравцями полями та ігровою валютою.

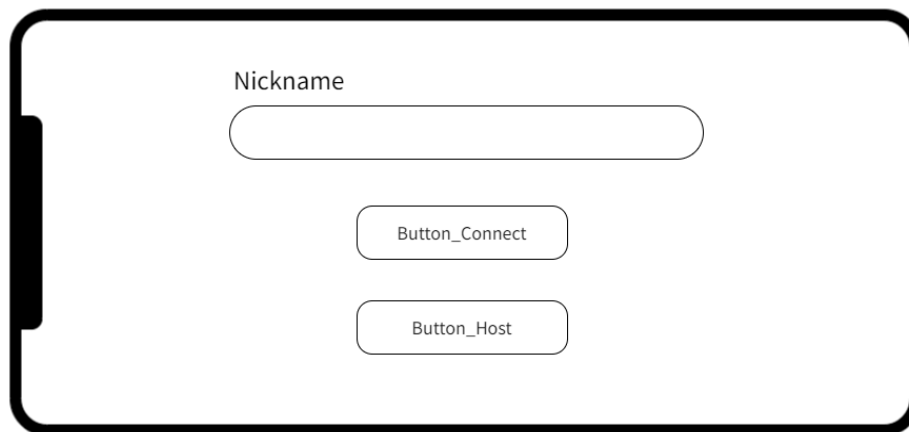
4.1.2 Елементів ігрового світу

- Ігрове поле з різними типами полів: стандартне, шансу, в'язниця, стартове, відправлення до в'язниці.

4.1.1 Користувацького інтерфейсу

4.1.3.1 Вікно під'єднання до ігрової кімнати (Рисунок 4.1)

- Вікно під'єднання до створеного лобі (Рисунок 4.2);
- У випадку невдалої спроби з'єднання спливає повідомлення про помилку, яка призвела до невдалої спроби (Рисунок 4.3);
- У випадку вдалої спроби з'єднання демонструється поточної ігрової кімнати (Рисунок 4.4).

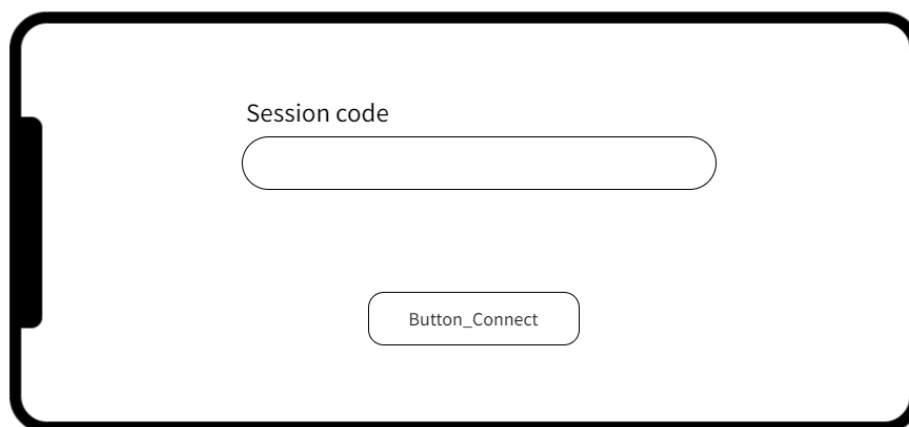


Nickname

Button_Connect

Button_Host

Рисунок 4.1 – Вікно вибору способу під’єднання до ігрової кімнати, де “Button_Connect” дозволяє приєднатися до створеної сесії за допомогою наданого коду, і “Button_Host” відповідає за створення такої ігрової сесії



Session code

Button_Connect

Рисунок 4.2 – Вікно під’єднання до створеного лобі



#ERROR MESSAGE

OK

Рисунок 4.3 – Спливаюче вікно про помилку у випадку невдалої спроби під’єднання до створеного лобі, або під час його створення

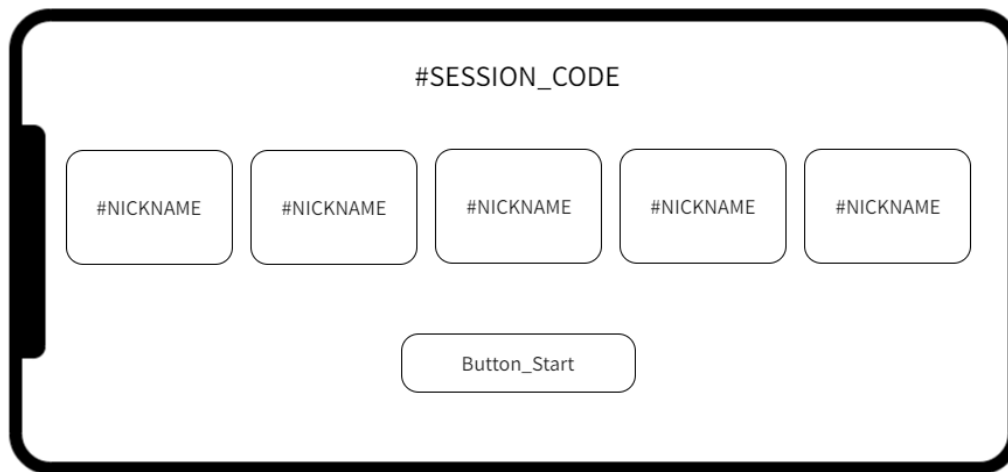


Рисунок 4.4 – Вікно перегляду лобі, де кнопка “Button_Start” відповідає за початок гри і доступна для взаємодії лише власнику лобі (хосту)

4.1.3.2 Інтерфейс під час процесу гри

- Вікно інтерфейсу на момент кидання кубика (Рисунок 4.5);
- Вікно інтерфейсу з інформацією у випадку потрапляння гравця на поле шанс монополії (Рисунок 4.6);
- Вікно інтерфейсу на момент потрапляння гравця на вільне поле, надається можливість придбати, або відмовитися (Рисунок 4.7);
- Вікно інтерфейсу для ініціалізації процесу обміну, яке викликається через натискання на гравця, з яким планується обмін (Рисунок 4.8);
- Вікно обміну між гравцями (Рисунок 4.9);
- Вікно поля (Рисунок 4.10);
- Вікно покращення поля (Рисунок 4.11);
- Вікно попередження про неправильні дії (Рисунок 4.12);
- Вигляд поля монополії, де зірка відповідає рівню побудованої монополії (Рисунок 4.13)

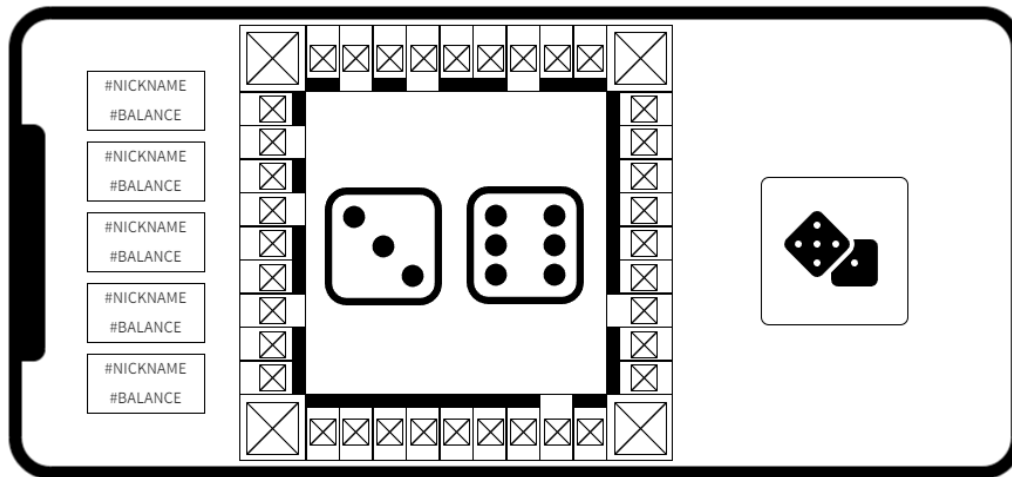


Рисунок 4.5 – Стан інтерфейсу на момент черги гравця, де “Button_Roll” відповідає за кидання кубика

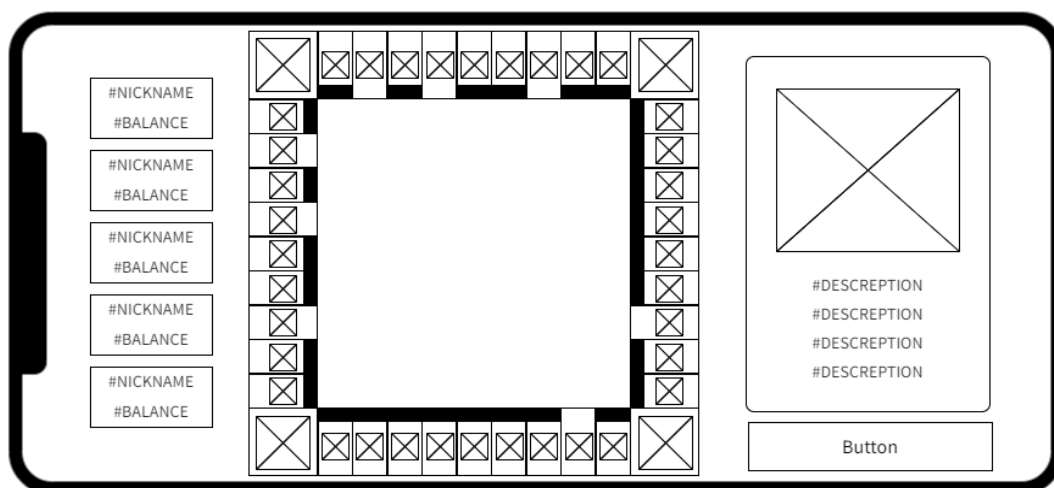


Рисунок 4.6 – Стан інтерфейсу на момент потрапляння гравця на поле шансу, де кнопка “Button” відповідає за підтвердження взаємодії з ним

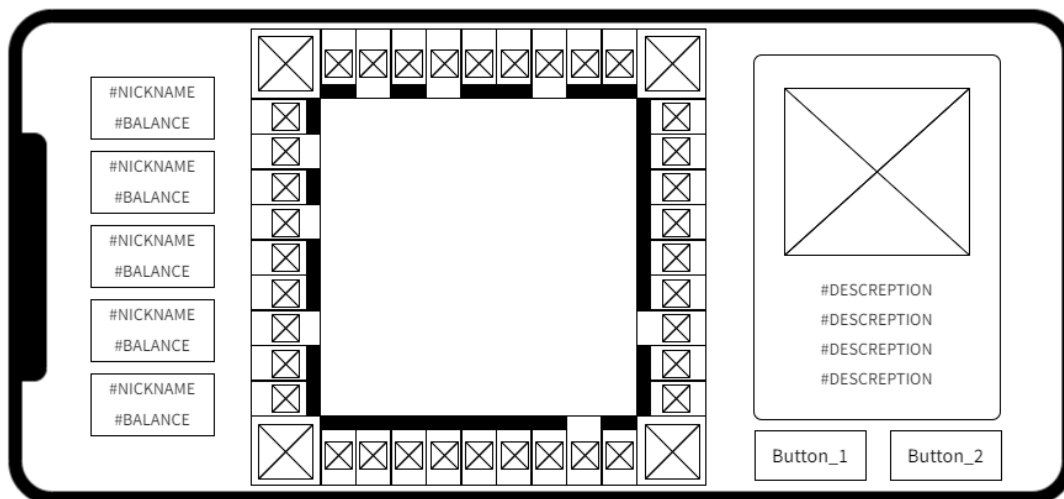


Рисунок 4.7 – Вікно інтерфейсу для взаємодії з полем, де “Button_2” відповідає за купівлю поля, а “Button_1” дозволяє відмовитися від пропозиції

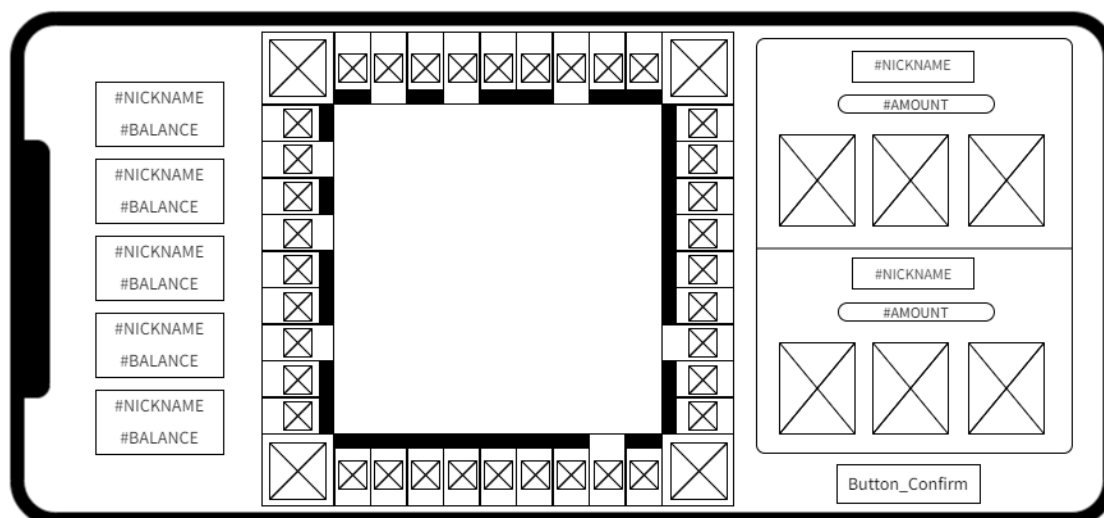


Рисунок 4.8 – Вікно ініціалізації процесу обміну, де “Button_Confirm” відповідає за підтвердження пропозиції обміну, значення у поле “Amount” вводиться від користувача за допомогою клавіатури

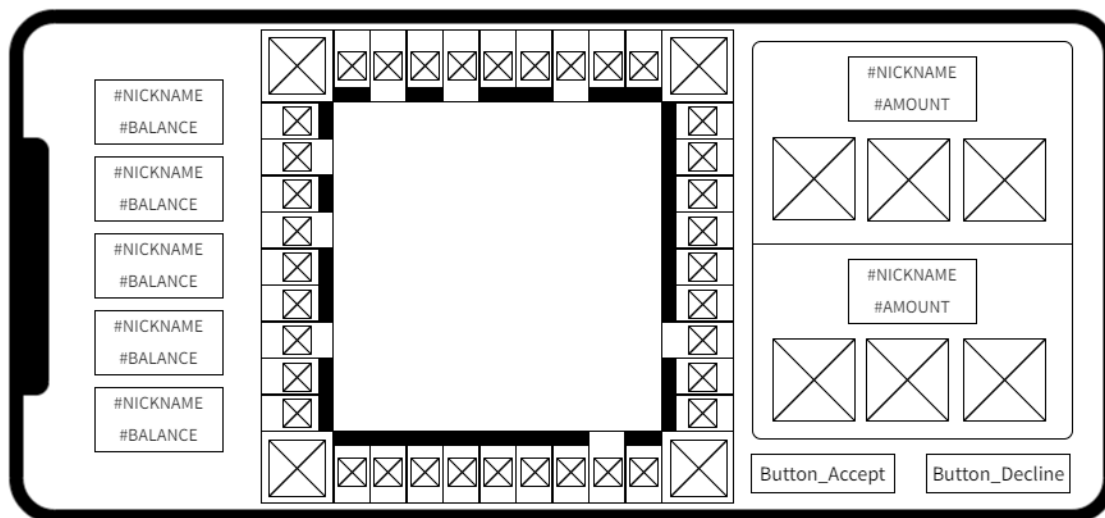


Рисунок 4.9 – Вікно обміну між гравцями, де “Button_Accept” відповідає за прийняття умов обміну, “Button_Decline” відповідає за відмову від пропозиції

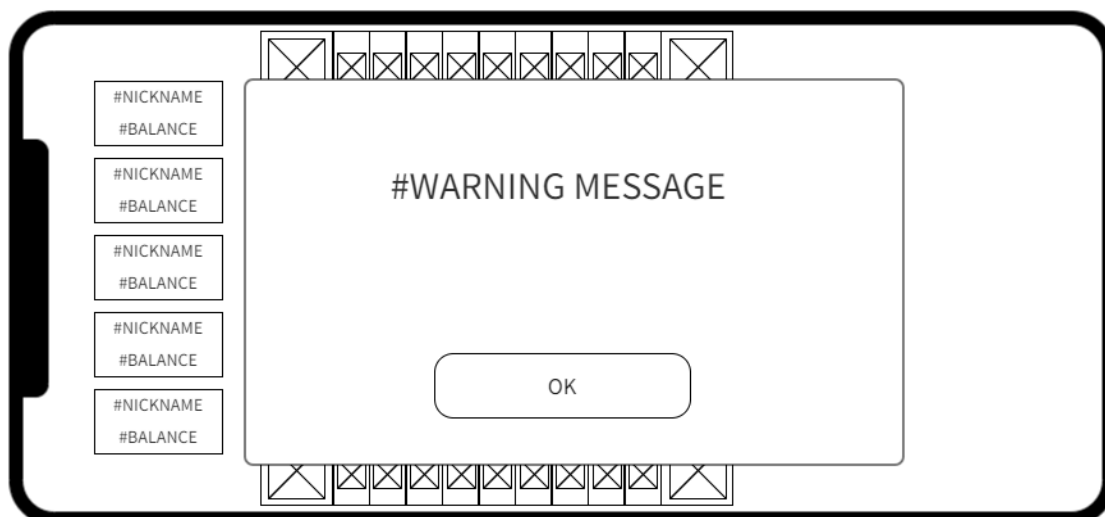


Рисунок 4.10 – Вікно повідомлення попередження про некоректні дії

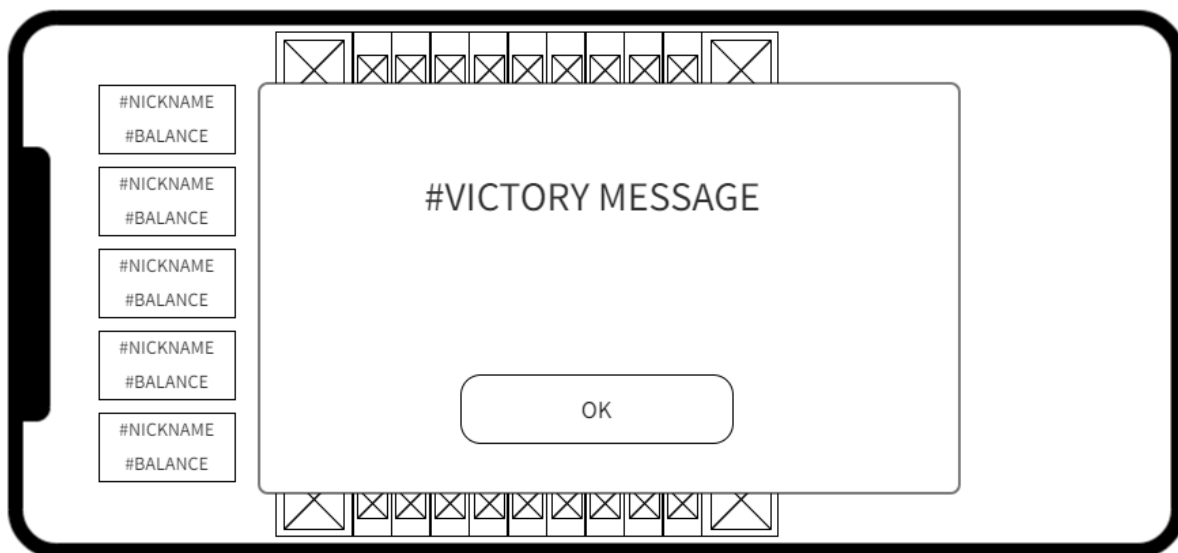


Рисунок 4.11 – Вікно повідомлення про перемогу

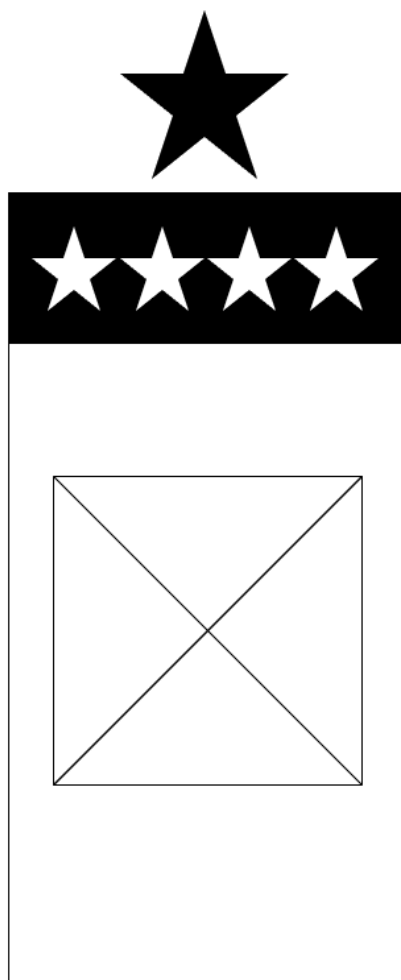


Рисунок 4.12 – Поле на дошці монополії, де кожна зірка відповідає рівню монополії

4.1.2 Для користувача:

- Під'єднання до ігрової кімнати з іншими гравцями;
- Можливість кидання кубиків для пересування ігровим полем, яка з'являється, коли настає черга відповідного гравця;
- Можливість купівлі поля у випадку знаходження на них та якщо вони вже не зайняті, або виставлення відповідного поля на аукціон;
- Можливість проводити обміни;
- Можливість закласти за умови, що на ньому немає побудованих покращень, оскільки у цьому випадку можна лише продавати покращення;
- Можливість розбудови монополії за умови зібраної відповідної монополії, підвищуючи вартість перебування інших гравців на цьому полі.

4.1.3 Для адміністратора системи:

Вимоги до адміністратора системи не висуваються.

4.1.4 Додаткові вимоги:

- Обмін даними у P2P-мережі між клієнтами відбувається завдяки Unity Transport Layer (дані серіалізуються та десеріалізуються у вигляді protobuf, отримані бінарні значення передаються мережею завдяки UDP).

4.2 Вимоги до надійності

- Передбачити контроль введення інформації та захист від некоректних дій користувача;
- Передбачити стабільність багатокористувацької сесії та порядок дій у випадку втрати інтернет-з'єднання у користувачів.

4.3 Умови експлуатації

Умови експлуатації згідно СанПін 2.2.2.542 – 96.

4.3.1 Вид обслуговування

Вимоги до виду обслуговування не висуваються.

4.3.2 Обслуговуючий персонал

Вимоги до обслуговуючого персоналу не висуваються.

4.4 Вимоги до складу і параметрів технічних засобів

Програмне забезпечення повинно функціонувати на мобільних пристроях з ARM архітектурою для смартфонів.

Мінімальна конфігурація технічних засобів:

- тип процесору: Qualcomm Snapdragon 400;
- об'єм ОЗП: 1 Гб;
- підключення до мережі Інтернет зі швидкість від 5 мегабіт;

Рекомендована конфігурація технічних засобів:

- тип процесору: Qualcomm Snapdragon 8 Gen 2;
- об'єм ОЗП: 2 Гб;
- підключення до мережі Інтернет зі швидкість від 15 мегабіт;

4.5 Вимоги до інформаційної та програмної сумісності

Програмне забезпечення повинно працювати під управлінням операційних систем Android.

4.5.1 Вимоги до вхідних даних

Вимоги до вхідних даних не висуваються.

4.5.2 Вимоги до вихідних даних

Вимоги до вихідних даних не висуваються.

4.5.3 Вимоги до мови розробки

Розробку виконати на мові програмування C#.

4.5.4 Вимоги до середовища розробки

Розробку виконати на платформі Windows 11 з використанням Unity 2023.3.7f1 та Visual Studio Community 2022.

4.5.5 Вимоги до представленню вихідних кодів

Вихідний код програми має бути представлений у вигляді Unity-скриптів.

4.6 Вимоги до маркування та пакування

Вимоги до маркування та пакування не висуваються.

4.7 Вимоги до транспортування та зберігання

Вимоги до транспортування та зберігання не висуваються.

4.8 Спеціальні вимоги

Згенерувати інсталяційну версію програмного забезпечення.

5 ВИМОГИ ДО ПРОГРАМНОЇ ДОКУМЕНТАЦІЇ

5.1 Попередній склад програмної документації

У склад супроводжувальної документації повинні входити наступні документи на аркушах формату А4:

- пояснювальна записка;
- технічне завдання;
- керівництво користувача;
- текст програми.

Графічна частина повинна бути виконана на аркушах формату А3 та містити наступні документи:

- схема структурна програмного забезпечення;
- схема структурна варіантів використання;
- схеми взаємодії об'єктів, об'єктна декомпозиція;
- креслення вигляду екранних форм.

5.2 Спеціальні вимоги до програмної документації

Програмні модулі, котрі розробляються, повинні бути задокументовані, тобто тексти програм повинні містити всі необхідні коментарі.

6 СТАДІЇ І ЕТАПИ РОЗРОБКИ

№	Назва етапу	Строк	Звітність
1.	Вивчення літератури за тематикою роботи	06.10	
2.	Розробка технічного завдання	07.10	Технічне завдання
3.	Аналіз вимог та уточнення специфікацій	08.10	Специфікації програмного забезпечення
4.	Проектування структури програмного забезпечення, проектування компонентів	15.11	Схема структурна програмного забезпечення та специфікація компонентів (діаграма класів, схема алгоритму)
5.	Програмна реалізація програмного забезпечення	07.01	Тексти програмного забезпечення
6.	Тестування програмного забезпечення	12.01	Тести, результати тестування
7.	Розробка матеріалів текстової частини роботи	13.01	Пояснювальна записка
8.	Розробка матеріалів графічної частини роботи	14.01	Графічний матеріал проекту
9.	Оформлення технічної документації роботи	15.01	Технічна документація

7 ПОРЯДОК КОНТРОЛЮ ТА ПРИЙМАННЯ

Тестування розробленого програмного продукту виконується відповідно до “Програми та методики тестування”.

АНОТАЦІЯ

Структура та обсяг роботи. Пояснювальна записка курсової роботи складається з 4 розділів, містить 29 рисунків, 42 таблиці, 25 джерел.

Мета. Розробка ставить перед собою досягнення наступних цілей:

- Реалізація класичної настільної гри «Монополія» на мобільному пристрої з операційною системою Android;
- Покращеного користувацького графічного інтерфейсу у порівнянні з існуючими аналогами, що забезпечить покращений користувацький досвід;
- Зміна ігрових правил та балансу з метою темпу гри для отримання більш швидких ігрових сесій у порівнянні з класичним варіантом.

У розділі аналізу вимог було встановлено конкретні функціональні, нефункціональні вимоги, які повинні бути враховані під час розробки програмного забезпечення. Ці вимоги визначають необхідні характеристики та функціональність, що є ключовими для її ефективного використання кінцевого продукту.

У розділі моделювання програмного забезпечення було розглянуто архітектуру розроблюваного мобільного застосунку, включаючи реалізовані програмні структури та сутності. Також були описані технічні особливості реалізації розробленого програмного продукту.

У розділі аналізу якості були визначені методики тестування та розписані основні тест-кейси, а також представлені результати стану системи після проведення тестування.

У розділі впровадження та супроводу було розглянуто процес розповсюдження гри та отримання нових версій гри для користувачів.

КЛЮЧОВІ СЛОВА: ГЕЙМДЕВ, ВІДЕОГРА, СМАРТФОН, НЕТКОД, UNITY, MONOPOLY, ANDROID.

Пояснювальна записка до курсової роботи

на тему: Гра для мобільного пристрою «Монополія»

КПІ.ІП-1402.045490.02.81

Київ – 2024

ЗМІСТ

ПЕРЕЛІК УМОВНИХ ПОЗНАЧЕНЬ.....	4
ВСТУП.....	5
1 АНАЛІЗ ВИМОГ ДО ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ.....	6
1.1 Загальні положення.....	6
1.2 Змістовний опис і аналіз предметної області.....	9
1.3 Аналіз існуючих технологій та успішних ІТ-проектів.....	10
1.3.1 Аналіз відомих алгоритмічних та технічних рішень.....	10
1.3.2 Аналіз допоміжних програмних засобів та засобів розробки.....	15
1.3.3 Аналіз відомих програмних продуктів.....	18
1.4 Аналіз вимог до програмного забезпечення.....	21
1.4.1 Розроблення функціональних вимог.....	29
1.4.2 Розроблення нефункціональних вимог.....	33
1.5 Постановка задачі.....	34
Висновки до розділу.....	35
2 МОДЕЛЮВАННЯ ТА КОНСТРУЮВАННЯ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ.....	36
2.1 Моделювання та аналіз програмного забезпечення.....	36
2.2 Архітектура програмного забезпечення.....	39
2.3 Конструювання програмного забезпечення.....	41
2.4 Аналіз безпеки даних.....	46
Висновки до розділу.....	47
3 АНАЛІЗ ЯКОСТІ ТА ТЕСТУВАННЯ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ....	48
3.1 Аналіз якості ПЗ.....	48
3.2 Опис процесів тестування.....	50
3.3 Опис контрольного прикладу.....	58
Висновки до розділу.....	67

	3
4 ВПРОВАДЖЕННЯ ТА СУПРОВІД ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ.....	68
4.1 Розгортання програмного забезпечення.....	68
4.2 Підтримка програмного забезпечення.....	69
Висновки до розділу.....	69
ВИСНОВКИ.....	70
СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ.....	71

ПЕРЕЛІК УМОВНИХ ПОЗНАЧЕНЬ

IDE	– Integrated Development Environment – інтегроване середовище розробки.
RPC	– Remote Procedure Call – віддалений виклик процедури.
UDP	– User Datagram Protocol – протокол дейтаграми користувача.
DTLS	– Datagram Transport Layer Security – дейтаграми транспортного рівня безпеки.
OSI	– Open System Interconnection model – середовище зв'язку відкритих систем.
P2P	– Peer-to-Peer – однорангова мережа.
API	– Application Programming Interface – прикладний програмний Інтерфейс.

ВСТУП

У сучасному інформаційному світі ринок відеоігор з року в рік приваблює все більше нових гравців, а відповідно, й інвестицій. Однією з основних причин розвитку цього сегменту бізнесу є зростання загального рівня життя, а отже, й доступності нових масових пристроїв, таких як мобільні телефони для різних користувачів з усього світу. Завдяки цьому розробники отримують можливість створювати контент, що здатний охопити та задовольнити запити навіть найбільш вибагливих до платформи гравців.

Монополія – одна з найпопулярніших настільних ігор у світі, яка завоювала сотні мільйонів шанувальників з усього світу. Актуальність створення мобільної гри з багатокористувацьким режимом, яка є реалізацією монополії, полягає у відповідності сучасним світовим тенденціям, оскільки це рішення дозволяє гравцям з різних куточків світу грати разом, незалежно від їх місця розташування. Мобільна монополія з багатокористувацьким режимом має ряд переваг перед традиційною настільною грою, серед яких основними є простота та доступність в будь-який час і в будь-якому місці.

У даній курсовій роботі буде розглянуто створення мобільної версії монополії з багатокористувацьким режимом, проведений аналіз існуючих рішень, та обґрунтована необхідність розробки нової мобільної монополії, яка відповідає вимогам сучасності та враховує актуальні тенденції в області розробки відеоігор.

1 АНАЛІЗ ВИМОГ ДО ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

1.1 Загальні положення

Ігрова індустрія – це напрямок інформаційних технологій, який займається розробкою, виданням та розповсюдженням відеоігор. Вона є однією з найбільш динамічних та прибуткових напрямів ведення бізнесу. Сфера розробки відеоігор є однією з найбільших галузей розважальної індустрії, її загальні річні прибутки перевершують доходи таких сегментів ринку, як кіно та серіали. Далеко не в останню чергу такі високі показники забезпеченні розмаїттям напрямків цієї галузі, серед яких окремо можна визначити: сама розробка відеоігор під різні платформи, їх видавництво, кіберспорт та багато інших. Такий різноманітний спектр напрямків свідчить про величезний вплив цієї індустрії на сучасну глобальну культуру [2].

Історія розробки відеоігор розпочинається з середини 20-го століття та визначається важливими трансформаціями, які охоплюють усі періоди її існування – від початку створення цієї галузі й до сучасності. Починаючи з перших відеоігор, сфера швидко розвивалася, зазнаючи вагомих змін на кожному етапі свого розвитку [1].

Перші експерименти зі створення відеоігор відбувалися завдяки окремим ентузіастам та науково-дослідницьким лабораторіям при університетах у 1950-1960-х роках, які відзначилися створенням гри під назвою «Spacewar!», яка була реалізована в 1962 році та вважається однією з перших повноцінних на той час відеоігор, яка і визначила початок цієї індустрії [3].

У 1970-80-х роках з'явилися перші аркадні ігрові автомати, а разом із ними були створені перші ігри для нових платформ, такі як «Pong» від компанії «Atari» та інші, що визначили новий етап в історії відеоігор. Втім, були всі шанси втрати цю галузь, оскільки у 1983 році світ сколихнула криза, що спричинила банкрутство багатьох компаній та надзвичайне падіння прибутків, що було пов'язане з перенасиченістю ринку іграми, переважно дуже середньої якості. Але можна сказати, що компанія «Nintendo» вийшла з неї

переможцем, саме вона, приблизно через десятиліття, у 1980-1990-х розробила перший виключно ігровий пристрій під назвою «Nintendo Entertainment System», який набув небаченої до цього популярності серед користувачів та ознаменував початок активного розповсюдження нових виключно ігрових пристроїв – консолей. Цей важливий етап не лише вплинув на подальший розвиток гральних приладів, але й визначив стандарти ігрової індустрії завдяки всесвітньовідомим класичним відеоіграм, які стали культовими і сформували основи сучасного ігрової культури [4].

З появою персональних комп'ютерів в 1980-90-х роках розробка ігор стала більш доступною для самостійних незалежних розробників, сприяючи зростанню їх кількості та поширенню ігор на різних платформах. Цей процес також сприяв і збільшенню кількості потенційних гравців, через появу нового масового багатозадачного пристрою. Також у 1990-2000-х роках ігрова індустрія пережила перехід до 3D-графіки, що зробило ігри більш реалістичними та вимогливими до заліза. Окремо варто відзначити появу доступного інтернету, який дав можливість легше розповсюджувати свої розробки та завдяки якому з'явилися перспективи для розвитку багатокористувацьких онлайн-проектів, де група друзів, чи велика спільнота не пов'язаних між собою людей, могли разом проводити час [2].

Новим витком розвитку стала поява на початку нового тисячоліття ще однієї платформи – смартфонів. Її виникнення також призвело до мобілізації ресурсів багатьох студій та незалежних розробників з метою зайняти нову нішу. І це не дивно, оскільки вкладання сил та коштів в неї дає можливість охопити сотні мільйонів девайсів та їхніх користувачів через її головну привабливість – не потрібно купляти жодних додаткових приладів для ігрового процесу, гравець може долучатися будь-де та будь-коли до улюбленої гри, а вже згаданий раніше розвиток Інтернету лише полегшив отримання доступу до більш широкої бібліотеки електронних розваг та, як не дивно, прибутків компаніям, які отримали додаткові можливості для монетизації своїх проектів [1].

Розробка відеогри – це складний і тривалий процес, який часто варіюється від проєкту до проєкту, але першим етапом потрібно правильно проаналізувати вимоги, які пов'язані з поточним проєктом. Під час цього аналізу необхідно визначити різноманітні аспекти, включаючи функціональні та нефункціональні аспекти гри, такі як ігролад, візуальна та звукова складові. Функціональними визначимо основні ігроладні механіки, кількість гравців тощо. Нефункціональними назвемо продуктивність самого проєкту, безпека даних, сумісність з різними платформами. До того ж, необхідно проаналізувати все з точки зору ігрового дизайну з метою забезпечення контролю рівня складності, взаємодії гравець-гра та гравець-гравець, візуальну та аудіо складову, які відповідатимуть загальному духу та вигляду продукту, оскільки все це впливатиме на ігровий досвід та подальше ставлення до гри у користувачів. З рештою, необхідне також тестування проєкту, аби кінцевий результат відповідав загальноприйнятим в індустрії нормам та забезпечив гідний рівень конкуренції [3].

Сучасні відеоігри різняться від своїх попередників, настільних ігор, але неможливо заперечувати наявність спільних ідей, концептів, ігrolаних особливостей тощо. Як мінімум, більшість сучасних виключно ігрових жанрів мають попередників саме серед настільних ігор, наприклад, прабатьком всіх комп'ютерних рольових ігор є настільна рольова гра «Dungeons & Dragons», а стратегічні ігри завдячують таким представникам як шахи, «Монополія» та інші. Тобто можна стверджувати, що самі відеоігри є еволюційним явищем, розвитком класичних розваг у більш нову, відповідну до часу форму [1].

Всесвітньо відома настільна гра «Монополія» була випущена в світ у 1935 році. Вона являє собою класичну покрокову стратегію з економічним ухилом і, фактично, знайомить з основами капіталізму, а також дає можливість через гіперболізовані його прояви цікаво та захоплююче провести час. Відповідно, метою цієї гри є змусити збанкрутувати інших гравців, ставши таким чином монополістом, і досягається це купівлею, продажем та, звісно, перемовинами.

1.2 Змістовний опис і аналіз предметної області

У світі постійного технологічного розвитку та високих вимог до якості програмного забезпечення, вивчення процесу використання знань предметної області в контексті розвитку ІТ-технологій стає надзвичайно важливим завданням.

Однією з ключових проблем у створених реалізаціях «Монополії» з багатокористувацьким режимом є перенавантажений інтерфейс, який ускладнює сприйняття гравцями гри та робить її менш зручною та зрозумілою для використання. Аналіз інтерфейсу дозволив ідентифікувати зайві компоненти та розробити план їх оптимізації, тим самим, забезпечивши більш зрозумілий та очевидний геймплей без використання зайвих непотрібних елементів інтерфейсу, кількість елементів та складність якого може перешкоджати певній частині аудиторії брати участь в ігровому процесі.

Іншою проблемою є відсутність реалізації «Монополії» у 2D, що може бути не зовсім зручним для окремих користувачів через підвищені системні вимоги у аналогах з 3D графікою та великою кількістю візуальних ефектів, які не несуть жодного сеансового навантаження, а часто відволікають від головної мети гри. До того ж, проблема системних вимог для мобільного пристрою може бути доволі вагома через потенційний виграш у зменшенні енергоспоживання пристрою.

Ще одним важливим недоліком є занадто повільний темп гри, викликаний невисокими цінами на нерухомість та низькими штрафами що унеможливорює проведення довгих ігрових сеансів, що особливо актуально для мобільного пристрою через відсутність постійного джерела живлення та необхідності постійної підзарядки. Істотне завищення цін та накладання обмежень на обмін, одночасну кількість лотів під час процесу обміну полів суттєво спрощує процес стратегічного планування для гравців та забезпечує більш швидкі ігрові сеанси.

1.3 Аналіз існуючих технологій та успішних IT-проектів

Проаналізуємо відоме на сьогодні алгоритмічне забезпечення у даній області та технічні рішення, що допоможуть у реалізації багатокористувацької мобільної Монополії. Далі будуть розглянуті допоміжні програмні засоби, засоби розробки та готові програмні рішення.

1.3.1 Аналіз відомих алгоритмічних та технічних рішень

У даній розробці немає необхідності застосовувати спеціальні обчислювальні, або інші види алгоритмів.

Оскільки у створюваному програмного забезпеченні наявний багатокористувацький режим, то виникає необхідність вирішення типу підтримуваної мережевої архітектури у проекті. Мережеві моделі для багатокористувацьких ігор описують архітектуру та дизайн, які використовуються для встановлення зв'язку між джерелом інформації та місцем призначення – наприклад, між клієнтом і сервером або між гравцями у одноранговій мережі. Серед доступних варіантів можна виокремити наступні: клієнт-серверна архітектура, однорангова централізована та однорангова децентралізована [7].

Клієнт-серверна модель з виділеним сервером – це тип мережі, у якій присутній 1 централізований сервер, який надсилає дані кінцевим користувачам, тобто клієнтами, які отримують цю інформацію, її обробляють та діють відповідно до неї. Клієнти також можуть надсилати дані на сервер. Ця модель забезпечує ефективний обмін інформацією між сервером та клієнтами, дозволяючи швидку передачу даних і забезпечуючи централізоване управління ресурсами. Важливим аспектом є можливість реалізації різних функцій на сервері, що полегшує синхронізацію та узгодженість в роботі всієї мережі. Крім того, використання цієї моделі може спрощувати моніторинг та обслуговування мережі завдяки її централізованій структурі [24].

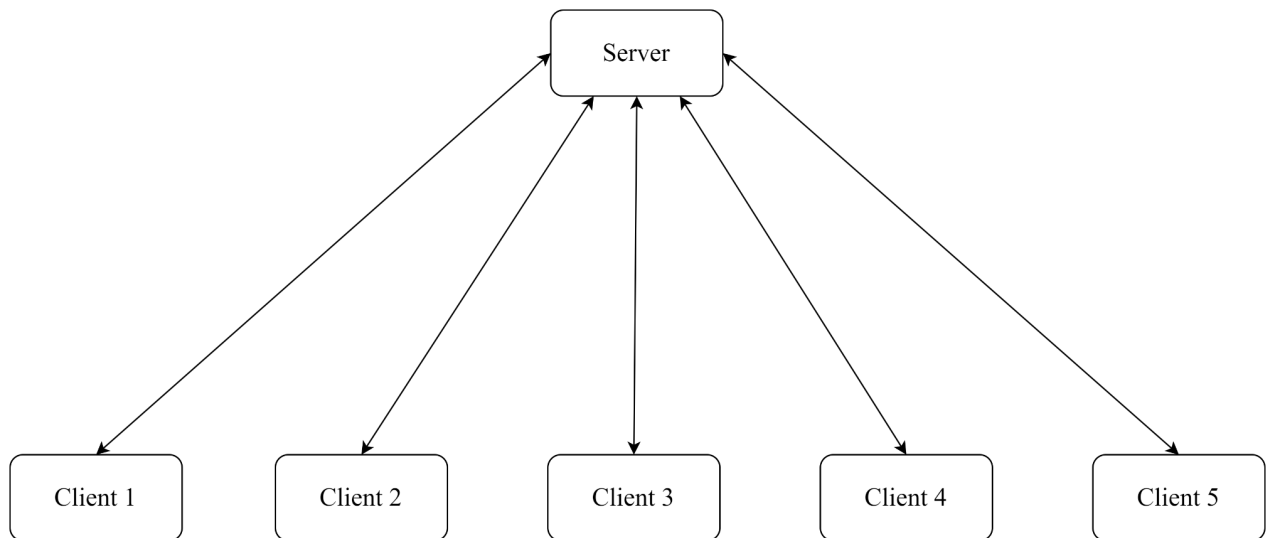


Рисунок 1.1 – Схема клієнт-серверної архітектури

Переваги клієнт-серверної архітектури:

- Централізоване управління грою, що полегшує синхронізацію даних між гравцями та забезпечує загальну стабільність гри;
- Сервер може виконувати обробку важких розрахунків, зменшуючи обсяг обчислювального завдання на стороні клієнта;
- Ця архітектура сприяє централізованому управлінню безпекою та мінімізує можливість шахрайства.

Недоліки клієнт-серверної архітектури:

- Велика фізична відстань між сервером і гравцями може призводити до високої затримки;
- Підтримка централізованого сервера вимагає значних фінансових витрат. Великий обсяг трафіку, потреби у потужності обчислень та безпеці вимагають інфраструктури високого класу, що може бути дорогим завданням для розробників;
- При збільшенні кількості гравців можуть виникати проблеми з масштабованістю серверів. Додавання нових потужностей може призводити до зменшення продуктивності та якості обслуговування існуючих.

Вищеописаний клієнт-серверний тип мережі не є оптимальним варіантом для розробки через потенційну дорогу вартість обслуговування, а також можливу проблему довгої обробки запитів.

Децентралізований peer-to-peer, P2P (з англ. – рівний до рівного) – варіант однорангової архітектури системи, в основі якої стоїть мережа рівноправних вузлів. Існує декілька видів однорангових мереж, серед яких пірінгова модель з'єднує всіх гравців між собою безпосередньо, тобто без сервера або виділеного хоста. На відміну від клієнт-серверних моделей, які вимагають виділеного сервера, за яким залишається вирішальне слово щодо загального стану гри, в P2P-моделі пристрій кожного гравця відповідає за ігрову логіку та взаємодію з іншими учасниками гри [8].

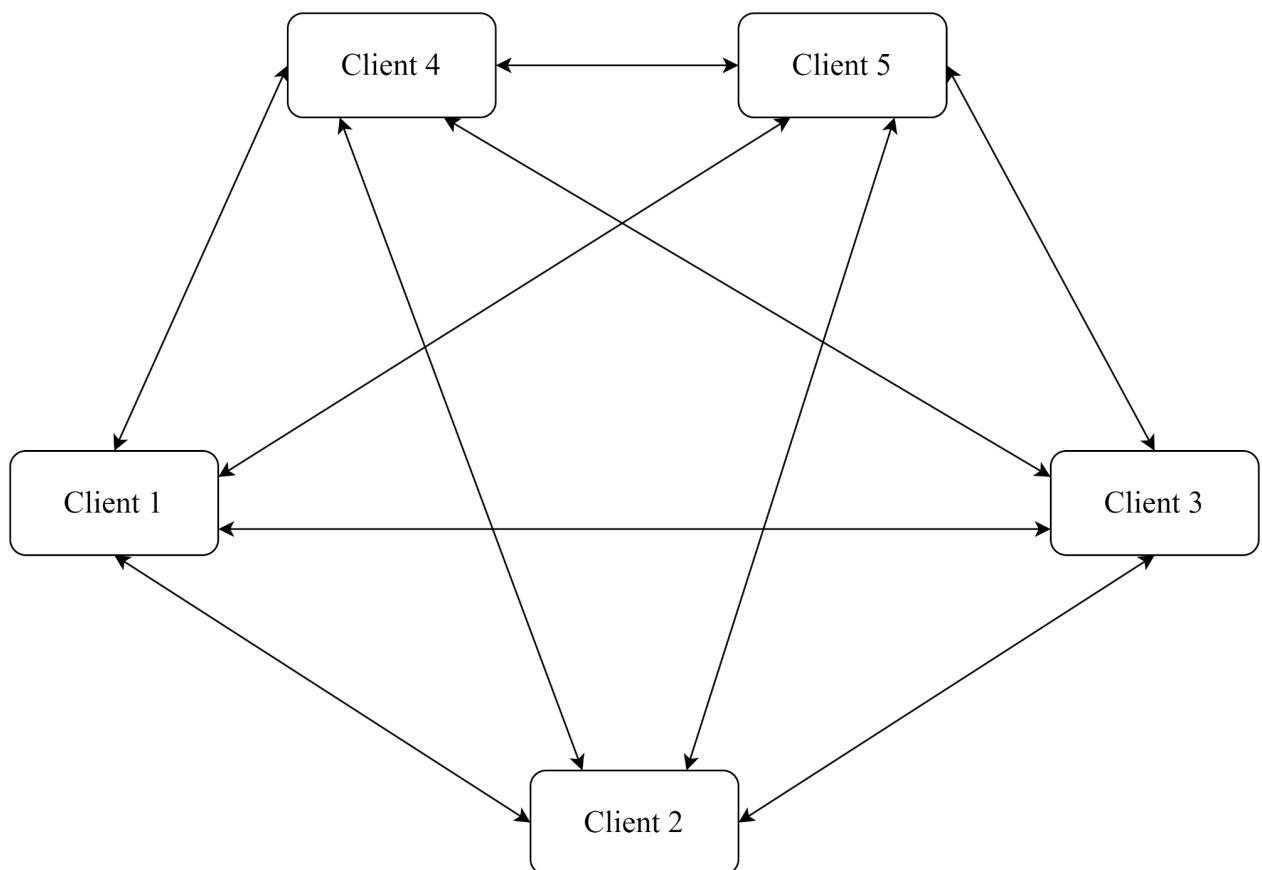


Рисунок 1.2 – Схема однорангової децентралізованої мережі

Переваги децентралізованої однорангової архітектури:

- Відсутність центрального сервера зменшує навантаження та залежність від централізованого вузла, що також забезпечує більшу стійкість до відмов;
- Рівномірне розподілення обробки може знизити затримки, оскільки кожен вузол може служити джерелом даних та обробляти їх;
- Як правило, є простішою у встановленні, оскільки не потрібно налаштовувати централізований сервер.

Недоліки децентралізованої однорангової архітектури:

- Складність масштабування через необхідність постійної синхронізації поточного стану гри між усіма її учасниками, оскільки вимагає ефективної координації та синхронізації між вузлами;
- Підвищена вразливість до протиправних дій, оскільки вся інформація про всі підключені клієнти у мережі відома іншим клієнтами, порівняно простий доступ до даних гри, що полегшує потенційне шахрайство;
- Може виникати затримка та нестабільність з'єднання через безпосередній обмін даними між вузлами;
- Можлива втрата даних через відсутність централізованої точки зберігання та обробки.

Вищеописана однорангова децентралізована архітектура мережі не є оптимальним варіантом через потенційні проблеми десинхронізації, відсутності центрального вузла, який здатен забезпечити загальну синхронізацію, а також загальну складність розробки та підвищеної складності масштабованості.

Централізована peer-to-peer, P2P (з англ. – рівний до рівного) мережа – це варіант однорангової архітектури, в якій забезпечується певний рівень централізації та координації за рахунок наявності центрального вузла, який є таким самим клієнтом-вузлом системи, як і інші гравці. Такий центральний вузол має назву host (з англ. – господар) – пристрій, який ініціює та контролює багатокористувацьку ігрову сесію. Основне визначення хоста полягає в тому, що це та сторона, яка виступає одночасно як гравець та центральний сервер [8].

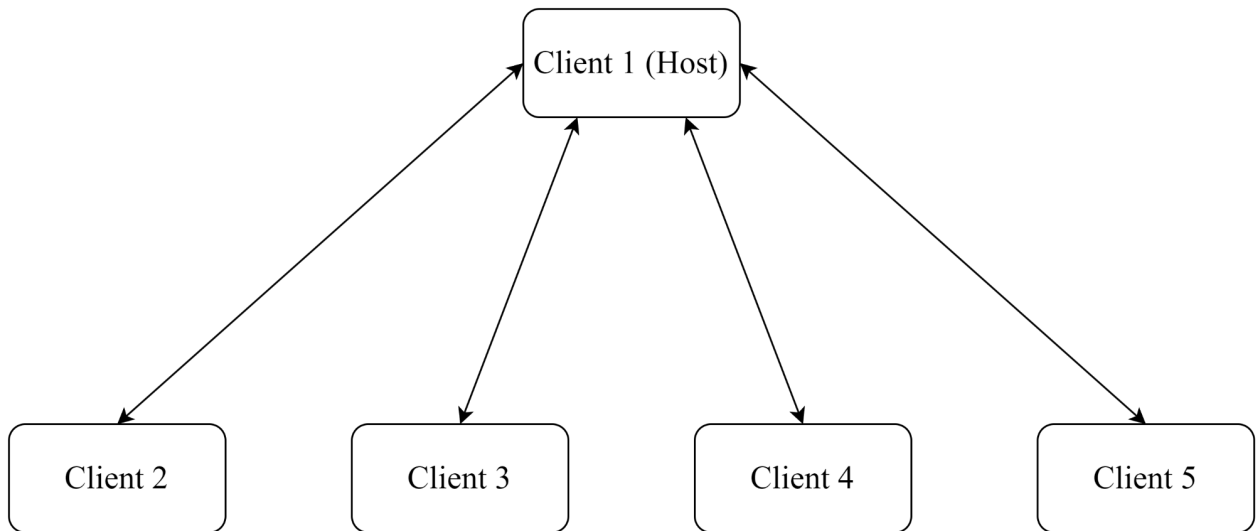


Рисунок 1.3 – Схема однорангової централізованої мережі

Переваги централізована однорангової архітектури:

- Простота управління станом гри у порівнянні з децентралізованою одноранговою мережею через наявність хоста;
- Централізоване управління може полегшити впровадження безпекових заходів, які ускладнюють потенційну можливість до шахрування, а також через те, що звичайні клієнти не мають всієї повноти інформації про інших клієнтів у мережі;
- Ефективний розподіл обчислювальних ресурсів, оскільки потрібні обчислення можуть відбуватися лише на 1 клієнті, який потім може поділитися результатами своєї роботи.

Недоліки централізованої однорангової архітектури:

- Якщо центральний вузол виявився нестійким, або вразливим, це може призвести до збоїв всієї системи. Таким чином, якщо центральний вузол перестане працювати, то це викличе збій у всієї ігрової сесії;
- У випадку масштабованості цей процес проходитиме простіше на відміну від децентралізованої мережі, проте на якомусь етапі наявність єдиного центрального вузла, який ще й паралельно виконує роль клієнта може

стати обмежуючим фактором виключно через складність виконуваних обчислень та завдань, які покладаються на сервер.

Таким чином, вищезгадана централізована однорангова мережа, незважаючи на певні недоліки є оптимальним рішенням для створюваної розробки, оскільки вона здатна забезпечити стабільну синхронізацію між клієнтами, завдяки відсутності необхідності підтримки виділених серверів, підвищеній безпеці у порівнянні з децентралізованою одноранговою.

1.3.2 Аналіз допоміжних програмних засобів та засобів розробки

Для створення мобільної гри, яка буде підтримувати багатокористувацький режим необхідно вибрати відповідний фреймворк, який здатен забезпечити наступні вимоги:

- Простоту кросплатформної розробки;
- Наявність засобів для створення однорангової централізованої мережі;
- Присутність багатьох інструментів, які здатні допомогти у створенні відповідного програмного продукту.

Враховуючи наведені вимоги, вибір пав на один з актуальних ігрових рушіїв – Unity. Однією з ключових переваг цього рушія – це легка кросплатформенна розробка без необхідності суттєвої додаткової оптимізації у випадку створення продукту під вибрану платформу від початку. Ще однією ключовою його перевагою є наявність багатого вибору бібліотек та засобів для створення багатокористувацької гри, як офіційні, так і від сторонніх розробників з відкритим вихідним кодом. До того ж, Unity має велику кількість інструментів, які пришвидшують розробку подібних рішень, що особливо актуально для незалежних самостійних розробників [13].

Оскільки розробка програмного продукту виконуватиметься за допомогою Unity, то було обрано мову програмування пав на C#, оскільки це єдина підтримувана мова розробки для даного рушія.

Вибір інтегрованого середовища розробки пав на Visual Studio, оскільки вона має покращену інтеграцію з Unity, завдяки можливості хуку до процесу гри у реальному часі та дебагу вихідного коду прямо під час гри у неї.

Стосовно вибору бібліотеки для написання неткоду, то Unity має багато різних варіантів, як офіційні, так і від сторонніх розробників, найвідоміші рішення на даний момент: Mirror, Fish-Net, NGO (Netcode for GameObjects).

Mirror – відкрита бібліотека для створення додатків, які підтримують інтеграцію з мережевими службами та підсистемами. Ця бібліотека розроблена для спрощення створення багатокористувацьких ігор та додатків [19].

Переваги бібліотеки Mirror:

- Простий та зрозумілий API, який надає високий рівень абстракції від безпосереднього оперування та відправкою даних по мережі на низькому рівні;
- Велика активна спільнота користувачів і гарна документація;
- Наявна вбудована підтримка для роботи з серверами, що дає можливість створювати ігри з різної мережевою архітектурою.

Недоліки бібліотеки Mirror:

- Менш розвинений функціонал у порівнянні з іншими рішеннями за рахунок відсутності просунутої системи лобі;
- Відсутність проксі;
- Обмежена інтеграція з іншими наявними сервісами Unity.

Fish-Net – це відкрита бібліотека для створення багатокористувацьких ігор та додатків на Unity. Бібліотека розроблена для спрощення створення мережових ігор, а також для надання високорівневого API, який абстрагує від низькорівневих деталей мережевого програмування [20].

Переваги бібліотеки Fish-net:

- Простий та зрозумілий API, який надає високий рівень абстракції від безпосереднього оперування та відправкою даних по мережі на низькому рівні;
- Наявність вбудованих компонентів для роботи з мережевими підключеннями, обміном даними та іншими аспектами мережевого програмування;
- Підтримка різних мережевих архітектур, включаючи P2P, клієнт-сервер та гібридні архітектури;
- Можливість масштабування ігор на велику кількість гравців.

Недоліки бібліотеки Fish-net:

- Бібліотека все ще перебуває в розробці, тому деякі функції можуть бути недоступні або мати обмежені можливості;
- Може бути складною для використання для початківців розробників.

NGO (Netcode for GameObjects) – це відкрита бібліотека для створення багатокористувацьких ігор та додатків на Unity. Бібліотека розроблена для спрощення створення мережевих ігор, а також для надання високорівневого API, який абстрагує від низькорівневих деталей мережевого програмування [15].

Переваги бібліотеки NGO:

- Офіційне рішення для багатокористувацьких ігор від Unity;
- Простий та зрозумілий API, який надає високий рівень абстракції від безпосереднього оперування та відправкою даних по мережі на низькому рівні;
- Наявність вбудованих компонентів для роботи з мережевими підключеннями, обміном даними та іншими аспектами мережевого програмування;
- Підтримка різних мережевих архітектур, включаючи P2P, клієнт-сервер та гібридні архітектури;

- Можливість масштабування ігор на велику кількість гравців;

Недоліки бібліотеки NGO:

- Бібліотека все ще перебуває в активній розробці, тому деякі функції можуть бути недоступні, мати обмежені можливості, або містити недоліки.

Бібліотекою для написання неткоду було обрано NGO (Netcode for GameObjects). Цей вибір був зроблений через широкий функціонал бібліотеки, який включає підтримку різних мережевих архітектур, масштабування на велику кількість гравців, а також через те, що NGO є офіційним рішенням від Unity, що гарантує його стабільну підтримку сумісність з боку розробників Unity [15].

1.3.3 Аналіз відомих програмних продуктів

Розглядаючи переваги та недоліки конкурентних продуктів можна визначити ключові аспекти, які можуть найбільше впливати на прийом ефективних рішень у сучасній сфері мобільних відеоігор.

Монополія – дуже відома настільна гра і тому має декілька створених програмних рішень на багатьох платформах, на мобільних пристроях найбільш популярні – це «Monopoly» від «Marmalade Game Studio» та «Monopoly GO» від «Scopely» [12].

«Monopoly» – це мобільна версія класичної настільної гри, розроблена студією «Marmalade Game Studio», гра вийшла 2016 року на iOS та Android [12].

Переваги «Monopoly» від «Marmalade Game Studio»:

- Графіка та анімація. «Monopoly» виконана у 3Д, має чудову графіку та анімацію, які роблять гру більш привабливою та захопливою;
- Можливість власних налаштувань правил гри.

Недоліки «Monopoly» від «Marmalade Game Studio»:

- Ця гра є реалізацією класичної монополії, а отже перенесла всі недоліки з настільної гри, а саме – довгу тривалість гри, нудні описи полів шанс, тощо.
- Наявність руйнуючих ігрових покупок у внутрішньому магазині, які можуть надати нечесну перевагу гравцям над своїми суперниками.

«Monopoly GO» – це мобільна версія класичної настільної гри, розроблена студією «Scopely», гра вийшла 2022 року на iOS та Android [12].

Переваги «Monopoly» від «Scopely»:

- Виконана у 3Д, має чудову графіку та анімацію, які роблять гру більш привабливою та захопливою.

Недоліки «Monopoly GO» від «Scopely»:

- Довга тривалість гри, ігрові сесії можуть бути досить тривалими, особливо якщо грати з великою кількістю гравців.
- Гра пропонує внутрішні покупки, які можуть надати нечесну перевагу гравцям над своїми суперниками;
- У деяких моментах ігровий інтерфейс занадто громіздкий та незрозумілий.

Для порівняння курсової роботи з аналогом можна скористатись таблицею 1.1.

Таблиця 1.1 – Порівняння з аналогом

Критерій порівняння	Курсачополія	Monopoly GO	Monopoly	Пояснення
Спосіб розповсюдження	Відкритий вихідний код	Безкоштовна базова версія, внутрішньоігрові покупки	Платна базова версія, внутрішньоігрові покупки	Спосіб отримання продукту користувачам

Продовження таблиці 1.1

Продуктивність	Гра виконана у 2Д, без додаткових візуальних ефектів	Гра виконана у 3Д, з використанням додаткових візуальних ефектів	Гра виконана у 3Д, з використанням додаткових візуальних ефектів	Продуктивність гри на нормальному рівні дає можливість користувачам з різним рівнем пристроїв доторкнутися до продукту
Зручність графічного користувацького інтерфейсу	Мінімальна кількість зайвих елементів, лише необхідний функціонал	Велика кількість вкладок з додатковим функціоналом, складність орієнтації для нових користувачів	Велика кількість вкладок з додатковим функціоналом, складність орієнтації для нових користувачів	Зручний користувацький інтерфейс дозволяє створити кращий ігровий досвід для користувачів
Середня тривалість гри	Швидкі ігрові сесії за рахунок змінених правил	Звичайні, довготривалі ігрові сесії	Звичайні, довготривалі ігрові сесії	Тривалість ігрових сесій є важливим аспектом, особливо, коли мова йде про мобільні пристрої

1.4 Аналіз вимог до програмного забезпечення

Мобільний застосунок, який розробляється, спрямований на створення реалізації мобільної гри «Монополія» з багатокористувацьким режимом до 5 гравців, забезпечуючи при цьому зручний графічний інтерфейс та покращені ігрові правила, які спрямовані на скорочення середнього часу ігрової сесії та покращення загального ігрового досвіду. Основна мета цього проекту полягає у реалізації мобільної гри, який надасть гравцям можливість зіграти у відому настільну гру, використовуючи свій мобільний пристрій.

Серед функціональних задач гравцям надані наступні можливості:

- Створювати власні лобі та бути хостом ігрової сесії, або приєднуватися до ігрових кімнат у ролі користувача-клієнта;
- Кидання кубика та пересування ігровим полем на такі кількість кроків, яке дорівнює числу, яке було отримано внаслідок випавшого значення;
- Можливість взаємодії з ігровими полями, а саме наступні варіанти: купити власність, яка нікому ще не належить, варіант закласти поле, яке належить гравцю та на якому ще не побудовано жодних покращень, у іншому випадку – надається можливість продати покращення поля, також надається можливість покращити поле, тим самим збільшивши ціну оренди за перебування інших гравців на обраному полі;
- Можливість проводити обміни з іншими гравцями у тому випадку, якщо варіант обміну задовольняє умови валідації;
- Здатність взаємодії з обмінами, а саме прийняти, або відхилити його.
- Можливість здатися, тим самим гравець лишається всіх своїх полів, статків і більше не приймає участь у грі.

Всі вище перераховані функціональні задачі можна побачити на діаграмі використання. Діаграму використання можна побачити на рисунку 1.2.

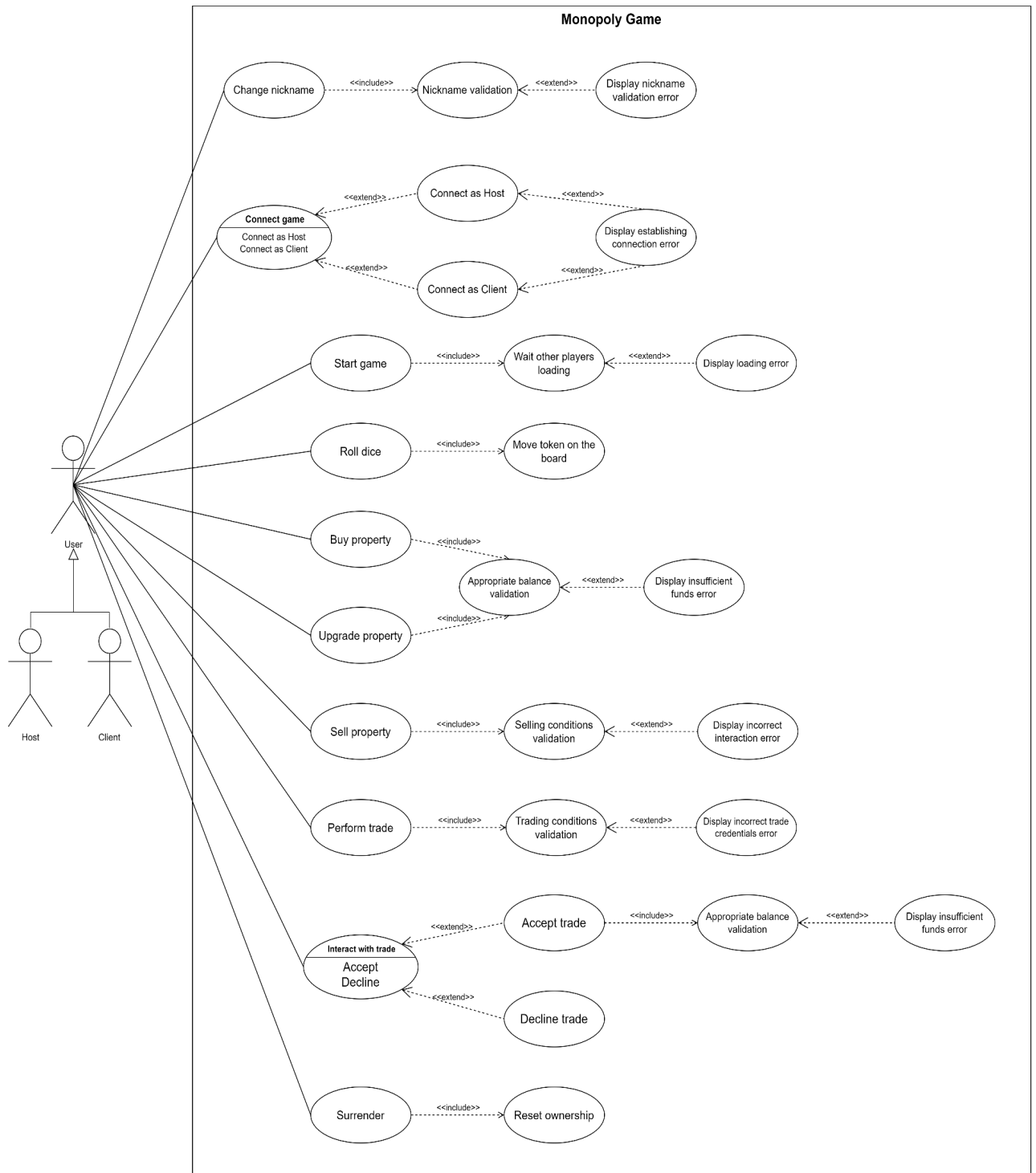


Рисунок 1.4 – Діаграма варіантів використання

В таблицях 1.2 – 1.13 наведені варіанти використання програмного забезпечення.

Таблиця 1.2 - Варіант використання UC-1

Use case name	Введення імені користувача
Use case ID	UC-01
Goals	Зміна ім'я гравця
Actors	Користувач
Trigger	Користувач бажає змінити своє ім'я
Pre-conditions	Щойно відкрита гра, користувач знаходиться у головному меню
Flow of Events	Користувач вводить своє ім'я у відповідне поле для вводу нікнейму і натискає на кнопку підключення до лобі як клієнт, або створення власного лобі
Extension	У випадку введення некоректного значення, то показується діалогове вікно з відповідним повідомленням про некоректно введений нікнейм
Post-Condition	Ініціалізація початку підключення до ігрової кімнати

Таблиця 1.3 - Варіант використання UC-2

Use case name	Створення ігрової кімнати
Use case ID	UC-02
Goals	Створення ігрового лобі
Actors	Користувач
Trigger	Користувач натиснув на кнопку створення гри
Pre-conditions	Введене ім'я користувача
Flow of Events	Користувач натискає на кнопку створення гри та, користувачу демонструється вікно з повідомленням про очікування
Extension	Повідомлення про помилку у випадку невдалого створення лоббі
Post-Condition	Створено нове ігрове лобі у якому знаходиться користувач

Таблиця 1.4 - Варіант використання UC-3

Use case name	Підключення до ігрової кімнати
Use case ID	UC-03
Goals	Підключитися до лобі
Actors	Користувач
Trigger	Користувач натиснув на кнопку підключитися як клієнт та ввів відповідний правильний код ігрової сесії
Pre-conditions	Введені ім'я та код ігрової сесії
Flow of Events	Користувач натискає на кнопку підключення до лобі та очікує завантаження у нього
Extension	У випадку невдалого підключення видається повідомлення про помилку і через яку саме причину користувач не зміг під'єднатися
Post-Condition	Гравець успішно перемістився до лобі

Таблиця 1.5 - Варіант використання UC-4

Use case name	Ініціалізація початку гри
Use case ID	UC-04
Goals	Початок гри
Actors	Користувач
Trigger	Користувач, який створив лобі натискає на кнопку початку гри
Pre-conditions	Гравець підключений до лобі та кількість гравців у лобі дорівнює мінімальній кількості гравців для початку гри
Flow of Events	Користувач, який створив лобі натискає на кнопку початку гри, виводиться діалогове вікно про очікування початку гри
Extension	У випадку невдалого початку гри відбувається повернення до лобі та показується діалогове вікно з відповідним повідомленням
Post-Condition	Гравці з лобі знаходяться безпосередньо у грі

Таблиця 1.6 - Варіант використання UC-5

Use case name	Кидання кубика
Use case ID	UC-05
Goals	Кинути ігровий кубик, який вказує на кількість полів, якими може переміститися гравець
Actors	Користувач
Trigger	Користувач натиснув на кнопку кидання кубика
Pre-conditions	Наступила черга гравця виконувати свій
Flow of Events	Користувач натискає на кнопку кидання кубика і відбувається переміщення ігровим полем відповідно до отриманого числа
Extension	-
Post-Condition	Гравець перемістився ігровим полем

Таблиця 1.7 - Варіант використання UC-6

Use case name	Придбання власності
Use case ID	UC-06
Goals	Придбати нову власність
Actors	Користувач
Trigger	Гравець приземлився на поле, яке нікому не належить
Pre-conditions	Поле власності є вільним та не належить жодному іншому гравцю
Flow of Events	Користувач може придбати, або відмовитися від придбання власності натискаючи на відповідні кнопки на панелі вибору взаємодії з ігровим полем
Extension	У випадку недостатньої кількості ігрової валюти для придбання поля виводиться діалогове вікно з відповідним повідомленням
Post-Condition	Гравець придбав поле, або відмовився від цього, завершив свій хід і право наступного ходу успішно передається іншому гравцю

Таблиця 1.8 - Варіант використання UC-7

Use case name	Розбудова монополії
Use case ID	UC-07
Goals	Побудувати покращення ігрового поля
Actors	Користувач
Trigger	Гравець натискає на відповідне поле, яке бажає покращити
Pre-conditions	У гравця зібрана повна монополія
Flow of Events	Користувач натискає на кнопку розбудови монополії, збільшується її рівень і відповідне значення демонструється за допомогою кількості зірок
Extension	У випадку недостатньої кількості засобів для придбання покращення поля виводиться діалогове вікно з відповідним повідомленням
Post-Condition	Гравець покращив поле

Таблиця 1.9 - Варіант використання UC-8

Use case name	Продаж власності
Use case ID	UC-08
Goals	Закласти власність, щоб повернути суму за її придбання
Actors	Користувач
Trigger	Користувач натискає на поле, яке йому належить, у відкритому вікні натискає на відповідну кнопку продажу власності
Pre-conditions	Поле куплене і гравець є її власником
Flow of Events	Користувач натискає на відповідне поле, яке бажає продати і натискає на відповідну кнопку продажу
Extension	У випадку, якщо поле вже є закладеним і його неможливо продати, то користувач отримує відповідне повідомлення з текстом причини через яке не вдалося продати поле
Post-Condition	Гравець заклав поле та отримав за це відповідну суму

Таблиця 1.10 - Варіант використання UC-9

Use case name	Ініціалізації здійснення обміну
Use case ID	UC-09
Goals	Зробити обмін між гравцями
Actors	Користувач
Trigger	Користувач натискає на панель відповідного гравця, з яким прагне здійснити обмін
Pre-conditions	Користувач отримав право ходу
Flow of Events	Користувач натискає на панель відповідного гравця, з яким прагне здійснити обмін, обирає поля, якими прагне обмінятись, кількість ігрової валюти, яка братиме участь у процесу обміну і натискає на кнопку пропозиції обміну
Extension	У випадку неправильних умов здійснення обміну з'являється діалогове вікно з відповідним текстом повідомленням про помилку
Post-Condition	Гравець надіслав пропозицію обміну іншому гравцю

Таблиця 1.11 - Варіант використання UC-10

Use case name	Прийняття обміну
Use case ID	UC-10
Goals	Прийняти запропонований обмін
Actors	Користувач
Trigger	Користувач натискає на кнопку прийняти обмін
Pre-conditions	Гравець отримав обмін від іншого гравця
Flow of Events	Користувач ознайомлюється з пропозицією обміну і приймає обмін натискаючи відповідну кнопку
Extension	-
Post-Condition	Обмін було здійснено успішно, поля та статки гравців оновилися, контроль повернувся до поточного гравця

Таблиця 1.12 - Варіант використання UC-11

Use case name	Відхилення обміну
Use case ID	UC-11
Goals	Відхилити запропонований обмін
Actors	Користувач
Trigger	Користувач натискає на кнопку відхилити обмін
Pre-conditions	Гравець отримав обмін від іншого гравця
Flow of Events	Користувач ознайомлюється з пропозицією обміну і приймає рішення про відхилення обміну і натискає на кнопку відхилити пропозицію обміну
Extension	-
Post-Condition	Обмін було відхилено, контроль повернувся до поточного гравця

Таблиця 1.13 - Варіант використання UC-12

Use case name	Ініціалізація процесу добровільної здачі
Use case ID	UC-12
Goals	Добровільно здатися та завершити гру
Actors	Користувач
Trigger	Користувач натискає на власну панель і йому демонструється діалогове вікно з пропозицією здатися
Pre-conditions	-
Flow of Events	Користувач натискає на власну панель і йому у спливаючому діалоговому вікні підтверджує свій намір достроково залишити гру і в залежності від обраного варіанту гравець продовжує гру, або добровільно її завершує
Extension	-
Post-Condition	Гравець успішно здався, позбувся усієї власності і більше не може брати участь у процесі гри

1.4.1 Розроблення функціональних вимог

Програмне забезпечення розділене на функціональні задачі. Кожна окремо взята функціональна задача має свій певний набір функцій. У таблиці 1.14 наведено загальну модель вимог, а в таблицях 1.15 – 1.25 наведений опис функціональних вимог до програмного забезпечення. Матрицю трасування вимог можна побачити у таблиці 1.26.

Таблиця 1.14 – Загальна модель вимог

ID	Призначення	Пріоритет
FR-01	Введення імені користувача	Високий
FR-02	Створення ігрової кімнати	Високий
FR-03	Підключення до ігрової кімнати	Високий
FR-04	Ініціалізація початку гри	Високий
FR-05	Кидання кубика	Середній
FR-06	Придбання власності	Середній
FR-07	Розбудова монополії	Середній
FR-08	Продаж власності	Середній
FR-09	Ініціалізації здійснення обміну	Низький
FR-10	Прийняття обміну	Низький
FR-11	Відхилення обміну	Низький
FR-12	Ініціалізація процесу добровільної здачі	Низький

Таблиця 1.15 – Функціональна вимога FR-1

Назва	Зміна імені
Опис	Система повинна надавати можливість користувачу змінювати власне ім'я і зберігати його після кожної зміни і відновлювати під час нового запуску гри

Таблиця 1.16 – Функціональна вимога FR-2

Назва	Створення власної ігрової кімнати
Опис	Гра має мати можливість створення власного лобі, якщо гравець прагне бути хостом ігрової сесії і відповідати за початок і завершення гри

Таблиця 1.17 – Функціональна вимога FR-3

Назва	Підключення до створеної ігрової кімнати
Опис	Гра має мати можливість підключення до ігрового лобі, у випадку, якщо користувач вводить відповідний правильний код ігрової сесії

Таблиця 1.18 – Функціональна вимога FR-4

Назва	Початок гри
Опис	Користувач-хост повинен мати можливість запустити гру, тим самим закриваючи лобі не дозволяючи іншим гравцям, які навіть знають правильний код під'єднатись до нього.

Таблиця 1.19 – Функціональна вимога FR-5

Назва	Кидання кубика
Опис	Коли настає черга відповідного гравця, то у цього користувача має бути можливість кинути кубик, щоб виконати переміщення ігровим полем, без кинутого кубика хід гравця не може бути зарахований, окрім певних виключних випадків.

Таблиця 1.20 – Функціональна вимога FR-6

Назва	Купівля власності
Опис	У випадку, коли гравець кинув кубик і перемістив на поле власності, яка нікому не належить, користувачу має надаватися змога купити цю власність.

Таблиця 1.21 – Функціональна вимога FR-7

Назва	Покращення власності монополії
Опис	Коли гравець зібрав повноцінну монополію, то користувачу має відкритися можливість покращувати її поля, тим самим підвищуючи ціну за оренду перебування на ній інших гравців.

Таблиця 1.22 – Функціональна вимога FR-8

Назва	Продаж, або закладення власності
Опис	Користувачу має бути доступна опція продажі покращень на полі зібраної монополії, або тимчасове закладання придбаної власності.

Таблиця 1.23 – Функціональна вимога FR-9

Назва	Здійснення обміну
Опис	Гравцю має бути надана можливість запропонувати обмін іншому гравцеві, у випадку якщо запропонована пропозиція обміну відповідає всім правилам та вимогам.

Таблиця 1.24 – Функціональна вимога FR-10

Назва	Можливість взаємодіяти з обміном
Опис	Гравцю, якому запропонували обмін має бути надана можливість прийняти його умови, або відхилити пропозицію з відповідними наслідками.

1.4.2 Розроблення нефункціональних вимог

Нефункціональні вимоги – це характеристики системи чи програмного забезпечення, які не стосуються конкретної функціональності, але визначають якісні аспекти їхньої роботи та характеристики. Ці можливості визначають "якість" системи та включають такі аспекти, як продуктивність, надійність, безпека та інші.

Таблиця 1.27 – Нефункціональні вимоги

Номер	Категорія	Опис
NFR-1	Продуктивність	Гра повинна забезпечувати нормальний рівень швидкодії та оптимізації, споживати якомога менше заряду батареї пристрою.
NFR-2	Надійність	У випадку виникнення критичних ситуацій, помилок, гравцю мають надаватися зрозумілі та інформативні повідомлення.
NFR-3	Сумісність	Гра повинна працювати на різних мобільних пристроях, щоб забезпечити доступність для широкого кола гравців.
NFR-4	Безпека	Гра повинна гарантувати безпеку взаємодії гравців та захист персональних даних користувачів шляхом використання зашифрованих шляхів їх передачі.
NFR-5	Зручність інтерфейсу	Гра повинна мати інтуїтивно зрозумілий та зручний для використання користувацький інтерфейс.

1.5 Постановка задачі

Мобільний застосунок спрямований на створення реалізації мобільної гри «Монополія» з багатокористувацьким режимом до 5 гравців, забезпечуючи при цьому зручний графічний інтерфейс та покращені ігрові правила, які спрямовані на скорочення середнього часу ігрової сесії та покращення загального ігрового досвіду. Основна мета цього проєкту полягає у реалізації мобільної гри, який надасть гравцям можливість зіграти у відому настільну гру, використовуючи свій мобільний пристрій.

Під час процесу розробки будуть вирішені та реалізовані наступні функціональні задачі: можливість створювати власні лобі та бути хостом ігрової сесії, або приєднуватися до ігрових кімнат у ролі користувача-клієнта; кидання кубика та пересування ігровим полем на такі кількість кроків, яке дорівнює числу, яке було отримано внаслідок випавшого значення; можливість взаємодії з ігровими полями, а саме наступні варіанти: купити власність, яка нікому ще не належить, варіант закласти поле, яке належить гравцю та на якому ще не побудовано жодних покращень, у іншому випадку – надається можливість продати покращення поля, також надається можливість покращити поле, тим самим збільшивши ціну оренди за перебування інших гравців на обраному полі; можливість проводити обміни з іншими гравцями у тому випадку, якщо варіант обміну задовольняє умови валідації; здатність взаємодії з обмінами, а саме прийняти, або відхилити його; можливість здатися, тим самим гравець лишається всіх своїх полів, статків і більше не приймає участь у грі.

Під час розробки мобільної гри основна увага буде спрямована на забезпечення високої продуктивності та ефективної оптимізації, мінімізації впливу гри на заряд батареї пристрою. Одночасно особлива увага приділяється сумісності гри з різними мобільними пристроями, забезпечуючи доступність для різноманітної аудиторії. Захист взаємодії гравців та конфіденційності їхніх даних буде досягнуто за допомогою шифрованих каналів передачі даних.

Висновки до розділу

В даному розділі курсової роботи було виконано комплексний аналіз предметної області, що включав в себе загальний огляд предметної області, визначення основних термінів і напрямків розробок. Додатково був проведений змістовний опис і аналіз поточного стану використання знань предметної області в сфері інформаційних технологій, з фокусом на реалізацію в існуючих сучасних програмних рішеннях.

Детально були розглянуті існуючі технології та успішні проекти в даній області, зокрема алгоритмічне забезпечення, технічні рішення, допоміжні програмні засоби, засоби розробки та готові конкурентні програмні рішення. Проведено аналіз відомих алгоритмів та технічних рішень, вибір оптимальних шляхів для розробки. Також проведений огляд допоміжних програмних засобів та засобів розробки, їх порівняльний аналіз для вибору найбільш підходящих інструментів для поточної розробки.

Додатково, було проведено порівняння розробленого програмного продукту з відомими аналогами, яке було представлене у вигляді таблиці, яка містить функціональні та нефункціональні особливості кожного продукту для більш глибокого та детального порівняння.

Також було виконано аналіз вимог до програмного забезпечення, включаючи опис головного функціоналу, діаграму варіантів використання та детальний опис варіантів використання. На основі цього визначено головні завдання та вимоги програмного забезпечення. Далі розроблені функціональні та нефункціональні вимоги, включаючи модель вимог, їх опис та матрицю трасування. Додатково було сформовано саме завдання курсової роботи, визначені головні задачі та підзадачі на шляху до виконання завдання.

2 МОДЕЛЮВАННЯ ТА КОНСТРУЮВАННЯ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

2.1 Моделювання та аналіз програмного забезпечення

Для моделювання та аналізу програмного забезпечення було використано BPMN (Business Process Model and Notation). BPMN – це стандарт для моделювання бізнес-процесів, який надає уніфікований мовний засіб для спільного розуміння, аналізу та оптимізації бізнес-процесів. Для опису бізнес процесу програмного забезпечення використовується BPMN модель (рисунок 2.1 та 2.2).

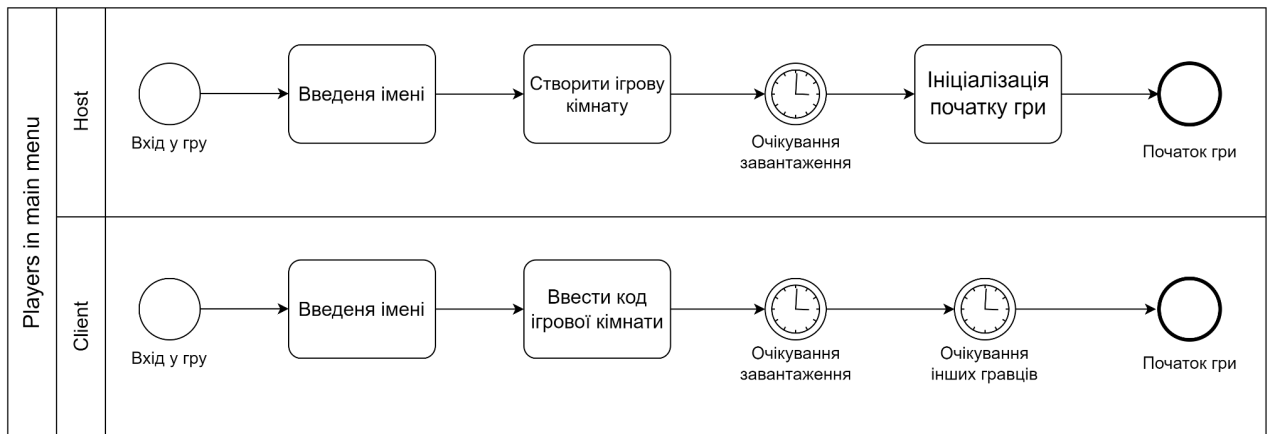


Рисунок 2.1 – BPMN-модель для гравців обох типів під час під’єднання до ігрової сесії та початку гри

Опис послідовності під час під’єднання до ігрової сесії:

- Користувач вводить ім’я у відповідне поле;
- Якщо користувач планує виконувати роль хоста ігрової сесії, то натискає відповідну кнопку;
- Якщо користувач планує приєднатися як клієнт, то вводить код ігрової сесії;
- Очікування завантаження у лобі;
- Очікування інших гравців та початку гри.

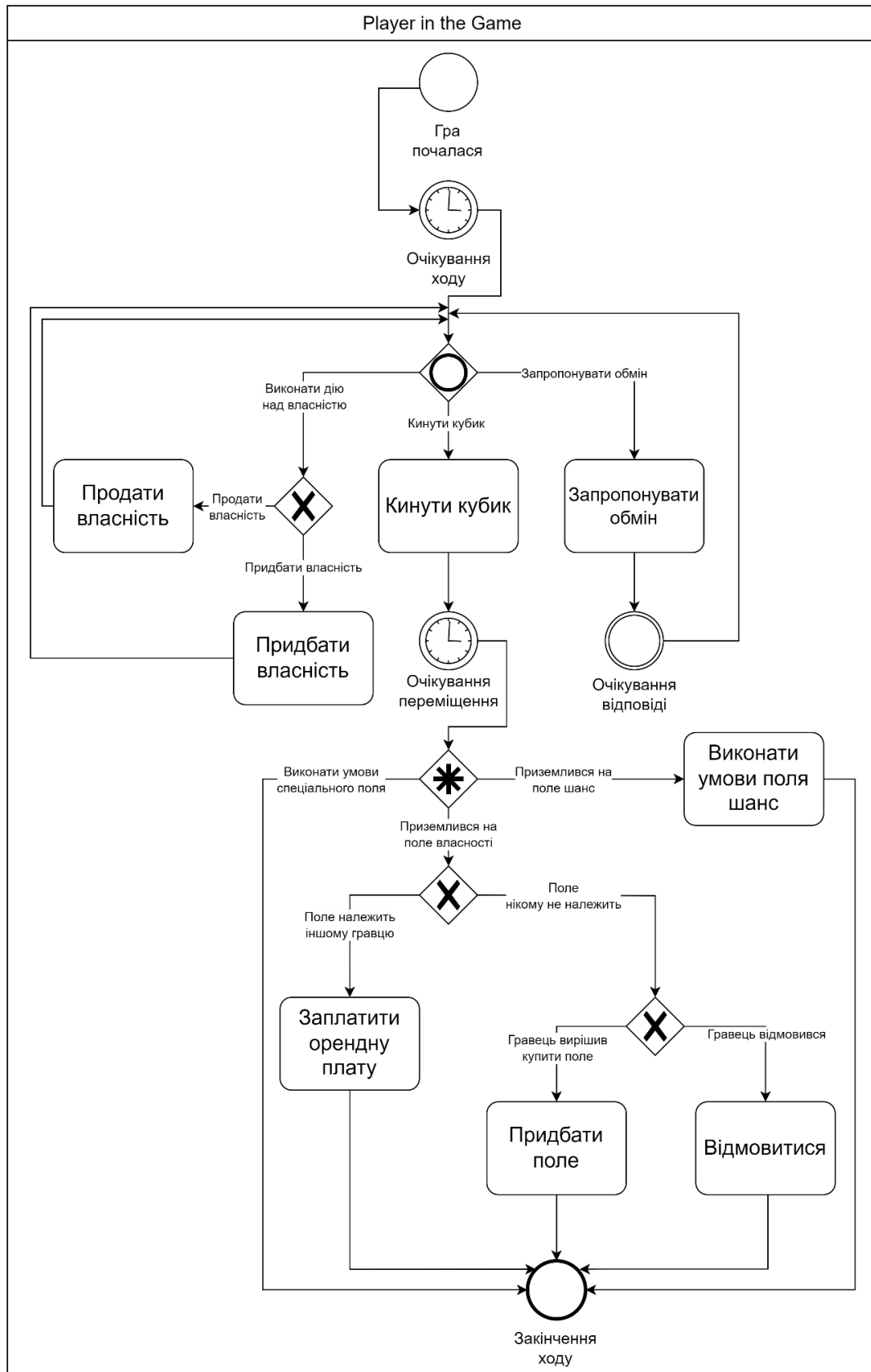


Рисунок 2.2 – BPMN-модель для гравця під час виконання ходу

Опис послідовності під час виконання ходу:

- Гравець очікує початку свого ходу;
- Якщо гравець обирає виконати дію над власністю, то виконує необхідну дію;
- Якщо гравець прагне почати процес обміну, то виконує необхідну дію;
- Якщо гравець кидає кубик, то відбувається переміщення ігровим полем;
- Очікування переміщення полем;
- Якщо гравець приземлився на поле шанс, то виконує необхідні дії;
- Якщо гравець приземлився на особливе поле, то виконує необхідні дії;
- Якщо гравець приземлився на поле власності, яка належить іншому гравцю, то сплачує оренду;
- Якщо гравець приземлився на поле власності, яке нікому не належить, то пропонується придбати цю власність.

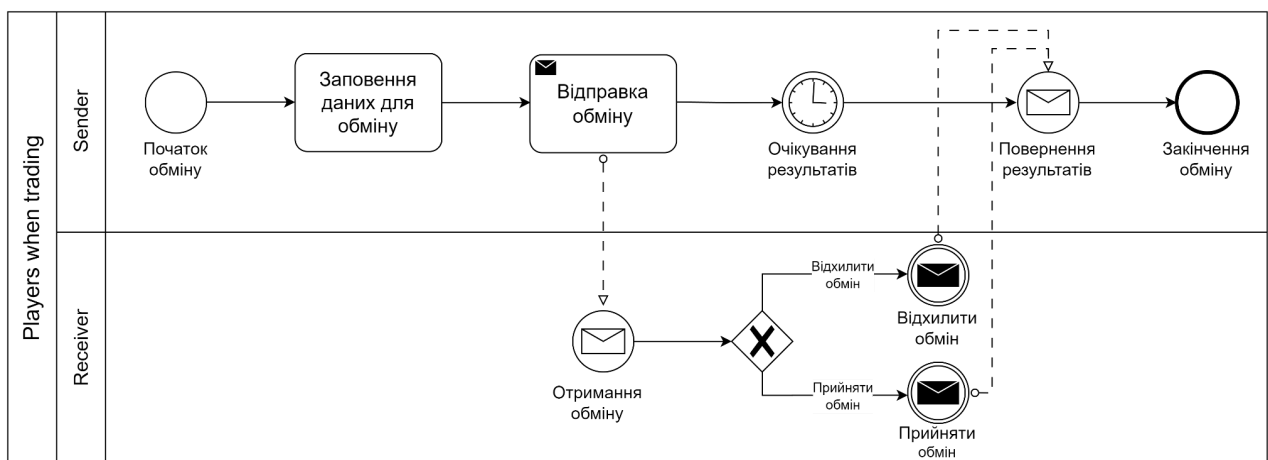


Рисунок 2.3 – BPMN-модель для гравця під виконання обміну

Опис послідовності під час виконання обміну:

- Заповнення даних для обміну;
- Відправка обміну;
- Прийняття, або відмові від обміну від іншого гравця;
- Повернення керування до відправника.

2.2 Архітектура програмного забезпечення

Під час розробки відеогри використання патернів проектування стає ключовим етапом для забезпечення ефективності, читабельності та легкості керування кодом і внесенням змін у нього. Для розробки використовувалися наступні патерни: Entity-Component-System (ECS) та Model-View-Controller (MVC).

Entity-Component-System (ECS) – у випадку використання цього патерну об'єкти в грі складаються з компонентів, а системи обробляють і взаємодіють з цими компонентами. Кожен модуль відповідає за конкретну функціональність, і об'єкти можуть легко модифікуватися та розширюватися за допомогою додавання або видалення компонентів [6].

Model-View-Controller (MVC) – відповідає за відокремлення компонентів гри, таких як логіка гри, графічний інтерфейс та дані гри, забезпечуючи модульність та гнучкість архітектури. У випадку розроблюваної мобільної реалізації «Монополії» модель включає в себе дані та ігрову логіку. Це може включати інформацію про стан гри (баланс гравців, поточна власність, обробка подій) та правила, за якими гра функціонує. Модель визначає, як гравець може рухатися, купувати власність, отримувати прибуток тощо. Представлення відповідає за те, як гра відображається користувачеві на екрані мобільного пристрою. Це може включати графічний інтерфейс та інші графічні аспекти відображення гри. Контролер відповідає за обробку введення користувача та взаємодію з моделлю. На мобільних пристроях це може включати елементи керування, такі як доторкання, ввід від користувача у текстові поля, тощо [5].

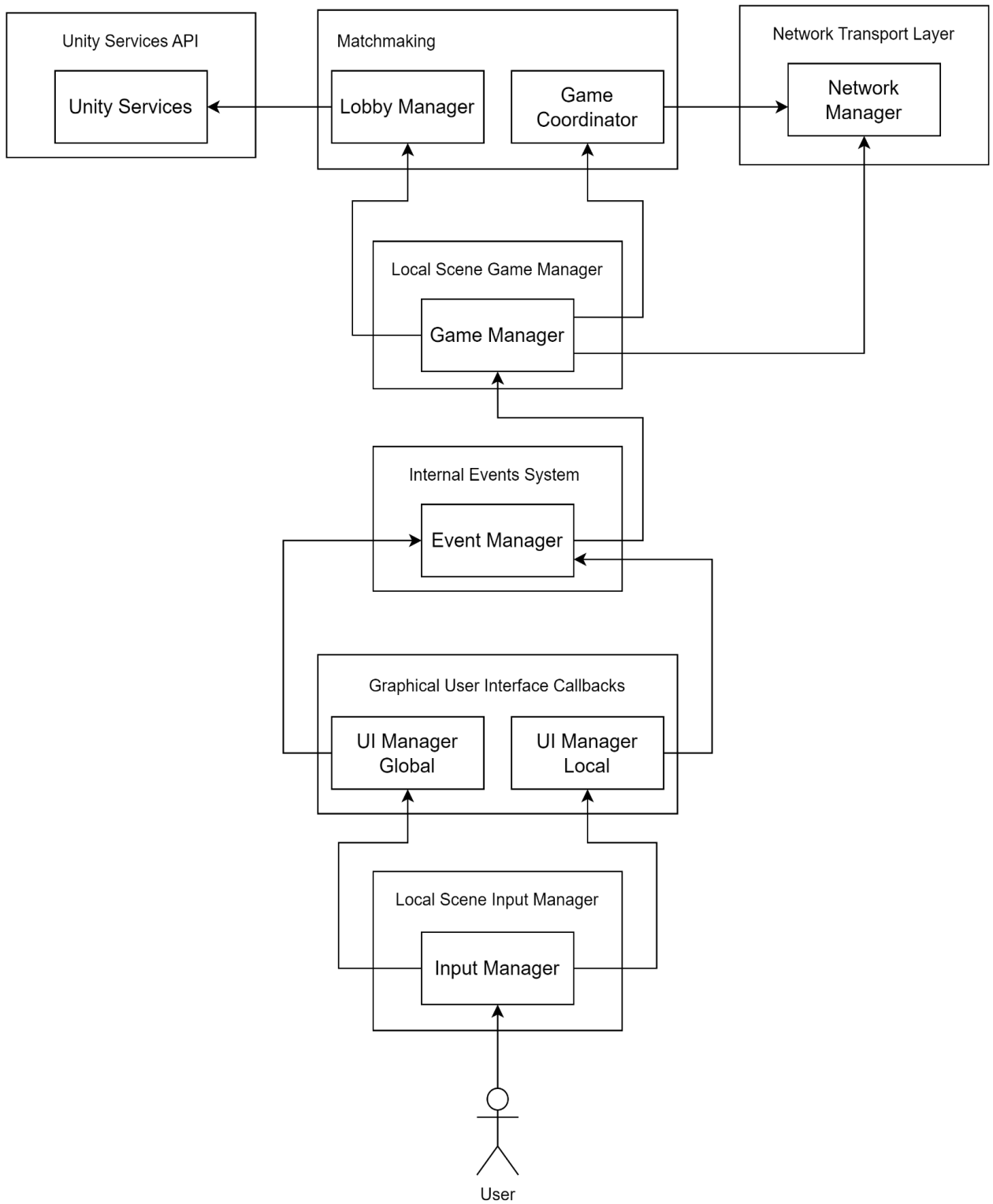


Рисунок 2.4 – Загальна діаграма архітектури програмного забезпечення

2.3 Конструювання програмного забезпечення

Розробка мобільного ігрового застосунку була досягнута за рахунок створення наступних класів, кожен з яких виконує свою функцію.

Таблиця 2.1 – Опис класів

Назва	Опис
GameCoordinator	Ігровий координатор, який відповідає за підключення гравців до спільної ігрової сесії, менеджмент ресурсів мережі, оновлення інформації про кожного гравця, тощо.
LobbyManager	Менеджер ігрового лобі, який відповідає за оновлення стану лобі, інформації про гравців у ньому, початок гри, тощо.
GameManager	Ігровий менеджер, який відповідає за синхронізацію стану гри, безпосередню ігрову логіку, розсилку повідомлень, оновлення значень.
MonopolyBoard	Допоміжний клас для роботи з полями монополії.
MonopolyNode	Клас-представлення поля монополії з відповідною інформацією про ціну за оренду, значення поточного рівня, тощо.
MonopolyPlayer	Клас-представлення гравця у монополію, з відповідними методами для взаємодії з ігровим світом.
MonopolyPlayerVisuals	Клас-представлення візуальної складової гравців.
MonopolySet	Клас для зберігання інформації про монополії.

Продовження таблиці 2.1

ChanceNodeSOEditor	Редактор інформації про поле шанс з монополії.
MonopolyNodeEditor	Редактор інформації про поле монополії.
ChanceNodeSO	Клас-шаблон для полів шанс.
ObjectPoolMessageBoxes	Клас-реалізація патерну Object Pool для спливаючих вікон.
ObjectPoolPlayersLobby	Клас-реалізація патерну Object Pool для представлення гравців у лобі.
PanelInfoUI	Клас-представлення інформативної панелі.
PanelMessageBoxUI	Клас-представлення панелі спливаючого вікна.
PanelMonopolyNodeUI	Клас-представлення панелі спливаючого вікна.
PanelOfferUI	Клас-представлення панелі пропозиції купівлі поля.
PanelPaymentChanceUI	Клас-представлення панелі сплати штрафу.
PanelPaymentPropertyUI	Клас-представлення панелі сплати податку.
PanelPlayerGameUI	Клас-представлення панелі гравця у грі.
PanelPlayerLobbyUI	Клас-представлення панелі гравця у лобі.
PanelReceiveTradeUI	Клас-представлення панелі отриманого обміну.
PanelTradeOfferUI	Клас-представлення панелі створення обміну.
UIManagerBootstrap	Клас, який відповідає за взаємодію з графічним користувацьким інтерфейсом на сцені завантаження необхідних компонентів гри.

Продовження таблиці 2.1

UIManagerGameLobby	Клас, який відповідає за взаємодію з графічним користувацьким інтерфейсом на сцені ігрового лобі.
UIManagerGlobal	Клас, який відповідає за взаємодію з графічним користувацьким інтерфейсом, який може бути викликаний на будь-якій сцені.
UIManagerMainMenu	Клас, який відповідає за взаємодію з графічним користувацьким інтерфейсом на сцені головного меню.
UIManagerMonopolyGame	Клас, який відповідає за взаємодію з графічним користувацьким інтерфейсом на самої гри.
UINodeTouchHandler	Клас, який відповідає за обробку вводу користувача.

Оскільки у розробленому ігровому застосунку наявний багатокористувацький режим, то існує необхідність розробки відповідних сутностей для роботи з підключенням до ігрової сесії та загальному менеджменту ігрового процесу одночасно для кількох гравців. Саме тому було прийнято рішення про розробку ігрового координатора.

Ігровий координатор – сутність, яка відповідає за організацію ігрового процесу та за взаємодію між різними складовими гри у багатокористувацький ігровій сесії, де на нього покладена роль пошуку матчів, під'єднання гравців до спільної ігрової сесії, менеджментом мережевих та інших ресурсів, тощо [16].

Ще одним важливим компонентом розробленого програмного забезпечення, на яке необхідно звернути увагу – LobbyManager, який виконує роль координатора завантаженого ігрового лобі. Його роль полягала у взаємодії

з API Unity-сервісів для оновлення поточного стану лобі, кількості гравців, інформації про них [14].

Через специфіку розробленого програмного забезпечення під час розробки використовувалися спеціалізовані для створення неткоду структури даних та методи. Специфіка написання неткоду передбачала використання механізмів серіалізації та десеріалізації даних для ефективного обміну інформацією між клієнтом та сервером. Система використовувала протоколи серіалізації, *protobufs* (*protocol buffers*), щоб перетворювати структури даних у бінарний формат, який може бути легко переданий через мережу. Це було важливо для забезпечення ефективного обміну даними та зменшення обсягу пакетів з інформацією, які передаються. Додатково варто відзначити фактичну неможливість передачі всіх типів даних, які зберігаються на купі пам'яті, тому дозволена лише серіалізація та передача примітивних типів даних [18].

Додатково варто відзначити *Remote Procedure Calls (RPC)*, які зіграли ключову роль у взаємодії між клієнтом і сервером. Використання *RPC* дозволило викликати методи на віддаленому сервері та клієнті так, ніби вони викликаються локально. Це полегшувало взаємодію між клієнтом та сервером, дозволяючи їм обмінюватися даними, оновлювати стан гри та виконувати дії в реальному часі. *RPC* є важливим механізмом, який спрямований на забезпечення взаємодії між програмами, що працюють на різних вузлах мережі. Це дозволяє викликати функції або процедури на віддалених серверах так, ніби вони викликаються локально. *RPC* виграє ключову роль у багатьох сценаріях, зокрема в розробці багатокористувацьких ігор, де взаємодія між гравцями та серверами є критично важливою. Цей механізм ґрунтується на ідеї абстрагування віддалених викликів від деталей мережевого обміну даними. Використання *RPC* дозволяє розробникам уникнути ручного управління мережевими протоколами та деталями взаємодії між клієнтами та серверами. Це робить процес розробки багатокористувацьких ігор більш ефективним та полегшує створення стійких до помилок та ефективних мережеских додатків [21].

Окрім того, для забезпечення ефективної взаємодії, використовувалися спеціальні структури даних, які містили параметри, необхідні для визначення та виклику відповідних функцій або методів на клієнті чи сервері. Вони служили інструментом для точного визначення, на якій конкретній машині мали виконатися відповідні дії локально з метою синхронізації стану гри [15].

Варто зазначити особливість створення багатокористувацьких ігор з одноранговим централізованим типом мережі, яка полягає у тому, що клієнти не можуть напряму викликати RPC інших клієнтів, оскільки це зроблено з метою забезпечення безпеки. Тому пайплайн синхронізації дій за допомогою неткоду має бути реалізований через локальний виклик методу оновлення стану гри і примусового виклику віддаленої процедури на сервері, який потім, у свою чергу, розсилає оновлення іншим зацікавленим клієнтам [16].

Опис утиліт, бібліотек та іншого стороннього програмного забезпечення, що використовується у розробці наведено в таблиці 2.22.

Таблиця 2.22 – Опис утиліт

№ п/п	Назва утиліти	Опис застосування
1	Microsoft Visual Studio	Головне середовище розробки програмного забезпечення курсової роботи.
2	Unity Engine	Програмне забезпечення необхідне для безпосереднього створення гри завдяки наданню широкого набору інструментів, певних готових компонентів, наявності користувацького інтерфейсу.
3	Adobe Illustrator	Програмне забезпечення, яке використовувалося для створення растрової графіки у грі, а саме іконок, логотипів полів та деяких інших графічних елементів.

2.4 Аналіз безпеки даних

Relay Service від Unity – це служба, яка забезпечує з'єднання між гравцями в багатокористувацьких іграх. Relay використовує протокол UDP для передачі даних між гравцями, а також забезпечує шифрування даних для безпеки. Протокол UDP (User Datagram Protocol) – це протокол передачі даних, який використовується для передачі даних в реальному часі. UDP не гарантує доставку даних, але він забезпечує низьку затримку і низький додаткове навантаження.

UDP – це один із протоколів транспортного рівня в мережевій моделі OSI та мережових протоколів. Протокол UDP є не надійним, тобто він не забезпечує гарантії доставки та порядку при передачі даних. Основна відмінність між UDP та іншим популярним протоколом транспортного рівня, таким як TCP (Transmission Control Protocol), полягає в тому, що UDP не встановлює стійке з'єднання перед передачею даних і не використовує підтвердження прийому. Це робить його швидшим, але менш надійним, особливо в умовах, коли частина даних може бути втрачена або доставлена не в потрібному порядку. Протокол UDP широко використовується для передачі даних у реальному часі, таких як передача відео та аудіо, ігри та інші додатки, де більший акцент робиться на швидкості передачі даних, ніж на гарантії доставки [9].

Інформація, яка передається через Relay, шифрується протоколом DTLS для безпеки. Протокол DTLS (Datagram Transport Layer Security) – це протокол шифрування, який використовується для захисту даних, які передаються через UDP [10]. DTLS забезпечує шифрування даних, щоб захистити їх від перехоплення і розшифровки. DTLS використовується в Relay від Unity для захисту даних, які передаються між гравцями, це допомагає забезпечити безпеку і конфіденційність для гравців [11].

Висновки до розділу

У цьому розділі був представлений обширний аналіз ключових аспектів розробки ігрового застосунку, основною особливістю якого є багатокористувацький режим. Були детально розглянуті ключові аспекти процесу розробки, враховуючи бізнес-процеси, архітектурні патерни, алгоритми та безпеку програмного забезпечення.

Перше, що слід відзначити, це створення опису бізнес-процесів розробки, представлені за допомогою BPMN-діаграм, що дозволяє ефективно визначити та оптимізувати різноманітні етапи розробки, забезпечуючи високий рівень продуктивності та якості.

Відомо, що ефективна, стабільна та масштабована архітектура грає важливу роль у забезпеченні стабільного та високопродуктивного продукту. Під час розробки було вирішено використовувати архітектурний паттерн Entity-Component-System (ECS) для створення гнучкої системи, де об'єкти гри складаються з компонентів, що полегшує модифікацію та розширення функціональності. Також був впроваджений Model-View-Controller (MVC) для відокремлення компонентів гри, що дозволяє досягти модульності та гнучкості в архітектурі.

Також важливо відзначити конструювання програмного забезпечення, на якому були описані головні виклики, проблеми та особливості, які виникають під час створення гри з багатокористувацьким режимом. Важливим рішенням було впровадження ігрового координатора, що дозволило ефективно керувати ігровими сесіями. Ігровий координатор відповідає за організацію ігрового процесу та взаємодію між гравцями, забезпечуючи швидке підключення та менеджмент ресурсів. У розділі було також розглянуто неткод, який включає в себе використання серіалізації, protobufs для ефективного обміну даними через мережу, а також RPC, які виявилися ключовим механізмом для забезпечення взаємодії між клієнтом і сервером. Використання RPC дозволило здійснювати виклики методів на віддаленому сервері та клієнті так, ніби вони викликаються локально, що важливо для реалізації взаємодії у реальному часі.

3 АНАЛІЗ ЯКОСТІ ТА ТЕСТУВАННЯ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

3.1 Аналіз якості ПЗ

Аналіз якості ПЗ було проведено з допомогою веб-сервісу Codacy, який автоматично генерує звіт якості коду за різними метриками. Перевагами цього сервісу є можливість обирати які самі файли з кодом необхідно включити в загальну оцінку. Ця перевага є особливо актуальною для комплексних ігрових застосунків через велику кількість авто згенерованих файлів з кодом, які не потребують оцінки [13].

Метрики, за якими проводитиметься оцінка якості коду розробки:

- Issues – ця метрика вказує на наявність помилок, або інших проблем у коді. Проблеми можуть бути різного характеру, такі як логічні помилки, некоректне використання змінних, тощо. Оцінка проблем дозволяє визначити, наскільки ефективно функціонал коду виконується і які можливі проблеми можуть виникнути під час його виконання.
- Complexity – визначається різними факторами, такими як кількість вузлів у контрольному потоці, різноманітність використаних алгоритмів, глибина вкладених структур, тощо. Вища складність може призвести до складнішого розуміння коду, важкості виправлення помилок і втрати продуктивності при роботі над ним. Зазвичай висока складність вказує на потенційні проблеми, такі як неефективність і важкість супроводу.
- Duplication – вказує на наявність однакового чи подібного коду у різних частинах програми. Це може бути показником недостатньої абстракції і наявності повторного використання коду. Виявлення та усунення дублювання може поліпшити читабельність, обслуговуваність і зменшити ризик виникнення помилок, оскільки будь-які зміни потрібно буде внести лише в одному місці.

Monopoly B

Рисунок 3.1 – Загальна оцінка вихідного коду проєкту

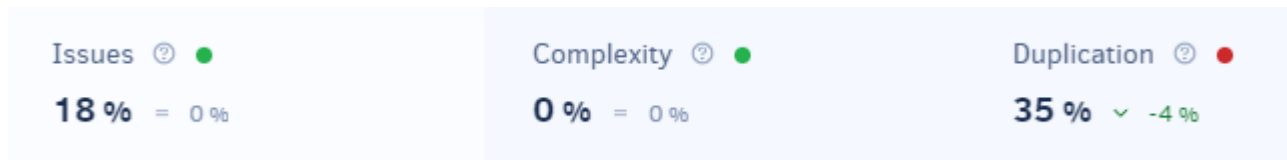


Рисунок 3.2 – Оцінка якості коду проєкту за метриками



Рисунок 3.3 – Кількість наявних проблем у коді

Щодо отриманих результатів важливо зазначити, що оцінка якості коду часто є суб'єктивною. Розробники можуть мати різні погляди на те, що вважається гарними підходами до використання певних синтаксичних конструкцій. Оцінки можуть сильно варіюватися в залежності від досвіду, стилю програмування та особистих уподобань. Також якість коду може залежати від контексту і вимог проєкту. Те, що вважається гарним рішенням для

одного проекту, може бути несхожим на іншому, залежно від специфіки завдань та потреб користувачів. Цей аспект є доволі актуальним для розробки ігор загалом, а особливо для багатокористувацьких проєктів, оскільки у них може використовуватися багато дублюючого коду синхронізації у неткодї, або код для створення редакторів певних компонент. Додатково варто зазначити, що зовнішні фактори, такі як обмежені ресурси, терміни проєкту, стратегічні цілі компанії можуть впливати на рішення розробників та якість їхнього коду [18].

3.2 Опис процесів тестування

Для тестування будуть використані функціональні та мануальні тести, які будуть перевіряти правильність виконання всіх функціональних вимог. Варто зазначити, що оскільки розроблюваним продуктом є ігровий мобільний додаток, то створення та впровадження автоматичних тестів різного рівня є бажаним, але часто неможливими завданням саме у сфері розробки відеоігор [23].

Було виконане мануальне тестування програмного забезпечення, опис відповідних тестів наведено у таблицях 3.1 – 3.30.

Таблиця 3.1 – Тест 1.1

Тест	Зміна імені користувача
Модуль	Головне меню
Номер тесту	1.1
Початковий стан системи	Користувач знаходиться у головному меню
Вхідні дані	Текстове значення імені користувача
Опис проведення тесту	У відповідне поле вводиться: коректне ім'я, яке має складатися хоча б з 1 символу, який не є пустим символом, або символом пробілу і максимальна довжина складає 10 символів.
Очікуваний результат	Зміна імені проходить успішно, користувач спокійно може приєднатися до ігрового лобі.

Продовження таблиці 3.2

Фактичний результат	Зміна імені проходить успішно, користувач успішно змінив своє ім'я та може перейти до створення власного лобі, або підключення до існуючого
---------------------	---

Таблиця 3.2 – Тест 1.2

Тест	Створення ігрової кімнати
Модуль	Головне меню
Номер тесту	1.2
Початковий стан системи	Гравець знаходиться у головному меню та ввів своє ім'я
Вхідні дані	-
Опис проведення тесту	Користувач натискає на кнопку створення лобі та очікує завантаження ігрової кімнати.
Очікуваний результат	Користувач успішно завантажився в ігрове лобі.
Фактичний результат	Після очікування завантаження гравець опинився в ігровому лобі.

Таблиця 3.3 – Тест 1.3

Тест	Підключення до ігрової кімнати
Модуль	Головне меню
Номер тесту	1.3
Початковий стан системи	Гравець знаходиться у головному меню та ввів своє ім'я
Вхідні дані	Код ігрової сесії

Продовження таблиці 3.3

Опис проведення тесту	Користувач натискає на кнопку підключення до лобі та очікує завантаження ігрової кімнати.
Очікуваний результат	Користувач успішно завантажився в ігрове лобі.
Фактичний результат	Після очікування завантаження гравець опинився в ігровому лобі.

Таблиця 3.4 – Тест 1.4

Тест	Початок матчу
Модуль	Ігрове лобі
Номер тесту	1.4
Початковий стан системи	Гравець знаходиться в ігровому лобі та є хостом цього лобі
Вхідні дані	-
Опис проведення тесту	Користувач наступає на кнопку початку гри, очікує завантаження гри, очікує завантаження інших гравців
Очікуваний результат	Після очікування завантаження гравці успішно загрузилися до гри.
Фактичний результат	Після очікування завантаження гри, очікування завантаження інших гравців гра успішно почалася.

Таблиця 3.5 – Тест 1.5

Тест	Кидання ігрового кубика
Модуль	Ігрова сесія
Номер тесту	1.5

Продовження таблиці 3.5

Початковий стан системи	Гравець знаходиться у грі та дочекався своєї черги робити хід.
Вхідні дані	-
Опис проведення тесту	Користувач натискає на кнопку кидання кубика, очікує переміщення ігровим полем на отриману кількість кроків.
Очікуваний результат	Гравцю було продемонстровано отримане значення внаслідок кидання кубиків і фішка, яка належить цьому користувачу виконала переміщення на відповідну кількість кроків.
Фактичний результат	Користувач отримав візуальне відображення значення скільки випало внаслідок кидання кубика і фішка була переміщена на задану кількість кроків.

Таблиця 3.6 – Тест 1.6

Тест	Придбання власності
Модуль	Ігрова сесія
Номер тесту	1.6
Початковий стан системи	Фішка гравця приземлилася на власність, яка нікому ще не належить.
Вхідні дані	-
Опис проведення тесту	Користувач натискає на кнопку придбання власності, після цього, у випадку достатню кількість ігрової валюти, оновлюється візуальна складова поля, встановлюючи колір власності у колір відповідного гравця.
Очікуваний результат	Оновлюється візуальна складова поля, встановлюючи колір власності у колір відповідного гравця.

Продовження таблиці 3.6

Фактичний результат	Гравець успішно придбав власність, колір цього поля був встановлений у колір гравця.
---------------------	--

Таблиця 3.7 – Тест 1.7

Тест	Розбудова монополії
Модуль	Ігрова сесія
Номер тесту	1.7
Початковий стан системи	Гравець має у наявності повністю зібрану хоча б 1 монополію.
Вхідні дані	-
Опис проведення тесту	Користувач натискає на кнопку покращення власності, після цього, у випадку достатню кількість ігрової валюти, оновлюється візуальна складова поля, збільшуючи кількість зірочок поля і оновлюючи плату за оренду цієї власності.
Очікуваний результат	Оновилися кількість зірок поля і збільшилася ціна за оренду даного поля.
Фактичний результат	Гравець успішно покращив власність, були здійснені відповідні візуальні зміни і збільшилася плата за оренду перебування інших гравців на цьому полі.

Таблиця 3.8 – Тест 1.8

Тест	Продаж власності
Модуль	Ігрова сесія
Номер тесту	1.8
Початковий стан системи	Гравець має хоча б 1 поле у власності поточного гравця.
Вхідні дані	-

Продовження таблиці 3.8

Опис проведення тесту	Користувач натискає на кнопку продажу власності, після цього оновлюється візуальна складова поля, яка позначає, що поле було закладено, або зменшується кількість зірочок і значення ціни за оренду зменшується, або встановлюється у 0.
Очікуваний результат	Власність з цього поля була продана, оновилося відповідна візуальна складова і було зменшена ціна за оренду перебування на ньому.
Фактичний результат	Гравець успішно продав власність, були здійснені відповідні візуальні зміни і зменшилася плата за оренду перебування інших гравців на цьому полі.

Таблиця 3.9 – Тест 1.9

Тест	Створення обміну
Модуль	Ігрова сесія
Номер тесту	1.9
Початковий стан системи	Гравець знаходиться у грі та дочекався своєї черги робити хід.
Вхідні дані	-
Опис проведення тесту	Користувач натискає на іншого гравця власності, після цього показується панель пропонування обміну, користувач здатен натиснути на поля, яке належить користувачу, з яким здійснює обмін та на свої поля своїх власностей і здатен ввести запропоновану кількість ігрової валюти.

Продовження таблиці 3.9

Очікуваний результат	Користувач може взаємодіяти з іншим гравцем у грі, натискати на його власності, що призводить до відкриття панелі пропозицій обміну. Після цього користувач може обрати поля, які належать йому та гравцю, з яким він хоче здійснити обмін, і вказати необхідну суму грошей в грі.
Фактичний результат	Користувач може взаємодіяти з іншим гравцем у грі, торкаючись його власностей, що призводить до відкриття панелі обміну. Після цього він має можливість вибрати свої власні поля та поля іншого гравця для обміну, вказати необхідну кількість ігрової валюти для запропонованого обміну.

Таблиця 3.10 – Тест 1.10

Тест	Прийняття пропозиції обміну
Модуль	Ігрова сесія
Номер тесту	1.10
Початковий стан системи	Гравець отримав пропозицію обміну від іншого гравця.
Вхідні дані	-
Опис проведення тесту	Користувач натискає на кнопку прийняти обмін, була списана відповідна до обміну кількість балансу, і власники полів власностей були успішно оновлені.
Очікуваний результат	Після підтвердження обміну, відбувається зняття відповідної кількості балансу, і власники полів власностей успішно оновлюються, хід повертається до поточного гравця.

Продовження таблиці 3.10

Фактичний результат	Після узгодження обміну, сума відповідного балансу сплачується, а власники земельних ділянок успішно оновлюють свою інформацію, після чого ініціатива повертається до поточного учасника гри.
---------------------	---

Таблиця 3.11 – Тест 1.11

Тест	Відхилення пропозиції обміну
Модуль	Ігрова сесія
Номер тесту	1.11
Початковий стан системи	Гравець отримав пропозицію обміну від іншого гравця.
Вхідні дані	-
Опис проведення тесту	Користувач натискає на кнопку відхилити обмін.
Очікуваний результат	Після відхилення обміну відбувається повернення керування до поточного гравця, якому приходить відповідне повідомлення, що інший гравець відмовився від пропозиції.
Фактичний результат	Обмін було відхилено, контроль ходу повернувся до поточного гравця, який має виконувати хід, було продемонстровано правильне інформативне повідомлення.

Таблиця 1.12 – Тест 1.12

Тест	Ініціалізація процесу добровільної здачі
Модуль	Ігрова сесія
Номер тесту	1.12

Продовження таблиці 1.12

Початковий стан системи	Гравець знаходиться у грі.
Вхідні дані	-
Опис проведення тесту	Користувач натискає на власну панель і йому демонструється діалогове вікно з пропозицією здатися, на як гравець погоджується.
Очікуваний результат	Гравець успішно здається, користувач лишається всієї свої власності, балансу, і залишається як сторонній спостерігач.
Фактичний результат	Користувач успішно здався та залишився у грі як сторонній спостерігач без права ходу.

3.3 Опис контрольного прикладу

У попередньому розділі було протестовано усі можливі взаємодії з ігровим застосунком. Тому у поточному розділі буде описаний контрольний приклад, який може продемонструвати ці всі взаємодії під час справжньої гри.

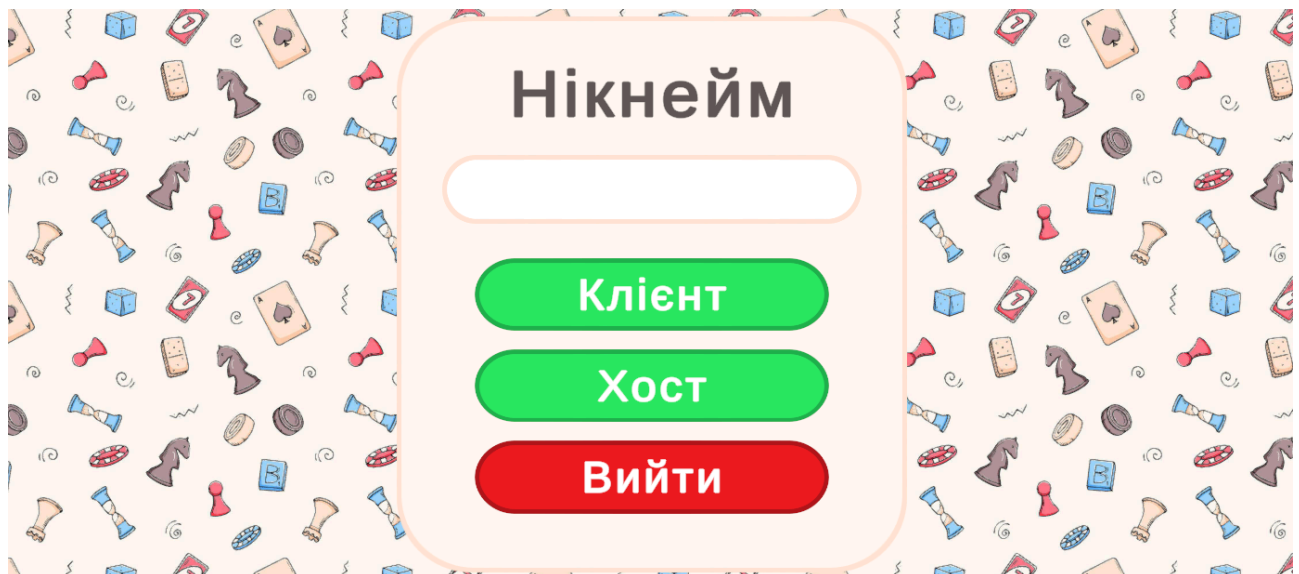


Рисунок 3.4 – Головне меню гри після відкриття

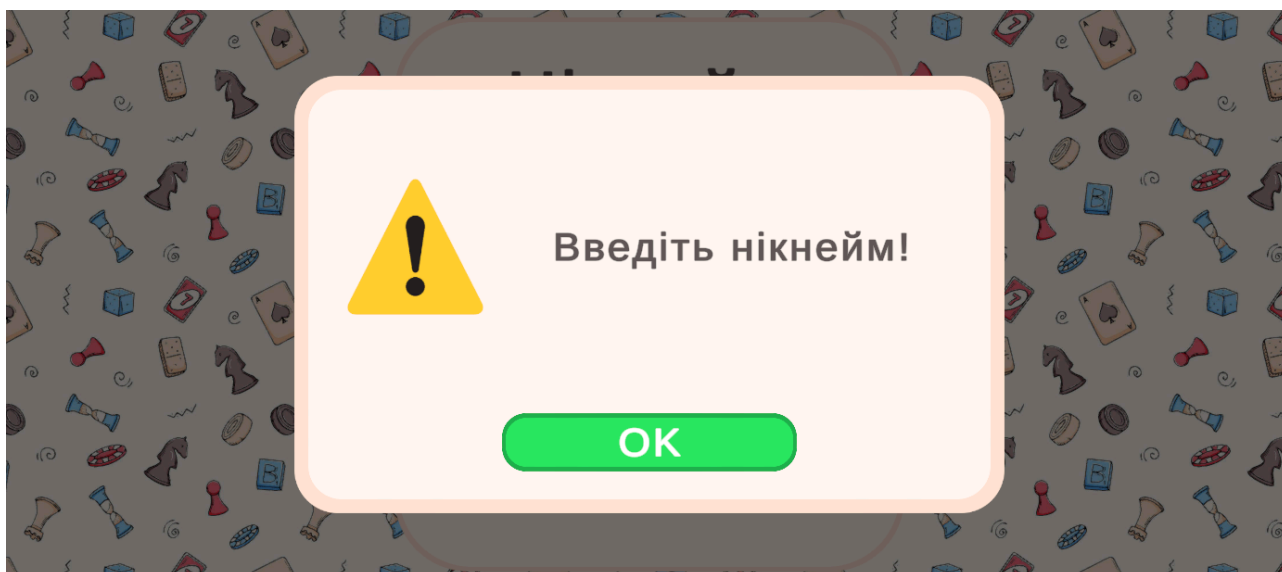


Рисунок 3.5 – Попередження у випадку введення пустого нікнейму

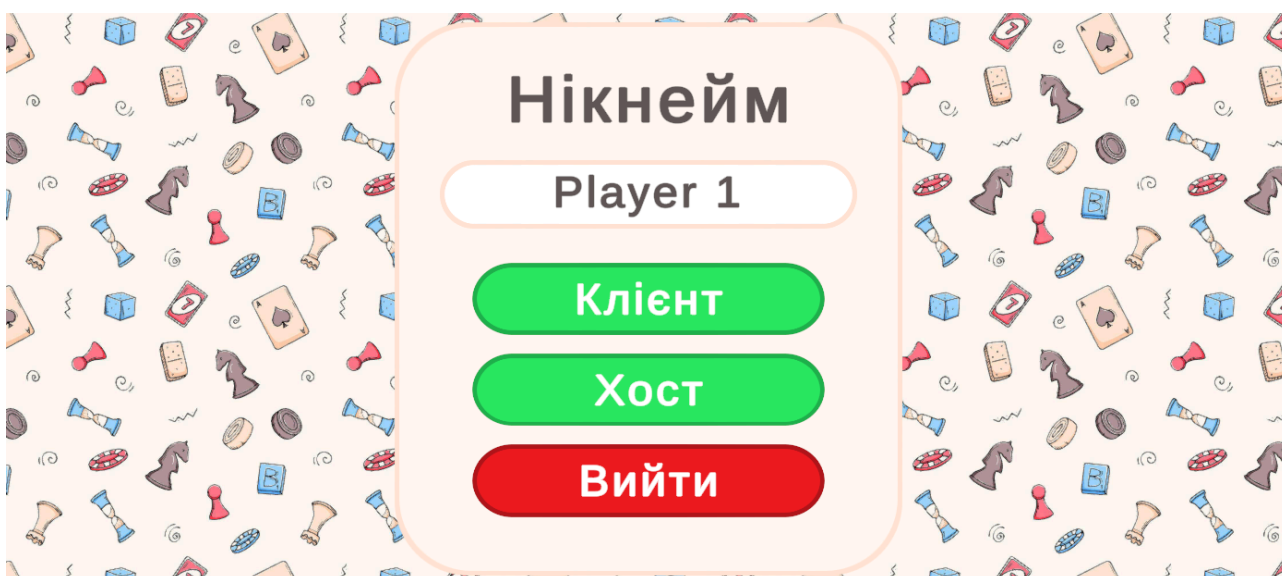


Рисунок 3.5 – Коректно введений нікнейм гравця

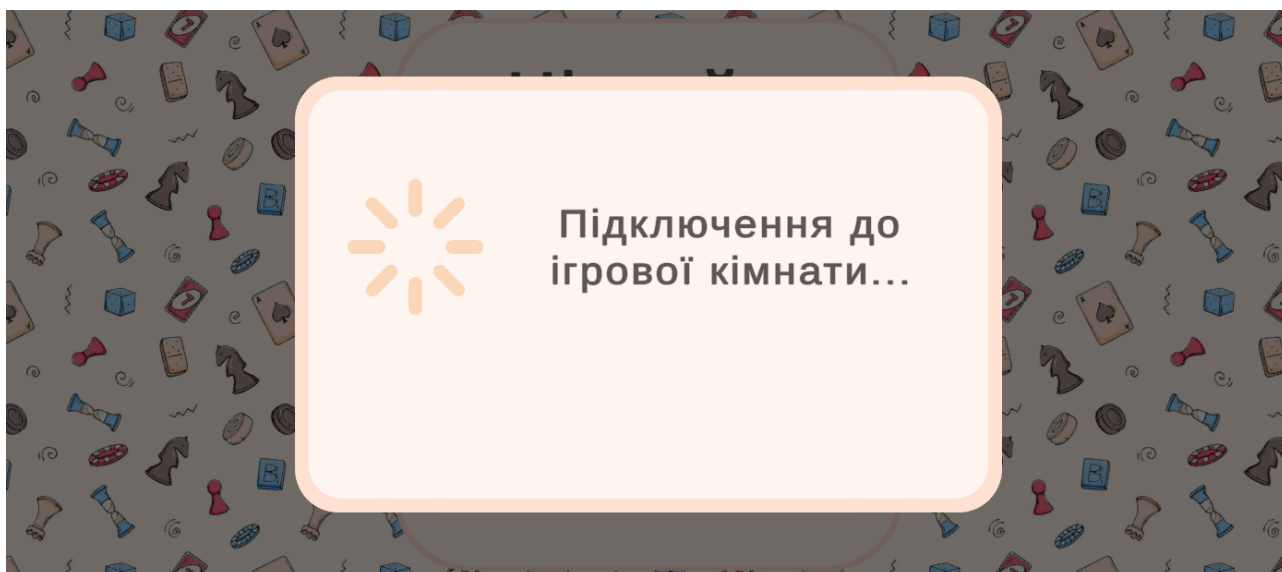


Рисунок 3.6 – Процес підключення новоствореного лобі

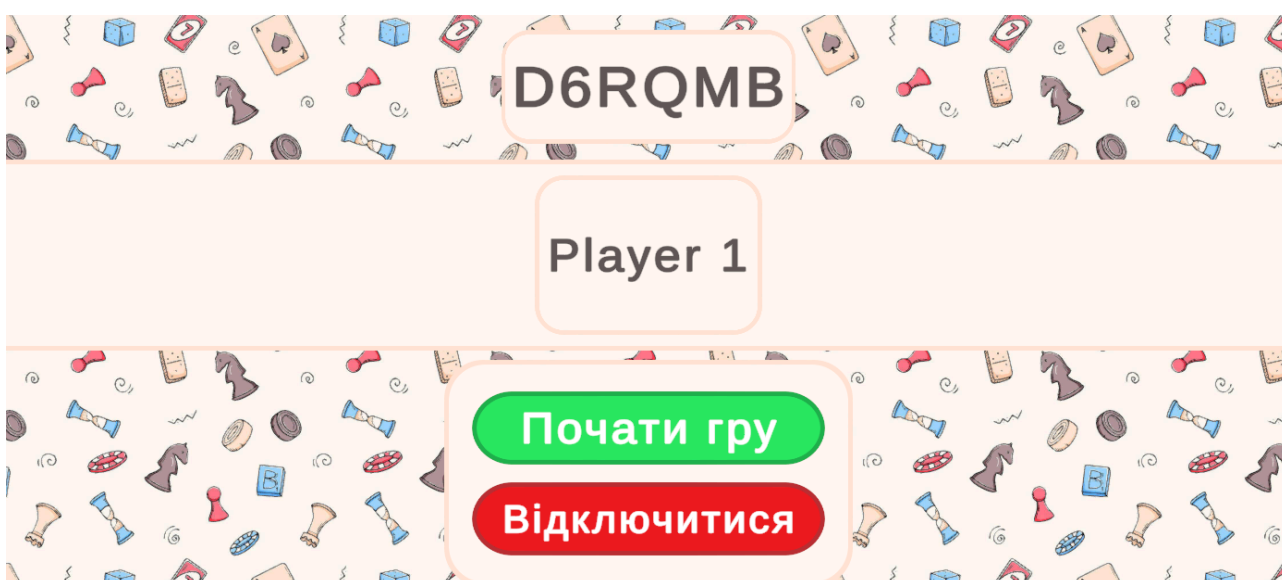


Рисунок 3.7 – Гравець, який перебуває у лобі та є хостом

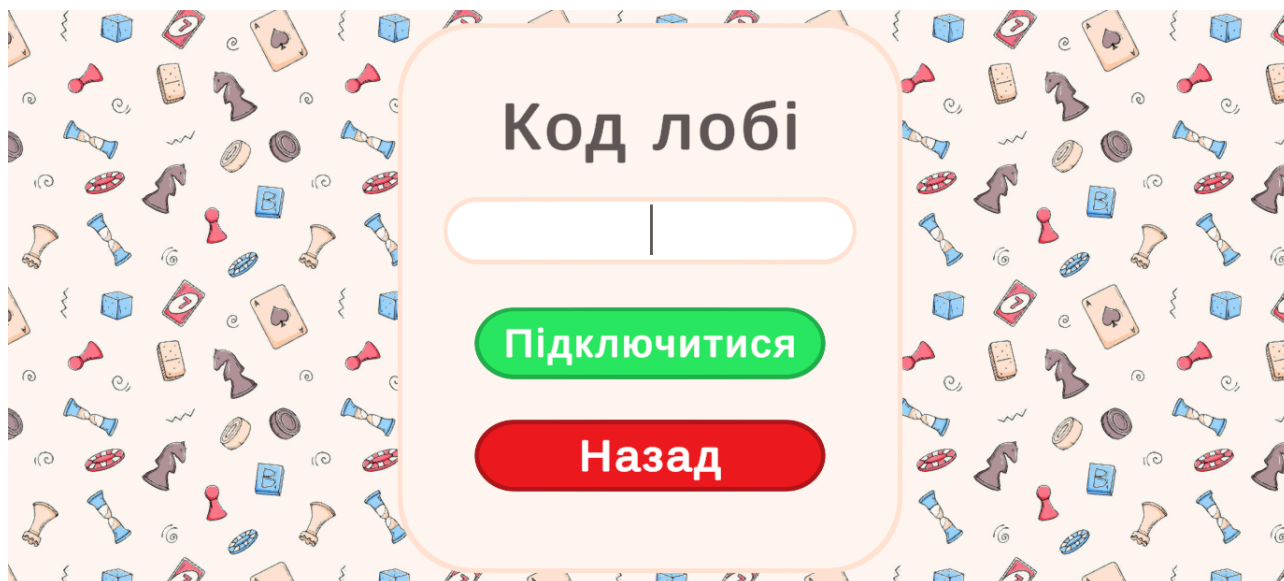


Рисунок 3.8 – Вікно введення коду лобі

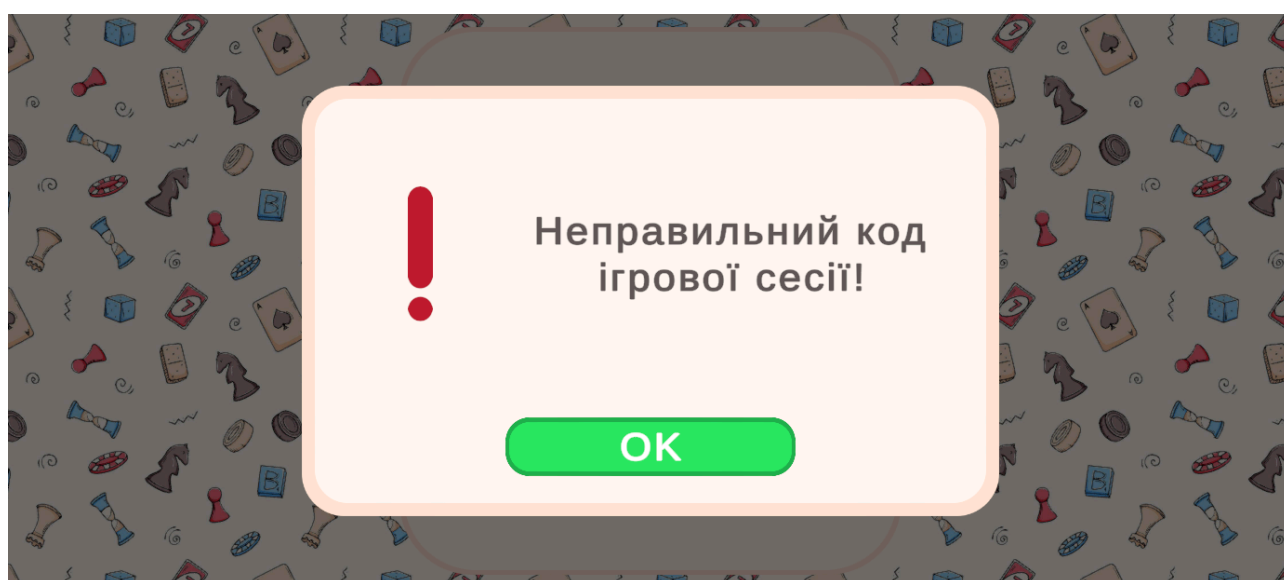


Рисунок 3.9 – Повідомлення у випадку неправильно введеного коду ігрової сесії



Рисунок 3.10 – Обидва гравці знаходяться у лобі



Рисунок 3.11 – Гра завантажилася і гравцю випала черга робити хід

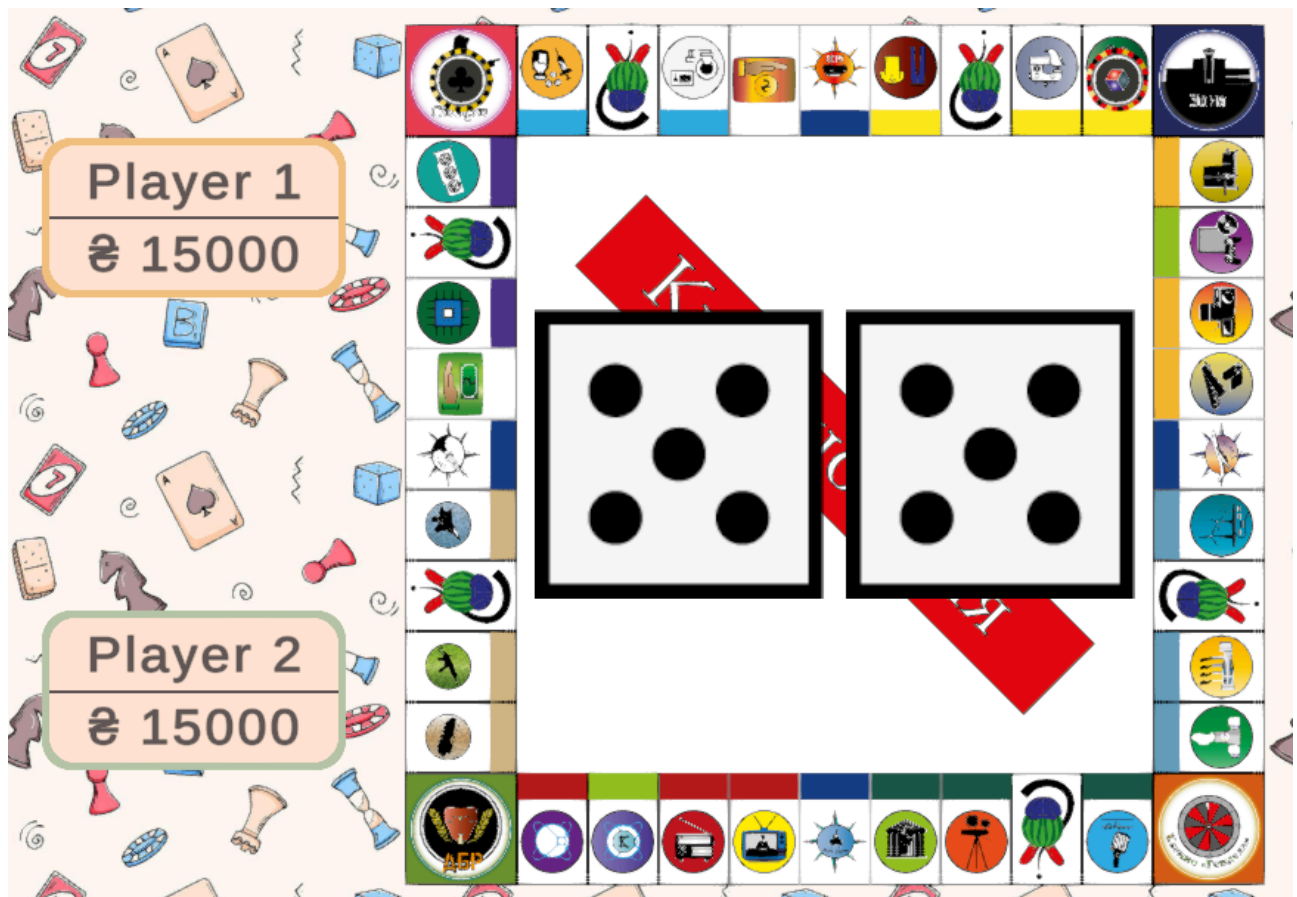


Рисунок 3.12 – Гравець виконує хід, показуються випавші значення кубиків



Рисунок 3.13 – Гравець приземлився на поле власності і йому було запропоновано придбати поле



Рисунок 3.14 – Гравець погодився з пропозицією і власність поля було успішно оновлено, також було списано відповідно суму з балансу поточного користувача



Рисунок 3.15 – Гравець запропонував здійснити обмін з іншим



Рисунок 3.16 – Іншому гравцю прийшла пропозиція обміну

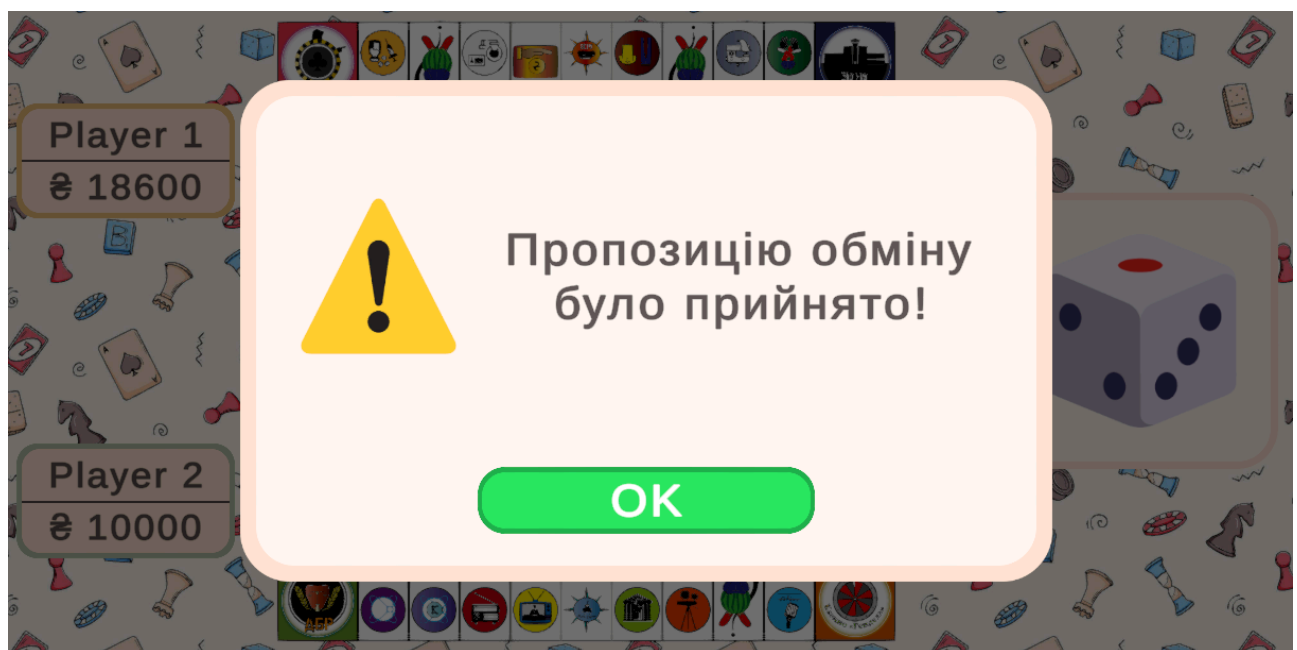


Рисунок 3.17 – Відправнику обміну прийшло повідомлення, що інший користувач успішно прийняв його пропозицію обміну



Рисунок 3.18 – Обмін було успішно здійснено, було оновлено власника поля та баланс гравців

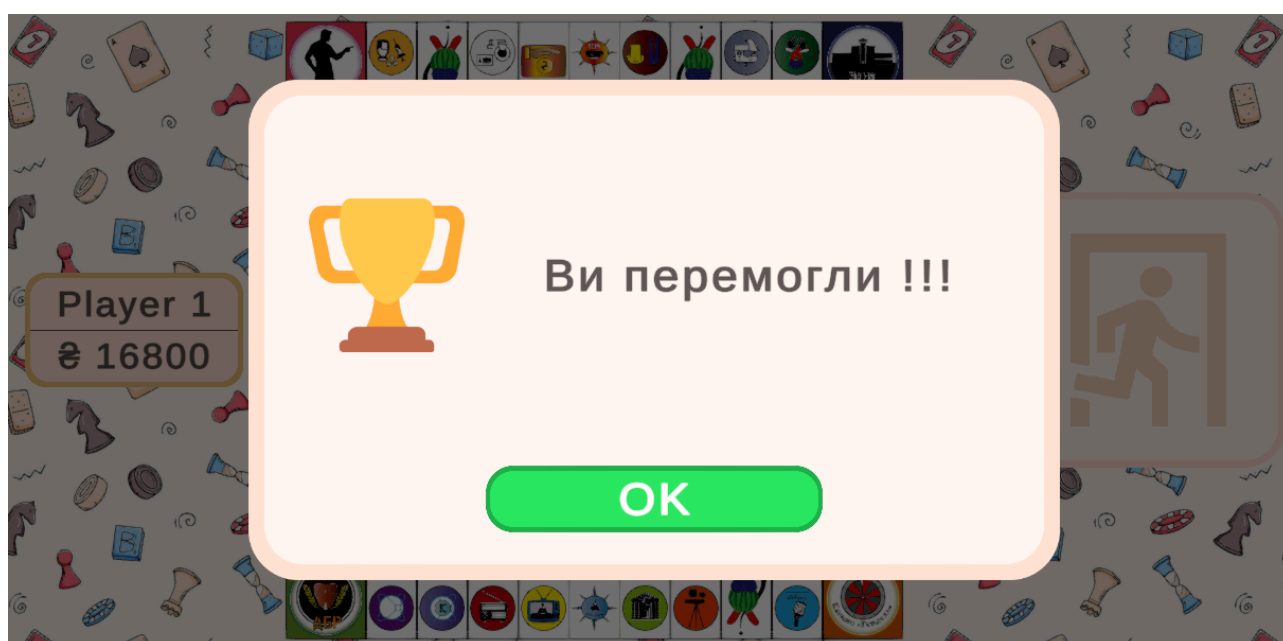


Рисунок 3.19 – Після добровільної здачі іншого гравця поточний користувач переміг у грі

Висновки до розділу

Аналіз якості програмного забезпечення за певними метриками дозволяє здійснити комплексне оцінювання різних аспектів продукту, що включає в себе якість коду, ефективність використання ресурсів, загальну якість архітектури, якість написаного та відповідність вимогам. Проте варто зазначити, що оцінка якості коду часто є суб'єктивною. Розробники можуть мати різні погляди на те, що вважається гарними підходами до використання певних синтаксичних конструкцій. Оцінки можуть сильно варіюватися в залежності від досвіду, стилю програмування та особистих уподобань. Також якість коду може залежати від контексту і вимог проекту. Те, що вважається гарним рішенням для одного проекту, може бути несхожим на іншому, залежно від специфіки завдань та потреб користувачів. Цей аспект є доволі актуальним для розробки ігор загалом, а особливо для багатокористувацьких проєктів, оскільки у них може використовуватися багато дублюючого коду синхронізації у неткодї, або код для створення редакторів певних компонент.

Тестування програмного забезпечення дозволяє визначити й покращити його якість, включаючи аспекти ефективності, відповідність вимогам та загальну надійність. Під час опису процесу тестування було вирішено використовувати функціональні тести, за допомогою яких відбувається перевірка головних функціональних вимог застосунку, які результати проведення цих тестів потім будуть продемонстровані під час контрольного тестування.

Контрольний приклад допомагає оцінити та підтвердити правильність роботи конкретної функціональності чи компоненту в програмному забезпеченні, забезпечуючи впевненість у його надійності та відповідності встановленим критеріям. Саме тому у розділі був наданий повний опис контрольного прикладу з усіма можливими розгалуженнями та ілюстраціями, які демонструють відповідні взаємодії, які були зазначені під час процесу тестування.

4 ВПРОВАДЖЕННЯ ТА СУПРОВІД ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

4.1 Розгортання програмного забезпечення

Оскільки розробленим програмним забезпеченням є мобільна гра з відкритим вихідним кодом, яка розроблена на платформу Android та під час розробки якої було використано нові бібліотеки та рішення від Unity, тому було прийнято рішення розміщувати її на платформі GitHub. Це було зроблено для того, щоб надати можливість іншим розробникам ознайомитися з вихідним кодом, на основі якого вони зможуть створювати свої рішення. Звичайний користувач має можливість завантажити вже готовий білд за наступним посиланням: <https://github.com/NgeNXQ/Monopoly/releases/tag/release>.

Процес розгортання для мобільної платформи готового білду стандартний. Для розгортання потрібно виконати наступні кроки:

- Перейти за відповідним посиланням на GitHub-репозиторій у вкладку релізи.
- Завантажити останній реліз у вигляді .apk-файлу.
- Натиснути на завантажений .apk-файл.
- Погодитися на встановлення гри і дочекатися завершення цього процесу.

У випадку успішного процесу встановлення у користувача з'явиться новий застосунок з відповідною зображенням, як на рисунку 4.1.



Рисунок 4.1 – Завантажена та встановлена гра на пристрої користувача

4.2 Підтримка програмного забезпечення

Нові версії програмного забезпечення будуть доступні для завантаження за тим же посиланням, за яким можна знайти і початковий реліз.

Окремо варто зауважити, що у випадку, якщо гравці намагатимуться зіграти з різними версіями програмного забезпечення, то як правило, гравці не зможуть приєднатися до спільної ігрової сесії, але можлива і непередбачувана поведінка.

Висновки до розділу

У даному розділі було детально висвітлено повний опис покрокового розгортання програмного забезпечення та опис підтримки цього застосунку. Перший підрозділ включає повний опис процесу завантаження та інсталювання гри для мобільної платформи Android.

Додатково був описаний процес підтримки програмного забезпечення. Крім того, важливим аспектом є додавання води у код для забезпечення оптимального функціонування програми. Згідно з цим підходом, користувач здатен буде завантажити нову версію гри за тим же посиланням на тому ж GitHub-репозиторії, забезпечуючи безперебійну роботу та вдосконалену функціональність.

ВИСНОВКИ

У результаті виконання курсової роботи було створено мобільний ігровий застосунок, який є реалізацією відомої настільної гри «Монополія». Розроблене програмне забезпечення було реалізовано з використанням найсучасніших технічних рішень та практик.

Під час реалізації курсової роботи було спроектовано мобільну багатокористувацьку гру, яка використовує одноранговий децентралізований тип мережі для побудови взаємодії між користувачами. Основним інструментом для створення гри виступило середовище розробки Unity та офіційне рішення від Unity для створення багатокористувацьких ігор – Netcode For GameObject.

Ключова особливість розробленого проєкту полягає у фактичній унікальності набору технічних інструментів, які використовувалися під час розробки мобільного ігрового застосунку з багатокористувацьким режимом, а також в унікальній реалізації відомої настільної гри з особливими правилами гри, які спеціально адаптовані для більш швидких ігрових сесій, що особливо важливо для мобільних пристроїв.

Під час реалізації проєкту особлива увага приділялася не лише технічним аспектам, але й вдосконаленню геймплейного досвіду користувачів. Важливо відзначити, що створений ігровий застосунок не лише відтворює класичний варіант гри, але і пропонує унікальні правила, створені спеціально для збільшення динаміки ігрового процесу.

Після реалізації застосунку він був протестований на пристроях з різними версіями Android, з різними розмірами екранів щоб переконатися, що додаток коректно відображається на різних пристроях з різними технічними показниками.

У зв'язку з вищезазначеним, вважається доцільним продовження розвитку цього проєкту з метою удосконалення і розширення функціоналу гри, а також її адаптації для інших платформ та операційних систем. Такі кроки сприятимуть подальшому розвитку та вдосконаленню з урахуванням сучасних тенденцій та потреб користувачів.

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

1. Світова історія ігор [Електронний ресурс] – Режим доступу:
<https://www.interaction-design.org/literature/article/a-brief-history-of-games>
2. Історія ігор всіх видів [Електронний ресурс] – Режим доступу:
<https://www.kaloumba.com/en/history-of-games/>
3. Історія сучасної ігрової індустрії [Електронний ресурс] – Режим доступу:
<https://www.redbull.com/us-en/history-of-video-games-design>
4. Історія відеоігор 20 століття [Електронний ресурс] – Режим доступу:
<https://www.history.com/topics/inventions/history-of-video-games>
5. MVC-паттерн [Електронний ресурс] – Режим доступу:
<https://www.geeksforgeeks.org/mvc-design-pattern/>
6. ECS-паттерн [Електронний ресурс] – Режим доступу:
<https://www.guru99.com/entity-component-system.html>
7. Мережева архітектура [Електронний ресурс] – Режим доступу:
<https://www.cisco.com/c/en/us/solutions/enterprise-networks/what-is-network-architecture.html>
8. P2P архітектура мережі [Електронний ресурс] – Режим доступу:
<https://www.servers.com/news/blog/differences-between-peer-to-peer-and-dedicated-game-server-hosting>
9. UDP-протокол [Електронний ресурс] – Режим доступу:
<https://www.geeksforgeeks.org/user-datagram-protocol-udp/>
10. DTLS-протокол [Електронний ресурс] – Режим доступу:
<https://datatracker.ietf.org/doc/html/rfc6347>
11. DTLS загальні положення [Електронний ресурс] – Режим доступу:
<https://www.ringcentral.com/gb/en/blog/definitions/dtls-datagram-transport-layer-security/>
12. Monopoly Board Game [Електронний ресурс] – Режим доступу:
<https://www.britannica.com/sports/Monopoly-board-game>.
13. Unity scripting API [Електронний ресурс] – Режим доступу:
<https://docs.unity3d.com/ScriptReference/>

14. Unity networking API [Електронний ресурс] – Режим доступу:
<https://docs-multiplayer.unity3d.com/>
15. Unity Netcode For GameObjects [Електронний ресурс] – Режим доступу:
<https://docs-multiplayer.unity3d.com/netcode/current/tutorials/get-started-ngo/>
16. Game architecture [Електронний ресурс] – Режим доступу:
<https://www.studytonight.com/3d-game-engineering-with-unity/game-development-architecture>
17. Game programming patterns [Електронний ресурс] – Режим доступу:
<https://gameprogrammingpatterns.com/architecture-performance-and-games.html>
18. Understanding netcode [Електронний ресурс] – Режим доступу:
<https://www.gamereplays.org/overwatch/portals.php?show=page&name=overwatch-a-guide-to-understanding-netcode>
19. Mirror Networking [Електронний ресурс] – Режим доступу:
<https://mirror-networking.gitbook.io/docs/>
20. Fish-Net Networking [Електронний ресурс] – Режим доступу:
<https://fish-networking.gitbook.io/docs/>
21. RPC [Електронний ресурс] – Режим доступу:
<https://www.techtarget.com/searchapparchitecture/definition/Remote-Procedure-Call-RPC>
22. Unity GUI [Електронний ресурс] – Режим доступу:
<https://blog.yarsalabs.com/gui-elements-in-unity/>
23. Unity розгортання на Android [Електронний ресурс] – Режим доступу:
<https://developer.android.com/games/engines/unity/build-in-unity>
24. Client-Server архітектура [Електронний ресурс] – Режим доступу:
<https://www.simplilearn.com/what-is-client-server-architecture-article>
25. Unity обробник подій [Електронний ресурс] – Режим доступу:
<https://docs.unity3d.com/Packages/com.unity.ugui@1.0/manual/EventSystem.html>

Факультет інформатики та обчислювальної техніки

Кафедра інформатики та програмної інженерії

“ЗАТВЕРДЖЕНО”

Керівник роботи

_____ Максим ГОЛОВЧЕНКО

“ ” _____ 2024 р.

ГРА ДЛЯ МОБІЛЬНОГО ПРИСТРОЮ «МОНОПОЛІЯ»

Текст програми

КПІ.ІІ-1402.045490.03.12

“ПОГОДЖЕНО”

Керівник роботи:

_____ Максим ГОЛОВЧЕНКО

Консультант:

_____ Максим ГОЛОВЧЕНКО

Виконавець:

_____ Денис БАБІЧ

Київ – 2024

Файл GameCoordinator.cs

```

using System;
using UnityEngine;
using Unity.Netcode;
using Unity.Services.Core;
using Unity.Services.Relay;
using System.Threading.Tasks;
using Unity.Services.Lobbies;
using System.Collections.Generic;
using UnityEngine.SceneManagement;
using Unity.Services.Relay.Models;
using Unity.Netcode.Transports.UTP;
using Unity.Services.Authentication;
using Unity.Services.Lobbies.Models;
using Unity.Networking.Transport.Relay;

#if UNITY_EDITOR
using ParrelSync;
#endif

internal sealed class GameCoordinator : MonoBehaviour
{
    public enum MonopolyScene : byte
    {
        Bootstrap,
        MainMenu,
        GameLobby,
        MonopolyGame
    }

    private const string CONNECTION_TYPE = "dtls";

    private Scene activeScene;

    private int initializationCount;

    private LinkedList<Type> objectsToLoad;

    private LinkedList<Type> initializedObjects;

    public static GameCoordinator Instance { get; private set; }

    public event Action OnAuthenticationFailed;

```

```

    public event Action<RelayServiceException>
OnEstablishingConnectionRelayFailed;

    public event Action<LobbyServiceException>
OnEstablishingConnectionLobbyFailed;

    public Player LocalPlayer { get; private set; }

    public MonopolyScene ActiveScene { get; private set; }

    private void Awake()
    {
        if (Instance != null)
            throw new System.InvalidOperationException($"Singleton
{this.GetType().FullName} has already been initialized.");

        Instance = this;
        UnityEngine.Object.DontDestroyOnLoad(this.gameObject);
    }

    private void OnEnable()
    {
        SceneManager.activeSceneChanged += this.HandleActiveSceneChanged;
    }

    private void OnDisable()
    {
        SceneManager.activeSceneChanged -= this.HandleActiveSceneChanged;
    }

    private async void Start()
    {
        this.objectsToLoad = new LinkedList<Type>();
        this.initializedObjects = new LinkedList<Type>();

        try
        {
#if UNITY_EDITOR
            InitializationOptions options = new InitializationOptions();
            options.SetProfile(ClonesManager.IsClone() ?
ClonesManager.GetArgument() : "Primary");
            await UnityServices.InitializeAsync(options);
#else
            await UnityServices.InitializeAsync();
#endif
        }
    }

```



```

        await AuthenticationService.Instance.SignInAnonymouslyAsync();

this.InitializeLocalPlayer(PlayerPrefs.GetString(LobbyManager.KEY_PLAYER_
NICKNAME));
    }
    catch
    {
        this.OnAuthenticationFailed?.Invoke();
        return;
    }

    await this.LoadSceneAsync(GameCoordinator.MonopolyScene.MainMenu);
}

#region Updating Player

public void UpdateLocalPlayer(string newNickname)
{
    newNickname = newNickname.Trim();

    this.LocalPlayer.Data[LobbyManager.KEY_PLAYER_NICKNAME].Value =
newNickname;

    PlayerPrefs.SetString(LobbyManager.KEY_PLAYER_NICKNAME,
newNickname);
    PlayerPrefs.Save();
}

public void InitializeLocalPlayer(string nickname)
{
    nickname = nickname.Trim();

    Player player = new Player(AuthenticationService.Instance.PlayerId)
    {
        Data = new Dictionary<string, PlayerDataObject>
        {
            { LobbyManager.KEY_PLAYER_NICKNAME, new
PlayerDataObject(PlayerDataObject.VisibilityOptions.Member, nickname) },
            { LobbyManager.KEY_PLAYER_SCENE, new
PlayerDataObject(PlayerDataObject.VisibilityOptions.Member,
GameCoordinator.Instance.ActiveScene.ToString()) }
        }
    };
};

```

```

        PlayerPrefs.SetString(LobbyManager.KEY_PLAYER_NICKNAME,
nickname);
        PlayerPrefs.Save();

        this.LocalPlayer = player;
    }

#endregion

#region Scenes Management

public void LoadSceneNetwork(MonopolyScene scene)
{
    NetworkManager.Singleton.SceneManager.LoadScene(scene.ToString(),
LoadSceneMode.Single);
}

public async Task LoadSceneAsync(MonopolyScene scene)
{
    await SceneManager.LoadSceneAsync(scene.ToString(),
LoadSceneMode.Single);
}

public void UpdateInitializedObjects(Type gameObject)
{
    if (this.objectsToLoad == null)
    {
        throw new System.InvalidOperationException($"You have to call
{nameof(this.SetupInitializedObjects)} at first.");
    }

    if (!this.objectsToLoad.Contains(gameObject))
    {
        throw new System.ArgumentException($" {nameof(gameObject)} is not in
{nameof(this.SetupInitializedObjects)}.");
    }

    if (this.initializedObjects.Contains(gameObject))
    {
        throw new System.ArgumentException($" {nameof(gameObject)} has
already been initialized.");
    }

    this.initializedObjects.AddLast(gameObject);

```

```

        if (this.initializedObjects.Count == this.objectsToLoad.Count)
        {
            LobbyManager.Instance?.UpdateLocalPlayerData();
        }
    }

    public void SetupInitializedObjects(params Type[] gameObjectsToLoad)
    {
        foreach (Type gameObject in gameObjectsToLoad)
        {
            this.objectsToLoad.AddLast(gameObject);
        }
    }

    private void HandleActiveSceneChanged(Scene previousActiveScene, Scene
newActiveScene)
    {
        this.objectsToLoad?.Clear();
        this.initializedObjects?.Clear();

        this.activeScene = SceneManager.GetActiveScene();

        switch (newActiveScene.name)
        {
            case nameof(GameCoordinator.MonopolyScene.MainMenu):
                this.ActiveScene = GameCoordinator.MonopolyScene.MainMenu;
                break;
            case nameof(GameCoordinator.MonopolyScene.GameLobby):
                {
                    this.ActiveScene = GameCoordinator.MonopolyScene.GameLobby;

                    this.SetupInitializedObjects(typeof(UIManagerGameLobby),
typeof(ObjectPoolPanelPlayerLobby));

                    LobbyManager.Instance?.OnGameLobbyLoaded?.Invoke();
                }
                break;
            case nameof(GameCoordinator.MonopolyScene.MonopolyGame):
                {
                    this.ActiveScene =
GameCoordinator.MonopolyScene.MonopolyGame;

                    this.SetupInitializedObjects(typeof(GameManager),
typeof(MonopolyBoard), typeof(UIManagerMonopolyGame));
                }
            }
        }
    }

```

```

        LobbyManager.Instance?.OnMonopolyGameLoaded?.Invoke();
    }
    break;
}
}

#endregion

#region Establishing Connection

public async Task HostLobbyAsync()
{
    if (this.LocalPlayer == null)
    {
        throw new
System.InvalidOperationException($" {nameof(this.LocalPlayer)} is null.");
    }

    try
    {
        Allocation hostAllocation = await
RelayService.Instance.CreateAllocationAsync(LobbyManager.MAX_PLAYERS);

        RelayServerData relayServerData = new RelayServerData(hostAllocation,
GameCoordinator.CONNECTION_TYPE);

NetworkManager.Singleton?.GetComponent<UnityTransport>().SetRelayServerD
ata(relayServerData);

        string relayCode = await
RelayService.Instance.GetJoinCodeAsync(hostAllocation.AllocationId);

        await LobbyManager.Instance?.HostLobbyAsync(relayCode);
    }
    catch (RelayServiceException relayServiceException)
    {
        this.OnEstablishingConnectionRelayFailed?.Invoke(relayServiceException);
    }
    catch (LobbyServiceException lobbyServiceException)
    {
        this.OnEstablishingConnectionLobbyFailed?.Invoke(lobbyServiceException);
    }
}

```

```

    }

    public async Task ConnectLobbyAsync(string joinCode)
    {
        if (this.LocalPlayer == null)
        {
            throw new
System.InvalidOperationException($"{nameof(this.LocalPlayer)} is null.");
        }

        try
        {
            JoinAllocation clientAllocation = await
RelayService.Instance.JoinAllocationAsync(joinCode);

            RelayServerData relayServerData = new RelayServerData(clientAllocation,
GameCoordinator.CONNECTION_TYPE);

NetworkManager.Singleton?.GetComponent<UnityTransport>().SetRelayServerD
ata(relayServerData);

            await LobbyManager.Instance?.ConnectLobbyAsync(joinCode);
        }
        catch (RelayServiceException relayServiceException)
        {
            this.OnEstablishingConnectionRelayFailed?.Invoke(relayServiceException);
        }
        catch (LobbyServiceException lobbyServiceException)
        {
            this.OnEstablishingConnectionLobbyFailed?.Invoke(lobbyServiceException);
        }
    }

    #endregion
}

```

Файл LobbyManager.cs

```

using System;
using System.Linq;
using UnityEngine;
using Unity.Netcode;
using System.Collections;
using System.Threading.Tasks;
using Unity.Services.Lobbies;
using System.Collections.Generic;
using Unity.Services.Lobbies.Models;

internal sealed class LobbyManager : MonoBehaviour
{
    private const float LOBBY_UPTIME = 25.0f;

    public const int MIN_PLAYERS = 2;

    public const int MAX_PLAYERS = 5;

    public const float LOBBY_LOADING_TIMEOUT = 15.0f;

    public const string KEY_PLAYER_SCENE = "Scene";

    public const string KEY_PLAYER_NICKNAME = "Nickname";

    public const string KEY_LOBBY_STATE = "State";

    public const string LOBBY_STATE_GAME = "Game";

    public const string LOBBY_STATE_LOBBY = "Lobby";

    public const string LOBBY_STATE_LOADING = "Loading";

    public const string LOBBY_STATE_PENDING = "Waiting";

    public const string LOBBY_STATE_RETURNING = "Returning";

    private string lobbyName
    {
        get => $"LOBBY_{this.JoinCode}";
    }

    private ILobbyEvents localLobbyEvents;

```

```

private QueryLobbiesOptions queryCurrentLobby
{
    get
    {
        return new QueryLobbiesOptions()
        {
            Filters = new List<QueryFilter>()
            {
                new QueryFilter(QueryFilter.FieldOptions.Name, this.JoinCode,
QueryFilter.OpOptions.CONTAINS)
            }
        };
    }
}

public static LobbyManager Instance { get; private set; }

public Action OnGameLobbyLoaded;

public Action OnMonopolyGameLoaded;

public Action OnGameLobbyFailedToLoad;

public Action OnMonopolyGameFailedToLoad;

public bool IsHost { get; private set; }

public string JoinCode { get; private set; }

public bool HasHostLeft { get; private set; }

public Lobby LocalLobby { get; private set; }

public bool HavePlayersLoaded
{
    get
    {
        return this.LocalLobby != null ? this.LocalLobby.Players.All(player =>
player.Data[LobbyManager.KEY_PLAYER_SCENE].Value.Equals(GameCoordin
ator.Instance.ActiveScene.ToString(), StringComparison.Ordinal)) : false;
    }
}

public bool HasLocalPlayerLeft { get; private set; }

```

```

public LobbyEventCallbacks LocalLobbyEventCallbacks { get; private set; }

private void Awake()
{
    if (Instance != null)
        throw new System.InvalidOperationException($"Singleton
{this.GetType().FullName} has already been initialized.");

    Instance = this;
    UnityEngine.Object.DontDestroyOnLoad(this.gameObject);
}

private void OnEnable()
{
    this.LocalLobbyEventCallbacks = new LobbyEventCallbacks();

    this.OnGameLobbyLoaded += this.HandleGameLobbyLoaded;
    this.OnMonopolyGameLoaded += this.HandleMonopolyGameLoaded;
    this.OnGameLobbyFailedToLoad +=
this.HandleGameLobbyFailedToLoadAsync;
    this.OnMonopolyGameFailedToLoad +=
this.HandleMonopolyGameFailedToLoad;

    this.LocalLobbyEventCallbacks.PlayerLeft += this.HandlePlayerLeft;
    this.LocalLobbyEventCallbacks.DataChanged += this.HandleDataChanged;
    this.LocalLobbyEventCallbacks.LobbyDeleted += this.HandleLobbyDeleted;
    this.LocalLobbyEventCallbacks.PlayerJoined += this.HandlePlayerJoined;
    this.LocalLobbyEventCallbacks.PlayerDataChanged +=
this.HandlePlayerDataChanged;
    this.LocalLobbyEventCallbacks.KickedFromLobby +=
this.HandleKickedFromLobbyAsync;

    NetworkManager.Singleton.OnTransportFailure +=
this.HandleTransportFailureAsync;
}

private void OnDisable()
{
    this.LocalLobbyEventCallbacks = new LobbyEventCallbacks();

    this.OnGameLobbyLoaded -= this.HandleGameLobbyLoaded;
    this.OnMonopolyGameLoaded -= this.HandleMonopolyGameLoaded;
    this.OnGameLobbyFailedToLoad -=
this.HandleGameLobbyFailedToLoadAsync;

```



```

        this.OnMonopolyGameFailedToLoad -=
this.HandleMonopolyGameFailedToLoad;

        this.LocalLobbyEventCallbacks.PlayerLeft -= this.HandlePlayerLeft;
        this.LocalLobbyEventCallbacks.DataChanged -= this.HandleDataChanged;
        this.LocalLobbyEventCallbacks.LobbyDeleted -= this.HandleLobbyDeleted;
        this.LocalLobbyEventCallbacks.PlayerJoined -= this.HandlePlayerJoined;
        this.LocalLobbyEventCallbacks.PlayerDataChanged -=
this.HandlePlayerDataChanged;
        this.LocalLobbyEventCallbacks.KickedFromLobby -=
this.HandleKickedFromLobbyAsync;

        if (NetworkManager.Singleton != null)
        {
            NetworkManager.Singleton.OnTransportFailure -=
this.HandleTransportFailureAsync;
        }
    }

    private async void OnDestroy()
    {
        if (this.IsHost)
        {
            this.StopCoroutine(this.PingLobbyCoroutine());
        }

        if (this.LocalLobby != null)
        {
            await this.DisconnectFromLobbyAsync();
        }
    }

    #region Start & End Game

    public void StartGameAsync()
    {
        if (!this.HavePlayersLoaded)
        {
            UIManagerGlobal.Instance.ShowMessageBox(PanelMessageBoxUI.Type.OK,
            UIManagerGameLobby.Instance.MessageNotAllPlayersLoaded,
            PanelMessageBoxUI.Icon.Warning);
            return;
        }
    }

```

```

        if (this.LocalLobby.Players.Count < LobbyManager.MIN_PLAYERS)
        {

            UIManagerGlobal.Instance.ShowMessageBox(PanelMessageBoxUI.Type.OK,
            UIManagerGameLobby.Instance.MessageTooFewPlayers,
            PanelMessageBoxUI.Icon.Warning);
            return;
        }

        this.UpdateLocalLobbyData(LobbyManager.LOBBY_STATE_LOADING,
        true);

        UIManagerGlobal.Instance.ShowMessageBox(PanelMessageBoxUI.Type.None,
        UIManagerGameLobby.Instance.MessagePendingGame,
        PanelMessageBoxUI.Icon.Loading);

        GameCoordinator.Instance.LoadSceneNetwork(GameCoordinator.MonopolyScene
        .MonopolyGame);
    }

    #endregion

    #region Lobby API

    private async Task LeaveLobbyAsync()
    {

        UIManagerGlobal.Instance.ShowMessageBox(PanelMessageBoxUI.Type.None,
        UIManagerGameLobby.Instance?.MessageDisconnecting ??
        UIManagerMainMenu.Instance?.MessageDisconnecting,
        PanelMessageBoxUI.Icon.Loading);

        NetworkManager.Singleton?.Shutdown();

        if (this != null)
        {
            await this.localLobbyEvents?.UnsubscribeAsync();
        }

        await
        GameCoordinator.Instance?.LoadSceneAsync(GameCoordinator.MonopolyScene.
        MainMenu);
    }

```

```

        if (!this.IsHost)
        {
            if (this.HasHostLeft)
            {

                UIManagerGlobal.Instance.ShowMessageBox(PanelMessageBoxUI.Type.OK,
                UIManagerGameLobby.Instance?.MessageHostDisconnected ??
                UIManagerMainMenu.Instance?.MessageHostDisconnected,
                PanelMessageBoxUI.Icon.Error);
            }
            else if (!this.HasLocalPlayerLeft)
            {

                UIManagerGlobal.Instance.ShowMessageBox(PanelMessageBoxUI.Type.OK,
                UIManagerGameLobby.Instance?.MessageKicked ??
                UIManagerMainMenu.Instance?.MessageKicked,
                PanelMessageBoxUI.Icon.Error);
            }
        }

        this.IsHost = false;
        this.LocalLobby = null;
        this.HasHostLeft = false;
        this.HasLocalPlayerLeft = false;
    }

    public async Task DisconnectFromLobbyAsync()
    {
        this.HasLocalPlayerLeft = true;

        if (this.IsHost)
        {
            this.StopCoroutine(this.PingLobbyCoroutine());
            await LobbyService.Instance.DeleteLobbyAsync(this.LocalLobby.Id);
        }
        else
        {
            await LobbyService.Instance.RemovePlayerAsync(this.LocalLobby.Id,
            GameCoordinator.Instance.LocalPlayer.Id);
        }
    }

    public async Task HostLobbyAsync(string relayCode)
    {
        this.IsHost = true;
    }

```

```

this.HasHostLeft = false;
this.JoinCode = relayCode;
this.HasLocalPlayerLeft = false;

CreateLobbyOptions lobbyOptions = new CreateLobbyOptions()
{
    Player = GameCoordinator.Instance.LocalPlayer,

    Data = new Dictionary<string, DataObject>()
    {
        { LobbyManager.KEY_LOBBY_STATE, new
DataObject(DataObject.VisibilityOptions.Member,
LobbyManager.LOBBY_STATE_LOBBY) }
    }
};

try
{
    this.LocalLobby = await
LobbyService.Instance.CreateLobbyAsync(this.lobbyName,
LobbyManager.MAX_PLAYERS, lobbyOptions);
    this.localLobbyEvents = await
LobbyService.Instance.SubscribeToLobbyEventsAsync(this.LocalLobby.Id,
this.LocalLobbyEventCallbacks);

    NetworkManager.Singleton?.StartHost();
}
catch (LobbyServiceException lobbyServiceException)
{
    throw lobbyServiceException;
}

if (this != null)
{
    this.StartCoroutine(this.PingLobbyCoroutine());
    await
GameCoordinator.Instance.LoadSceneAsync(GameCoordinator.MonopolyScene.G
ameLobby);
}
}

public async Task ConnectLobbyAsync(string joinCode)
{
    this.IsHost = false;
    this.HasHostLeft = false;

```

```

this.JoinCode = joinCode;
this.HasLocalPlayerLeft = false;

JoinLobbyByIdOptions joinOptions = new JoinLobbyByIdOptions()
{
    Player = GameCoordinator.Instance.LocalPlayer
};

try
{
    QueryResponse queryResponse = await
Lobbies.Instance.QueryLobbiesAsync(this.queryCurrentLobby);
    this.LocalLobby = await
LobbyService.Instance.JoinLobbyByIdAsync(queryResponse.Results.FirstOrDefault().Id, joinOptions);
    this.localLobbyEvents = await
LobbyService.Instance.SubscribeToLobbyEventsAsync(this.LocalLobby.Id,
this.LocalLobbyEventCallbacks);

    NetworkManager.Singleton?.StartClient();
}
catch (LobbyServiceException lobbyServiceException)
{
    throw lobbyServiceException;
}
catch (NullReferenceException nullReferenceException)
{
    throw new
LobbyServiceException(LobbyExceptionReason.InvalidJoinCode, "Invalid Join
Code.", nullReferenceException);
}
}

public async Task KickFromLobbyAsync(string playerId)
{
    await LobbyService.Instance.RemovePlayerAsync(this.LocalLobby.Id,
playerId);
}

#endregion

#region Lobby Ping

private IEnumerator PingLobbyCoroutine()
{

```

```

        WaitForSeconds waitForSeconds = new
        WaitForSeconds(LobbyManager.LOBBY_UPTIME);

        while (this.LocalLobby != null)
        {
            Lobbies.Instance.SendHeartbeatPingAsync(this.LocalLobby?.Id);
            yield return waitForSeconds;
        }
    }

#endregion

#region Lobby Update

    public async void UpdateLocalPlayerData()
    {
        GameCoordinator.Instance.LocalPlayer.Data[LobbyManager.KEY_PLAYER_SCENE] = new PlayerDataObject(PlayerDataObject.VisibilityOptions.Member,
        GameCoordinator.Instance.ActiveScene.ToString());

        UpdatePlayerOptions updatePlayerOptions = new UpdatePlayerOptions()
        {
            Data = GameCoordinator.Instance.LocalPlayer.Data
        };

        this.LocalLobby = await
        LobbyService.Instance.UpdatePlayerAsync(this.LocalLobby.Id,
        GameCoordinator.Instance.LocalPlayer.Id, updatePlayerOptions);
    }

    public async void UpdateLocalLobbyData(string lobbyState, bool isPrivate =
    true)
    {
        this.LocalLobby.Data[LobbyManager.KEY_LOBBY_STATE] = new
        DataObject(DataObject.VisibilityOptions.Member, lobbyState);

        UpdateLobbyOptions updateLobbyOptions = new UpdateLobbyOptions()
        {
            IsPrivate = isPrivate,
            Data = this.LocalLobby.Data
        };

        await Lobbies.Instance.UpdateLobbyAsync(this.LocalLobby.Id,
        updateLobbyOptions);
    }

```

```

    }

    #endregion

    #region Lobby Callbacks

    private void HandleLobbyDeleted()
    {
        this.HasHostLeft = true;
    }

    private async void HandleKickedFromLobbyAsync()
    {
        await this.LeaveLobbyAsync();
    }

    private async void HandleTransportFailureAsync()
    {
        await LobbyManager.Instance.DisconnectFromLobbyAsync();
    }

    private void HandlePlayerLeft(List<int> leftPlayers)
    {
        foreach (int playerIndex in leftPlayers)
        {
            this.LocalLobby.Players.RemoveAt(playerIndex);
        }
    }

    private void HandlePlayerJoined(List<LobbyPlayerJoined> joinedPlayers)
    {
        foreach (LobbyPlayerJoined newPlayer in joinedPlayers)
        {
            this.LocalLobby.Players.Add(newPlayer.Player);
        }
    }

    private void HandleDataChanged(Dictionary<string,
    ChangedOrRemovedLobbyValue<DataObject>> changedLobbyData)
    {
        foreach (string key in changedLobbyData.Keys)
        {
            this.LocalLobby.Data[key] = changedLobbyData[key].Value;
        }
    }

```

```

switch (this.LocalLobby.Data[LobbyManager.KEY_LOBBY_STATE].Value)
{
    case LobbyManager.LOBBY_STATE_LOADING:
    case LobbyManager.LOBBY_STATE_PENDING:

        UIManagerGlobal.Instance.ShowMessageBox(PanelMessageBoxUI.Type.None,
        UIManagerGameLobby.Instance?.MessagePendingGame ??
        UIManagerMonopolyGame.Instance?.MessageWaitingOtherPlayers,
        PanelMessageBoxUI.Icon.Loading, stateCallback: () => this.HavePlayersLoaded);
        break;
    case LobbyManager.LOBBY_STATE_RETURNING:

        UIManagerGlobal.Instance.ShowMessageBox(PanelMessageBoxUI.Type.None,
        UIManagerGameLobby.Instance?.MessageFailedToConnect ??
        UIManagerMonopolyGame.Instance?.MessagePlayersFailedToLoad,
        PanelMessageBoxUI.Icon.Loading, stateCallback: () => this.HavePlayersLoaded);
        break;
    }
}

private void HandlePlayerDataChanged(Dictionary<int, Dictionary<string,
ChangedOrRemovedLobbyValue<PlayerDataObject>>> changedPlayerData)
{
    foreach (int playerIndex in changedPlayerData.Keys)
    {
        foreach (string key in changedPlayerData[playerIndex].Keys)
        {
            this.LocalLobby.Players[playerIndex].Data[key] =
changedPlayerData[playerIndex][key].Value;
        }
    }
}

#endregion

#region Loading Callbacks

private void HandleGameLobbyLoaded()
{
    if (this.IsHost)
    {
        this.UpdateLocalLobbyData(LobbyManager.LOBBY_STATE_LOBBY,
false);
    }
}

```



```

private void HandleMonopolyGameLoaded()
{
    if (this.IsHost)
    {
        this.UpdateLocalLobbyData(LobbyManager.LOBBY_STATE_LOBBY,
true);
    }
}

```

```

private void HandleMonopolyGameFailedToLoad()
{
    if (this.IsHost)
    {

this.UpdateLocalLobbyData(LobbyManager.LOBBY_STATE_RETURNING,
true);

```

```

GameCoordinator.Instance.LoadSceneNetwork(GameCoordinator.MonopolyScene
.GameLobby);
    }
}

```

```

private async void HandleGameLobbyFailedToLoadAsync()
{
    await this.DisconnectFromLobbyAsync();
}

#endregion
}

```

Файл TradeCredentialsSerializer.cs

```

using Unity.Netcode;

public struct TradeCredentials : INetworkSerializable
{
    public ulong SenderId;
    public ulong ReceiverId;

    public int SenderOffer;
    public int ReceiverOffer;

```

```

public int SenderNodeIndex;
public int ReceiverNodeIndex;

public void NetworkSerialize<T>(BufferSerializer<T> serializer) where T :
IReaderWriter
{
    serializer.SerializeValue(ref this.SenderId);
    serializer.SerializeValue(ref this.ReceiverId);
    serializer.SerializeValue(ref this.SenderOffer);
    serializer.SerializeValue(ref this.ReceiverOffer);
    serializer.SerializeValue(ref this.SenderNodeIndex);
    serializer.SerializeValue(ref this.ReceiverNodeIndex);
}
}

```

Файл GameManager.cs

```

using System;
using System.Linq;
using UnityEngine;
using Unity.Netcode;
using System.Collections;
using System.Collections.Generic;
using System.Collections.ObjectModel;

internal sealed class GameManager : NetworkBehaviour
{
    #region Setup

    #region Values

    [Header("Values")]

    [Space]

```

```
[SerializeField] [Range(0, 100_000)] private int startingBalance = 15_000;
```

```
[Space]
```

```
[SerializeField] [Range(0, 10)] private int maxTurnsInJail = 3;
```

```
[Space]
```

```
[SerializeField] [Range(0, 10)] private int maxDoublesInRow = 2;
```

```
[Space]
```

```
[SerializeField] [Range(0, 100_000)] private int circleBonus = 2_000;
```

```
[Space]
```

```
[SerializeField] [Range(0, 100_000)] private int exactCircleBonus = 3_000;
```

```
[Space]
```

```
    [SerializeField] [Range(0.0f, 100.0f)] private float playerMovementSpeed =  
35.0f;
```

```
#endregion
```

```
#region Visuals
```

```
[Space]
```

```
[Header("Visuals")]
```

```
#region Player
```

```
[Space]
```

```
[Header("Player")]
```

[Space]

[SerializeField] private GameObject player;

[Space]

[SerializeField] private GameObject playerPanel;

#endregion

#region Players Tokens

[Space]

[Header("Players Visuals")]

[Space]

[SerializeField] private MonopolyPlayerVisuals[] monopolyPlayersVisuals =
new MonopolyPlayerVisuals[5];

#endregion

#endregion

#endregion

private int rolledDoubles;

private List<MonopolyPlayer> players;

private ulong[] targetCurrentClient;

private ulong[] targetClientOtherClients;

```
private List<ulong[]> targetHostOtherClients;

public static GameManager Instance { get; private set; }

public int CircleBonus
{
    get => this.circleBonus;
}

public int MaxTurnsInJail
{
    get => this.maxTurnsInJail;
}

public int TotalRollResult
{
    get => this.FirstDieValue + this.SecondDieValue;
}

public int MaxDoublesInRow
{
    get => this.maxDoublesInRow;
}

public int StartingBalance
{
    get => this.startingBalance;
}
```

```

public bool HasRolledDouble
{
    get => this.FirstDieValue == this.SecondDieValue;
}

public int ExactCircleBonus
{
    get => this.exactCircleBonus;
}

public float PlayerMovementSpeed
{
    get => this.playerMovementSpeed;
}

public MonopolyPlayer CurrentPlayer
{
    get
    {
        if (this.CurrentPlayerIndex >= 0 && this.CurrentPlayerIndex <
this.players.Count)
        {
            return this.players[this.CurrentPlayerIndex];
        }
        else
        {
            return null;
        }
    }
}

```

```

public int FirstDieValue { get; private set; }

public int SecondDieValue { get; private set; }

public int CurrentPlayerIndex { get; private set; }

public ServerRpcParams ServerParamsCurrentClient
{
    get
    {
        return new ServerRpcParams
        {
            Receive = new ServerRpcReceiveParams { SenderClientId =
NetworkManager.Singleton.LocalClientId }
        };
    }
}

public ClientRpcParams ClientParamsCurrentClient
{
    get
    {
        this.targetCurrentClient[0] =
NetworkManager.Singleton.ConnectedClientsIds[this.CurrentPlayerIndex];

        return new ClientRpcParams
        {
            Send = new ClientRpcSendParams { TargetClientIds =
this.targetCurrentClient }

```

```

        };
    }
}

```

```

public ClientRpcParams ClientParamsHostOtherClients
{
    get
    {
        return new ClientRpcParams
        {
            Send = new ClientRpcSendParams { TargetClientIds =
this.targetHostOtherClients[this.CurrentPlayerIndex] }
        };
    }
}

```

```

public ClientRpcParams ClientParamsClientOtherClients
{
    get
    {
        return new ClientRpcParams
        {
            Send = new ClientRpcSendParams { TargetClientIds =
this.targetClientOtherClients }
        };
    }
}

```

```

    public ReadOnlyCollection<MonopolyPlayerVisuals> MonopolyPlayersVisuals
    { get; private set; }

```



```

private void Awake()
{
    if (Instance != null)
    {
        throw new System.InvalidOperationException($"Singleton
{this.GetType().FullName} has already been initialized.");
    }

    Instance = this;
}

private void Start()
{
    UIManagerGlobal.Instance.ShowMessageBox(PanelMessageBoxUI.Type.None,
    UIManagerMonopolyGame.Instance.MessageWaitingOtherPlayers,
    PanelMessageBoxUI.Icon.Loading,          stateCallback:          ()          =>
    LobbyManager.Instance.HavePlayersLoaded);

    this.players = new List<MonopolyPlayer>();

    this.MonopolyPlayersVisuals = new
    ReadOnlyCollection<MonopolyPlayerVisuals>(this.monopolyPlayersVisuals);

    if (LobbyManager.Instance.IsHost)
    {
        this.StartCoroutine(this.WaitOtherPlayersCoroutine());
    }
}

```

```

        GameCoordinator.Instance?.UpdateInitializedObjects(this.GetType());
    }

    private void OnEnable()
    {
        if (NetworkManager.Singleton != null &&
        NetworkManager.Singleton.IsHost)
        {
            NetworkManager.Singleton.OnClientDisconnectCallback +=
            this.HandleClientDisconnectCallback;
        }
    }

    private void OnDisable()
    {
        if (NetworkManager.Singleton != null &&
        NetworkManager.Singleton.IsHost)
        {
            NetworkManager.Singleton.OnClientDisconnectCallback -=
            this.HandleClientDisconnectCallback;
        }
    }

    #region Callbacks

    public MonopolyPlayer GetPlayerById(ulong clientId)
    {
        return this.players.Where(player => player.OwnerClientId ==
        clientId).FirstOrDefault();
    }

```

```

public ClientRpcParams GetRedirectionRpc(ulong clientId)
{
    this.targetCurrentClient[0] = clientId;

    return new ClientRpcParams
    {
        Send = new ClientRpcSendParams { TargetClientIds =
this.targetCurrentClient }
    };
}

private void HandleClientDisconnectCallback(ulong surrenderedClientId)
{
    if (this.players.Any(player => player.OwnerClientId == surrenderedClientId))
    {
        this.RemovePlayerServerRpc(surrenderedClientId,
this.ServerParamsCurrentClient);
    }
}

[ServerRpc]
public void RemovePlayerServerRpc(ulong surrenderedClientId,
ServerRpcParams serverRpcParams)
{
    bool hasCurrentLeft = false;

    if (this.players.Any(player => player.OwnerClientId == surrenderedClientId))
    {

```

```

                                int    surrenderedPlayerIndex    =
this.players.IndexOf(this.players.Where(player => player.OwnerClientId ==
surrenderedClientId).First());

```

```

    if (this.CurrentPlayer == this.players[surrenderedPlayerIndex])
    {
        hasCurrentLeft = true;
    }

```

```

    this.players.RemoveAt(surrenderedPlayerIndex);

```

```

    this.targetHostOtherClients.RemoveAt(surrenderedPlayerIndex);

```

```

                                this.targetClientOtherClients    =
this.targetClientOtherClients?.Where(clientId => clientId !=
surrenderedClientId).ToArray();

```

```

        this.targetHostOtherClients = this.targetHostOtherClients.Select(array =>
array.Where(id => id != surrenderedClientId).ToArray()).ToList();

```

```

        if (this.players.Count == 1 && this.players.First().OwnerClientId ==
NetworkManager.Singleton.LocalClientId &&
NetworkManager.Singleton.IsConnectedClient)

```

```

    {
        UIManagerMonopolyGame.Instance.HidePaymentProperty();
        UIManagerMonopolyGame.Instance.HideButtonRollDice();
        UIManagerMonopolyGame.Instance.HidePaymentChance();
        UIManagerMonopolyGame.Instance.HideMonopolyNode();
        UIManagerMonopolyGame.Instance.HideReceiveTrade();
        UIManagerMonopolyGame.Instance.HideTradeOffer();
        UIManagerMonopolyGame.Instance.HideOffer();

```

```

        UIManagerMonopolyGame.Instance.ShowButtonDisconnect();

```

```

UIManagerGlobal.Instance.ShowMessageBox(PanelMessageBoxUI.Type.OK,
UIManagerMonopolyGame.Instance.MessageWon,
PanelMessageBoxUI.Icon.Trophy);
    }
    else
    {
        this.RemovePlayerClientRpc(surrenderedClientId,
this.ClientParamsClientOtherClients);

        if (hasCurrentLeft)
        {

this.SwitchPlayerForcefullyServerRpc(this.ServerParamsCurrentClient);
        }
    }
}
}

[ClientRpc]
    private void RemovePlayerClientRpc(ulong surrenderedClientId,
ClientRpcParams clientRpcParams)
    {
        this.players.Remove(this.players.Where(player => player.OwnerClientId ==
surrenderedClientId).First());

        this.targetClientOtherClients = this.targetClientOtherClients?.Where(clientId
=> clientId != surrenderedClientId).ToArray();

        if (this.players.Count == 1 && this.players.First().OwnerClientId ==
NetworkManager.Singleton.LocalClientId)

```

```

{
    UIManagerMonopolyGame.Instance.HidePaymentProperty();
    UIManagerMonopolyGame.Instance.HideButtonRollDice();
    UIManagerMonopolyGame.Instance.HidePaymentChance();
    UIManagerMonopolyGame.Instance.HideMonopolyNode();
    UIManagerMonopolyGame.Instance.HideReceiveTrade();
    UIManagerMonopolyGame.Instance.HideTradeOffer();
    UIManagerMonopolyGame.Instance.HideOffer();

    UIManagerMonopolyGame.Instance.ShowButtonDisconnect();

    UIManagerGlobal.Instance.ShowMessageBox(PanelMessageBoxUI.Type.OK,
    UIManagerMonopolyGame.Instance.MessageWon,
    PanelMessageBoxUI.Icon.Trophy);
}
}

#endregion

#region Initialization

private IEnumerator WaitOtherPlayersCoroutine()
{
    float elapsedTime = 0f;

    while (!LobbyManager.Instance.HavePlayersLoaded && elapsedTime <
LobbyManager.LOBBY_LOADING_TIMEOUT)
    {
        elapsedTime += Time.deltaTime;
        yield return null;
    }
}

```

```

    }

    if (!LobbyManager.Instance.HavePlayersLoaded)
    {
        LobbyManager.Instance?.OnMonopolyGameFailedToLoad?.Invoke();
    }
    else
    {
        this.InitializeGameServerRpc(this.ServerParamsCurrentClient);
    }
}

public void AddPlayer(MonopolyPlayer monopolyPlayer)
{
    this.players.Add(monopolyPlayer);
}

[ServerRpc]
private void InitializeGameServerRpc(ServerRpcParams serverRpcParams)
{
    this.targetCurrentClient = new ulong[1];
    this.targetHostOtherClients = new List<ulong[]>();
    this.targetClientOtherClients =
NetworkManager.Singleton.ConnectedClientsIds.Where((value) => value !=
NetworkManager.Singleton.ConnectedClientsIds[0]).ToArray();

    for (int i = 0; i < NetworkManager.Singleton?.ConnectedClientsIds.Count;
++i)
    {
        this.CurrentPlayerIndex = i;
    }
}

```

```

this.targetHostOtherClients.Add(NetworkManager.Singleton.ConnectedClientsIds.
Where((value) => value
NetworkManager.Singleton.ConnectedClientsIds[i]).ToArray());

        this.SwitchPlayerClientRpc(this.CurrentPlayerIndex,
this.ClientParamsClientOtherClients);

        this.player = GameObject.Instantiate(this.player);
        this.playerPanel = GameObject.Instantiate(this.playerPanel);

this.player.GetComponent<NetworkObject>().SpawnAsPlayerObject(NetworkMa
nager.Singleton.ConnectedClientsIds[i], true);

this.playerPanel.GetComponent<NetworkObject>().SpawnWithOwnership(Networ
kManager.Singleton.ConnectedClientsIds[i], true);
    }

        this.CurrentPlayerIndex = 0;

        this.SwitchPlayerClientRpc(this.CurrentPlayerIndex,
this.ClientParamsClientOtherClients);

        this.CurrentPlayer.PerformTurnClientRpc(this.ClientParamsCurrentClient);
    }

#endregion

```


#region Turn-based Game Loop

```
[ServerRpc(RequireOwnership = false)]
public void SwitchPlayerServerRpc(ServerRpcParams serverRpcParams)
{
    if (this.HasRolledDouble)
    {
        ++this.rolledDoubles;

        if (this.rolledDoubles >= this.MaxDoublesInRow)
        {
            this.rolledDoubles = 0;
            this.CurrentPlayer.GoToJailClientRpc(this.ClientParamsCurrentClient);
        }
    }
    else
    {
        this.rolledDoubles = 0;
        this.CurrentPlayerIndex = ++this.CurrentPlayerIndex % this.players.Count;
    }

    this.SwitchPlayerClientRpc(this.CurrentPlayerIndex,
this.ClientParamsClientOtherClients);

    UIManagerMonopolyGame.Instance.HideButtonRollDiceClientRpc(this.ClientPar
amsClientOtherClients);

    this.CurrentPlayer.PerformTurnClientRpc(this.ClientParamsCurrentClient);
}
```

```

[ServerRpc(RequireOwnership = false)]
    public void SwitchPlayerForcefullyServerRpc(ServerRpcParams
serverRpcParams)
{
    this.rolledDoubles = 0;

    this.CurrentPlayerIndex = ++this.CurrentPlayerIndex % this.players.Count;

    this.SwitchPlayerClientRpc(this.CurrentPlayerIndex,
this.ClientParamsClientOtherClients);

    UIManagerMonopolyGame.Instance.HideButtonRollDiceClientRpc(this.ClientPar
amsClientOtherClients);

    this.CurrentPlayer.PerformTurnClientRpc(this.ClientParamsCurrentClient);
}

[ClientRpc]
    private void SwitchPlayerClientRpc(int currentPlayerIndex, ClientRpcParams
clientRpcParams)
{
    this.CurrentPlayerIndex = currentPlayerIndex;
}

#endregion

#region Rolling Dice & Syncing

```

```

public void RollDice()
{
    const int MIN_DIE_VALUE = 1;
    const int MAX_DIE_VALUE = 6;

    this.FirstDieValue = UnityEngine.Random.Range(MIN_DIE_VALUE,
MAX_DIE_VALUE + 1);
    this.SecondDieValue = UnityEngine.Random.Range(MIN_DIE_VALUE,
MAX_DIE_VALUE + 1);

    this.RollDiceServerRpc(this.FirstDieValue, this.SecondDieValue,
this.ServerParamsCurrentClient);
}

[ServerRpc(RequireOwnership = false)]
private void RollDiceServerRpc(int firstDieValue, int secondDieValue,
ServerRpcParams serverRpcParams)
{
    this.RollDiceClientRpc(firstDieValue, secondDieValue,
this.ClientParamsHostOtherClients);
}

[ClientRpc]
private void RollDiceClientRpc(int firstDieValue, int secondDieValue,
ClientRpcParams clientRpcParams)
{
    this.FirstDieValue = firstDieValue;
    this.SecondDieValue = secondDieValue;
}

```

```
#endregion
}
```

Файл MonopolyBoard.cs

```
using UnityEngine;
using System.Collections.Generic;

public sealed class MonopolyBoard : MonoBehaviour
{
    #region Setup

    #region Special nodes

    [Header("Special nodes")]

    [Space]
    [SerializeField] private MonopolyNode jail;

    [Space]
    [SerializeField] private MonopolyNode start;

    [Space]
    [SerializeField] private MonopolyNode sendJail;

    [Space]
    [SerializeField] private MonopolyNode freeParking;

    #endregion
}
```

#region Monopolies

[Space]

[Header("Monopolies")]

[Space]

```
[SerializeField] private List<MonopolySet> monopolies = new
List<MonopolySet>();
```

#endregion

#region Chance & Tax nodes

[Space]

[Header("Chance & Tax nodes")]

[Space]

```
[SerializeField] private List<ChanceNodeSO> taxNodes = new
List<ChanceNodeSO>();
```

[Space]

```
[SerializeField] private List<ChanceNodeSO> chanceNodes = new
List<ChanceNodeSO>();
```

#endregion

#endregion

```
private List<MonopolyNode> nodes;
```

```

public static MonopolyBoard Instance { get; private set; }

public List<MonopolySet> Monopolies { get => this.monopolies; }

public int NumberOfNodes { get => this.nodes.Count; }

public MonopolyNode NodeJail { get => this.jail; }

public MonopolyNode NodeStart { get => this.start; }

public MonopolyNode NodeSendToJail { get => this.sendJail; }

public MonopolyNode NodeFreeParking { get => this.freeParking; }

private void Awake()
{
    if (Instance != null)
    {
        throw new System.InvalidOperationException($"Singleton
{this.GetType().FullName} has already been initialized.");
    }

    Instance = this;
}

private void Start()
{
    this.nodes = new List<MonopolyNode>();

    foreach (Transform child in this.transform)

```

```

    {
        if (child.TryGetComponent(out MonopolyNode monopolyNode))
        {
            this.nodes.Add(monopolyNode);
        }
    }

    GameCoordinator.Instance?.UpdateInitializedObjects(this.GetType());
}

public MonopolyNode this[int index]
{
    get
    {
        if (index < 0 || index >= this.nodes.Count)
        {
            throw new System.IndexOutOfRangeException($"{nameof(index)} is
out of range.");
        }

        return this.nodes[index];
    }
}

public int this[MonopolyNode monopolyNode]
{
    get
    {
        if (monopolyNode == null)
        {

```

throw new

```

System.NullReferenceException($"{nameof(monopolyNode)} is null.");
    }

    return this.nodes.IndexOf(monopolyNode);
}
}

public ChanceNodeSO GetTaxNode()
{
    return this.taxNodes[UnityEngine.Random.Range(0, this.taxNodes.Count)];
}

public ChanceNodeSO GetChanceNode()
{
    return this.chanceNodes[UnityEngine.Random.Range(0,
this.chanceNodes.Count)];
}

public MonopolySet GetMonopolySet(MonopolyNode monopolyNode)
{
    if (monopolyNode == null)
    {
        throw new System.ArgumentNullException($"{nameof(monopolyNode)}
is null.");
    }

    foreach (MonopolySet monopolySet in this.monopolies)
    {
        if (monopolySet.Contains(monopolyNode))

```



```

        {
            return monopolySet;
        }
    }

    return null;
}

public int GetDistance(int fromNodeIndex, int toNodeIndex)
{
    int clockwiseDistance = (toNodeIndex - fromNodeIndex +
this.NumberOfNodes) % this.NumberOfNodes;
    int counterclockwiseDistance = (fromNodeIndex - toNodeIndex +
this.NumberOfNodes) % this.NumberOfNodes;

    return Mathf.Min(clockwiseDistance, counterclockwiseDistance) ==
counterclockwiseDistance ? -counterclockwiseDistance : clockwiseDistance;
}

public int GetDistance(MonopolyNode fromNode, MonopolyNode toNode)
{
    int clockwiseDistance = (this[toNode] - this[fromNode] +
this.NumberOfNodes) % this.NumberOfNodes;
    int counterclockwiseDistance = (this[fromNode] - this[toNode] +
this.NumberOfNodes) % this.NumberOfNodes;

    return Mathf.Min(clockwiseDistance, counterclockwiseDistance) ==
counterclockwiseDistance ? -counterclockwiseDistance : clockwiseDistance;
}
}

```

Файл MonopolyNode.cs

```
using System.Linq;
using UnityEngine;
using Unity.Netcode;
using UnityEngine.UI;
using System.Collections.Generic;

public sealed class MonopolyNode : NetworkBehaviour
{
    #region Setup (Editor)

    [SerializeField] private Type type;

    [SerializeField] private Image imageLogo;

    [SerializeField] private Sprite spriteLogo;

    [SerializeField] private Image imageOwner;

    [SerializeField] private Image imageMonopolyType;

    [SerializeField] private Image imageMortgageStatus;

    [SerializeField] private Image imageLevel1;

    [SerializeField] private Image imageLevel2;

    [SerializeField] private Image imageLevel3;
```

```
[SerializeField] private Image imageLevel4;
```

```
[SerializeField] private Image imageLevel5;
```

```
[SerializeField] private int pricePurchase;
```

```
[SerializeField] private int priceUpgrade;
```

```
[SerializeField] private List<int> pricesRent = new List<int>();
```

```
#endregion
```

```
public enum Type : byte
```

```
{
```

```
    Tax,
```

```
    Jail,
```

```
    Start,
```

```
    Chance,
```

```
    SendJail,
```

```
    Property,
```

```
    Gambling,
```

```
    Transport,
```

```
    FreeParking
```

```
}
```

```
private const int LEVEL_MORTGAGE = 0;
```

```
private const int LEVEL_OWNERSHIP = 1;
```

```
public const int PROPERTY_MIN_LEVEL = 0;
```

```

public const int PROPERTY_MAX_LEVEL = 6;

public NetworkVariable<int> Level { get; private set; }

public Type NodeType
{
    get => this.type;
}

public int PriceRent
{
    get
    {
        return this.pricesRent[this.LocalLevel] * (this.NodeType ==
MonopolyNode.Type.Gambling ? GameManager.Instance.TotalRollResult : 1);
    }
}

public bool IsMortgaged
{
    get => this.LocalLevel == 0;
}

public int PriceUpgrade
{
    get
    {
        if (this.NodeType == MonopolyNode.Type.Property)
        {

```

```

        return this.LocalLevel == 0 ? this.pricePurchase : this.priceUpgrade;
    }
    else
    {
        return this.pricePurchase;
    }
}
}

```

```

public int PricePurchase
{
    get => this.pricePurchase;
}

```

```

public int PriceDowngrade
{
    get
    {
        if (this.NodeType == MonopolyNode.Type.Property)
        {
            return this.LocalLevel == 1 ? this.pricePurchase : this.priceUpgrade;
        }
        else
        {
            return this.pricePurchase;
        }
    }
}

```

```

public Sprite NodeSprite

```

```

{
    get => this.spriteLogo;
}

public bool IsTradable
{
    get
    {
        if (this.NodeType == MonopolyNode.Type.Property)
        {
            return this.LocalLevel == MonopolyNode.LEVEL_OWNERSHIP ? true
: false;
        }
        else if (this.NodeType == MonopolyNode.Type.Transport || this.NodeType
== MonopolyNode.Type.Gambling)
        {
            return this.LocalLevel > MonopolyNode.LEVEL_MORTGAGE ? true :
false;
        }
        else
        {
            return false;
        }
    }
}

public bool IsUpgradable
{
    get
    {

```

```

    if (this.NodeType == MonopolyNode.Type.Property)
    {
        bool isEquallySpread = this.AffiliatedMonopoly.NodesInSet.All(node =>
node.LocalLevel >= this.LocalLevel);
        return (isEquallySpread && this.LocalLevel <
MonopolyNode.PROPERTY_MAX_LEVEL) || this.IsMortgaged;
    }
    else if (this.NodeType == MonopolyNode.Type.Transport || this.NodeType
== MonopolyNode.Type.Gambling)
    {
        if (this.LocalLevel == MonopolyNode.LEVEL_MORTGAGE)
        {
            return true;
        }
        else
        {
            return false;
        }
    }
    else
    {
        return false;
    }
}

```

```

public bool IsDowngradable

```

```

{
    get
    {

```

```

        if (this.NodeType == MonopolyNode.Type.Property)
        {
            bool isEquallySpread = this.AffiliatedMonopoly.NodesInSet.All(node =>
node.LocalLevel <= this.LocalLevel);
            return isEquallySpread && this.LocalLevel >
MonopolyNode.PROPERTY_MIN_LEVEL;
        }
        else if (this.NodeType == MonopolyNode.Type.Gambling || this.NodeType
== MonopolyNode.Type.Transport)
        {
            return this.LocalLevel > MonopolyNode.PROPERTY_MIN_LEVEL ?
true : false;
        }
        else
        {
            return false;
        }
    }
}

public int LocalLevel { get; private set; }

public MonopolyPlayer Owner { get; private set; }

public MonopolySet AffiliatedMonopoly { get; private set; }

private void Awake()
{
    this.imageLogo.sprite = this.spriteLogo;

```



```

        this.Level = new NetworkVariable<int>(1,
NetworkVariableReadPermission.Everyone,
NetworkVariableWritePermission.Server);

        switch (this.NodeType)
        {
            case MonopolyNode.Type.Property:
            case MonopolyNode.Type.Gambling:
            case MonopolyNode.Type.Transport:
                {
                    this.LocalLevel = 1;
                    this.AffiliatedMonopoly =
MonopolyBoard.Instance.GetMonopolySet(this);
                    this.imageMonopolyType.color =
this.AffiliatedMonopoly.ColorOfSet;
                }
                break;
        }
    }

    private void OnEnable()
    {
        this.Level.OnValueChanged += this.HandleLevelChanged;
    }

    private void OnDisable()
    {
        this.Level.OnValueChanged -= this.HandleLevelChanged;
    }

```

#region Visuals

```
private void UpdateVisualsSpecial()
{
    if (this.Owner == null)
    {
        this.imageOwner.gameObject.SetActive(false);
        this.imageMortgageStatus.gameObject.SetActive(false);
    }
    else
    {
        if (this.LocalLevel == MonopolyNode.LEVEL_MORTGAGE)
        {
            this.imageMortgageStatus.gameObject.SetActive(true);
        }
        else
        {
            this.imageMortgageStatus.gameObject.SetActive(false);

            this.imageOwner.gameObject.SetActive(true);
            this.imageOwner.color = this.Owner.PlayerColor;
        }
    }
}
```

```
private void UpdateVisualsProperty()
{
    if (this.Owner == null)
    {
        this.imageOwner.gameObject.SetActive(false);
```

```

    this.imageLevel1.gameObject.SetActive(false);
    this.imageLevel2.gameObject.SetActive(false);
    this.imageLevel3.gameObject.SetActive(false);
    this.imageLevel4.gameObject.SetActive(false);
    this.imageLevel5.gameObject.SetActive(false);
    this.imageMortgageStatus.gameObject.SetActive(false);
}
else
{
    switch (this.LocalLevel)
    {
        case MonopolyNode.LEVEL_MORTGAGE:
        {
            this.imageMortgageStatus.gameObject.SetActive(true);
        }
        break;
        case MonopolyNode.LEVEL_OWNERSHIP:
        {
            this.imageOwner.gameObject.SetActive(true);
            this.imageOwner.color = this.Owner.PlayerColor;

            this.imageLevel1.gameObject.SetActive(false);
            this.imageMortgageStatus.gameObject.SetActive(false);
        }
        break;
        case 2:
        {
            this.imageLevel1.gameObject.SetActive(true);
            this.imageLevel2.gameObject.SetActive(false);
        }
    }
}

```

```
        break;
    case 3:
    {
        this.imageLevel2.gameObject.SetActive(true);
        this.imageLevel3.gameObject.SetActive(false);
    }
    break;
    case 4:
    {
        this.imageLevel3.gameObject.SetActive(true);
        this.imageLevel4.gameObject.SetActive(false);
    }
    break;
    case 5:
    {
        this.imageLevel1.gameObject.SetActive(true);
        this.imageLevel2.gameObject.SetActive(true);
        this.imageLevel3.gameObject.SetActive(true);
        this.imageLevel4.gameObject.SetActive(true);
        this.imageLevel5.gameObject.SetActive(false);
    }
    break;
    case 6:
    {
        this.imageLevel5.gameObject.SetActive(true);
        this.imageLevel1.gameObject.SetActive(false);
        this.imageLevel2.gameObject.SetActive(false);
        this.imageLevel3.gameObject.SetActive(false);
        this.imageLevel4.gameObject.SetActive(false);
    }
```

```

        break;
    }
}

#endregion

#region Ownership

public void ResetOwnership()
{
    this.Owner = null;
    this.LocalLevel = MonopolyNode.LEVEL_OWNERSHIP;

    if (this.NodeType == MonopolyNode.Type.Property)
    {
        this.UpdateVisualsProperty();
    }
    else
    {
        this.UpdateVisualsSpecial();
    }

    this.ResetOwnershipServerRpc(GameManager.Instance.ServerParamsCurrentClient);
}

public void UpdateOwnership(ulong ownerId)
{

```

```
this.LocalLevel = MonopolyNode.LEVEL_OWNERSHIP;
this.Owner = GameManager.Instance.GetPlayerById(ownerId);

if (this.Owner == null)
{
    return;
}

if (this.NodeType == MonopolyNode.Type.Property)
{
    this.UpdateVisualsProperty();
}
else
{
    this.UpdateVisualsSpecial();
}

this.UpdateOwnershipServerRpc(ownerId,
GameManager.Instance.ServerParamsCurrentClient);

if (this.NodeType != MonopolyNode.Type.Transport && this.NodeType !=
MonopolyNode.Type.Gambling)
{
    return;
}

if (!this.Owner.HasPartialMonopoly(this, out _))
{
    return;
}
```

```

        foreach (MonopolyNode node in
this.AffiliatedMonopoly.OwnedByPlayerNodes)
    {
        if (!node.IsMortgaged)
        {
            while (node.LocalLevel <
this.AffiliatedMonopoly.OwnedByPlayerCount)
            {
                node.Upgrade();
            }
        }
    }
}

[ServerRpc(RequireOwnership = false)]
public void ResetOwnershipServerRpc(ServerRpcParams serverRpcParams)
{
    this.LocalLevel = MonopolyNode.LEVEL_OWNERSHIP;
    this.Level.Value = MonopolyNode.LEVEL_OWNERSHIP;

    if (this.Owner != null)
    {
        this.Owner = null;

        if (this.NodeType == MonopolyNode.Type.Property)
        {
            this.UpdateVisualsProperty();
        }
        else

```

```

    {
        this.UpdateVisualsSpecial();
    }
}

```

```

this.ResetOwnershipClientRpc(GameManager.Instance.ClientParamsClientOtherC
lients);
}

```

```

[ClientRpc]
private void ResetOwnershipClientRpc(ClientRpcParams clientRpcParams)
{
    if (this.Owner != null)
    {
        this.Owner = null;
        this.LocalLevel = MonopolyNode.LEVEL_OWNERSHIP;

        if (this.NodeType == MonopolyNode.Type.Property)
        {
            this.UpdateVisualsProperty();
        }
        else
        {
            this.UpdateVisualsSpecial();
        }
    }
}

```

```

[ServerRpc(RequireOwnership = false)]

```



```

private void UpdateOwnershipServerRpc(ulong ownerId, ServerRpcParams
serverRpcParams)
{
    this.LocalLevel = MonopolyNode.LEVEL_OWNERSHIP;
    this.Level.Value = MonopolyNode.LEVEL_OWNERSHIP;

    MonopolyPlayer playerOwner =
GameManager.Instance.GetPlayerById(ownerId);

    if (this.Owner != playerOwner)
    {
        this.Owner = playerOwner;

        if (this.NodeType == MonopolyNode.Type.Property)
        {
            this.UpdateVisualsProperty();
        }
        else
        {
            this.UpdateVisualsSpecial();
        }
    }

    this.UpdateOwnershipClientRpc(ownerId,
GameManager.Instance.ClientParamsClientOtherClients);
}

[ClientRpc]
private void UpdateOwnershipClientRpc(ulong ownerId, ClientRpcParams
clientRpcParams)

```

```

{
    MonopolyPlayer playerOwner =
GameManager.Instance.GetPlayerById(ownerId);

    if (this.Owner != playerOwner)
    {
        this.Owner = playerOwner;
        this.LocalLevel = MonopolyNode.LEVEL_OWNERSHIP;

        if (this.NodeType == MonopolyNode.Type.Property)
        {
            this.UpdateVisualsProperty();
        }
        else
        {
            this.UpdateVisualsSpecial();
        }
    }
}

#endregion

#region Upgrade/Downgrade

public void Upgrade()
{
    if (this.NodeType == MonopolyNode.Type.Property)
    {
        ++this.LocalLevel;
    }
}

```

```

        this.UpdateVisualsProperty();

        this.ChangeLevelServerRpc(this.LocalLevel,
GameManager.Instance.ServerParamsCurrentClient);
    }
    else
    {
        this.LocalLevel = MonopolyNode.LEVEL_OWNERSHIP;

        this.UpdateVisualsSpecial();

        this.ChangeLevelServerRpc(this.LocalLevel,
GameManager.Instance.ServerParamsCurrentClient);

        if (this.Owner.HasPartialMonopoly(this, out _))
        {
            this.LocalLevel = this.AffiliatedMonopoly.OwnedByPlayerCount;
            this.ChangeLevelServerRpc(this.LocalLevel,
GameManager.Instance.ServerParamsCurrentClient);
        }
    }
}

public void Downgrade()
{
    if (this.NodeType == MonopolyNode.Type.Property)
    {
        --this.LocalLevel;

        this.UpdateVisualsProperty();
    }
}

```

```

    }
    else
    {
        this.LocalLevel = MonopolyNode.LEVEL_MORTGAGE;

        this.UpdateVisualsSpecial();
    }

    this.ChangeLevelServerRpc(this.LocalLevel,
GameManager.Instance.ServerParamsCurrentClient);
}

[ServerRpc(RequireOwnership = false)]
public void ChangeLevelServerRpc(int level, ServerRpcParams
serverRpcParams)
{
    this.Level.Value = level;
}

#endregion

private void HandleLevelChanged(int previousValue, int newValue)
{
    this.LocalLevel = newValue;

    if (this.NodeType == MonopolyNode.Type.Property)
    {
        this.UpdateVisualsProperty();
    }
    else

```

```

    {
        this.UpdateVisualsSpecial();
    }
}
}

```

Файл MonopolyPlayer.cs

```

using System;
using System.Linq;
using UnityEngine;
using Unity.Netcode;
using UnityEngine.UI;
using System.Collections;
using System.Collections.Generic;

public sealed class MonopolyPlayer : NetworkBehaviour
{
    #region Setup

    #region Visuals

    [Header("Visuals")]

    [Space]
    [SerializeField] private Image playerImageToken;

    #endregion

    #endregion
}

```

```
private bool isInJail;
```

```
private int turnsInJail;
```

```
private bool isSkipTurn;
```

```
public Action OnBalanceUpdated;
```

```
public bool IsTrading { get; set; }
```

```
public bool HasBuilt { get; private set; }
```

```
public bool HasRolled { get; private set; }
```

```
public string Nickname { get; private set; }
```

```
public bool IsAbleToBuild { get; private set; }
```

```
public bool HasCompletedTurn { get; private set; }
```

```
public Color PlayerColor { get; private set; }
```

```
public MonopolyNode SelectedNode { get; set; }
```

```
public MonopolyNode CurrentNode { get; private set; }
```

```
public MonopolyPlayer PlayerTradingWith { get; set; }
```

```
public NetworkVariable<int> Balance { get; private set; }
```

```

public List<MonopolyNode> OwnedNodes { get; private set; }

public ChanceNodeSO CurrentChanceNode { get; private set; }

private void Awake()
{
    this.Balance = new
NetworkVariable<int>(GameManager.Instance.StartingBalance,
NetworkVariableReadPermission.Everyone,
NetworkVariableWritePermission.Owner);
}

public override void OnNetworkSpawn()
{
    this.OwnedNodes = new List<MonopolyNode>();
    this.Balance.Value = GameManager.Instance.StartingBalance;
    this.CurrentNode = MonopolyBoard.Instance.NodeStart;
    this.transform.position =
MonopolyBoard.Instance.NodeStart.transform.position;
    this.PlayerColor =
GameManager.Instance.MonopolyPlayersVisuals[GameManager.Instance.Current
PlayerIndex].ColorPlayerToken;
    this.playerImageToken.sprite =
GameManager.Instance.MonopolyPlayersVisuals[GameManager.Instance.Current
PlayerIndex].SpritePlayerToken;
    this.Nickname =
LobbyManager.Instance.LocalLobby.Players[GameManager.Instance.CurrentPlaye
rIndex].Data[LobbyManager.KEY_PLAYER_NICKNAME].Value;

```

```

GameManager.Instance.AddPlayer(this);

this.Balance.OnValueChanged += this.HandleBalanceChanged;

if (this.OwnerClientId == NetworkManager.Singleton?.LocalClientId)
{
    UIManagerMonopolyGame.Instance.ButtonRollDiceClicked +=
this.HandleButtonRollDiceClicked;
}
}

public override void OnNetworkDespawn()
{
    this.Balance.OnValueChanged -= this.HandleBalanceChanged;

    if (this.OwnerClientId == NetworkManager.Singleton?.LocalClientId)
    {
        this.Surrender();
        UIManagerMonopolyGame.Instance.ButtonRollDiceClicked -=
this.HandleButtonRollDiceClicked;
    }
}

#region Monopoly

public bool HasFullMonopoly(MonopolyNode monopolyNode, out
MonopolySet monopolySet)
{
    monopolySet = MonopolyBoard.Instance.GetMonopolySet(monopolyNode);

```



```

        return monopolySet?.NodesInSet.Intersect(this.OwnedNodes).Count() ==
monopolySet.NodesInSet.Count;
    }

```

```

    public bool HasPartialMonopoly(MonopolyNode monopolyNode, out
MonopolySet monopolySet)
    {
        monopolySet = MonopolyBoard.Instance.GetMonopolySet(monopolyNode);
        return monopolySet?.NodesInSet.Intersect(this.OwnedNodes).Count() > 1;
    }

```

```

#endregion

```

```

#region Movement

```

```

    private void Move(int steps)
    {
        this.IsAbleToBuild = false;

        const float POSITION_THRESHOLD = 0.01f;

        Vector3 targetPosition;

        bool movedOverStart = false;

        int currentNodeIndex = MonopolyBoard.Instance[this.CurrentNode];

        this.StartCoroutine(MoveCoroutine());

        IEnumerator MoveCoroutine()

```

```

{
    while (steps != 0)
    {
        if (steps < 0)
        {
            ++steps;
            currentNodeIndex = Mathf.Abs(--currentNodeIndex +
MonopolyBoard.Instance.NumberOfNodes);
            currentNodeIndex = currentNodeIndex %
MonopolyBoard.Instance.NumberOfNodes;
        }
        else
        {
            --steps;
            currentNodeIndex = ++currentNodeIndex %
MonopolyBoard.Instance.NumberOfNodes;
        }

        targetPosition =
MonopolyBoard.Instance[currentNodeIndex].transform.position;

        if (MonopolyBoard.Instance.NodeStart ==
MonopolyBoard.Instance[currentNodeIndex])
        {
            movedOverStart = true;
        }

        yield return StartCoroutine(MoveStepCoroutine(targetPosition));
    }
}

```

```

        this.CurrentNode = MonopolyBoard.Instance[currentNodeIndex];

        if (movedOverStart && this.CurrentNode !=
MonopolyBoard.Instance.NodeStart)
        {
            this.Balance.Value += GameManager.Instance.CircleBonus;
        }

        this.HandleLanding();
    }

    IEnumerator MoveStepCoroutine(Vector3 targetPosition)
    {
        while (Vector3.Distance(this.transform.position, targetPosition) >
POSITION_THRESHOLD)
        {
            this.transform.position = Vector3.MoveTowards(this.transform.position,
targetPosition, GameManager.Instance.PlayerMovementSpeed * Time.deltaTime);
            yield return null;
        }

        this.transform.position = targetPosition;
    }
}

private void HandleLanding()
{
    switch (this.CurrentNode.NodeType)
    {
        case MonopolyNode.Type.Tax:

```

```

        this.HandleChanceLanding();
        break;
    case MonopolyNode.Type.Jail:
        this.HandleJailLanding();
        break;
    case MonopolyNode.Type.Start:
        this.HandleStartLanding();
        break;
    case MonopolyNode.Type.Chance:
        this.HandleChanceLanding();
        break;
    case MonopolyNode.Type.SendJail:
        this.HandleSendJailLanding();
        break;
    case MonopolyNode.Type.Property:
        this.HandlePropertyLanding();
        break;
    case MonopolyNode.Type.Gambling:
        this.HandlePropertyLanding();
        break;
    case MonopolyNode.Type.Transport:
        this.HandlePropertyLanding();
        break;
    case MonopolyNode.Type.FreeParking:
        this.HandleFreeParkingLanding();
        break;
    }
}

private IEnumerator PerformTurnCoroutine()

```

```

{
    yield return new WaitUntil(() => this.HasCompletedTurn);

    if (this.isInJail)
    {

```

```

GameManager.Instance.SwitchPlayerForcefullyServerRpc(GameManager.Instance
.ServerParamsCurrentClient);
    }
    else
    {

```

```

GameManager.Instance.SwitchPlayerServerRpc(GameManager.Instance.ServerPar
amsCurrentClient);
    }
}

```

```

[ClientRpc]
public void PerformTurnClientRpc(ClientRpcParams clientRpcParams)
{
    this.HasBuilt = false;
    this.IsTrading = false;
    this.HasRolled = false;
    this.IsAbleToBuild = true;
    this.HasCompletedTurn = false;
    this.CurrentChanceNode = null;
    this.PlayerTradingWith = null;

    this.StartCoroutine(this.PerformTurnCoroutine());
}

```

```

    if (this.isSkipTurn)
    {
        this.isSkipTurn = false;
        this.HasCompletedTurn = true;
        return;
    }

    UIManagerMonopolyGame.Instance.ShowButtonRollDice();
}

#endregion

#region Utility

public void GoToJail()
{
    this.isInJail = true;
    this.turnsInJail = 0;
    this.Move(MonopolyBoard.Instance.GetDistance(this.CurrentNode,
MonopolyBoard.Instance.NodeJail));

    if (this.CurrentChanceNode != null)
    {
        this.CurrentChanceNode = null;
    }

    UIManagerGlobal.Instance.ShowMessageBox(PanelMessageBoxUI.Type.OK,
    UIManagerMonopolyGame.Instance.MessageSentJail,
    PanelMessageBoxUI.Icon.Warning);
}
}

```

```

public void Surrender()
{

this.DeclineTradeServerRpc(GameManager.Instance.ServerParamsCurrentClient);

    UIManagerMonopolyGame.Instance.HidePaymentProperty();
    UIManagerMonopolyGame.Instance.HideButtonRollDice();
    UIManagerMonopolyGame.Instance.HidePaymentChance();
    UIManagerMonopolyGame.Instance.HideMonopolyNode();
    UIManagerMonopolyGame.Instance.HideReceiveTrade();
    UIManagerMonopolyGame.Instance.HideTradeOffer();
    UIManagerMonopolyGame.Instance.HideOffer();

    foreach (MonopolyNode node in this.OwnedNodes)
    {
        node.ResetOwnership();
    }

this.SurrenderServerRpc(GameManager.Instance.ServerParamsCurrentClient);
}

private void ReleaseFromJail()
{
    this.turnsInJail = 0;
    this.isInJail = false;
}

public void HandleJailLanding()

```

```
{
    this.HasCompletedTurn = true;
}
```

```
public void HandleStartLanding()
{
    this.Balance.Value += GameManager.Instance.ExactCircleBonus;
    this.HasCompletedTurn = true;
}
```

```
public void HandleChanceLanding()
{
    this.CurrentChanceNode = MonopolyBoard.Instance.GetChanceNode();

    if (this.CurrentChanceNode.ChanceType != ChanceNodeSO.Type.Penalty)
    {
```

```
    UIManagerMonopolyGame.Instance.ShowInfo(this.CurrentChanceNode.Description, this.CallbackChance);
    }
    else
    {
```

```
    UIManagerMonopolyGame.Instance.ShowPaymentChance(this.CurrentChanceNode.Description, this.CallbackPayment);
    }
```

```
    UIManagerMonopolyGame.Instance.ShowInfoServerRpc(this.CurrentChanceNode.Description, GameManager.Instance.ServerParamsCurrentClient);
```



```
}
```

```
public void HandleSendJailLanding()
```

```
{
```

```
    UIManagerGlobal.Instance.ShowMessageBox(PanelMessageBoxUI.Type.OK,  
    UIManagerMonopolyGame.Instance.MessageSentJail,  
    PanelMessageBoxUI.Icon.Warning);
```

```
        this.GoToJail();
```

```
}
```

```
public void HandleFreeParkingLanding()
```

```
{
```

```
    this.HasCompletedTurn = true;
```

```
}
```

```
[ClientRpc]
```

```
public void GoToJailClientRpc(ClientRpcParams clientRpcParams)
```

```
{
```

```
    UIManagerGlobal.Instance.ShowMessageBox(PanelMessageBoxUI.Type.OK,  
    UIManagerMonopolyGame.Instance.MessageSentJail,  
    PanelMessageBoxUI.Icon.Warning);
```

```
        this.GoToJail();
```

```
}
```

```
[ServerRpc(RequireOwnership = false)]
```

```
private void SurrenderServerRpc(ServerRpcParams serverRpcParams)
```

```

{

GameManager.Instance.RemovePlayerServerRpc(serverRpcParams.Receive.SenderClientId, GameManager.Instance.ServerParamsCurrentClient);

    if
(NetworkManager.Singleton.ConnectedClients.ContainsKey(serverRpcParams.Receive.SenderClientId))
    {
        NetworkClient client =
NetworkManager.Singleton.ConnectedClients[serverRpcParams.Receive.SenderClientId];

        foreach (NetworkObject ownedObject in client.OwnedObjects)
        {
            if (!(bool)ownedObject.IsSceneObject && ownedObject.IsSpawned)
            {
                ownedObject.Despawn();
            }
        }
    }
}

#endregion

#region Property

private void UpgradeProperty()
{

```

```

        if (this.SelectedNode.NodeType == MonopolyNode.Type.Transport ||
            this.SelectedNode.NodeType == MonopolyNode.Type.Gambling)

```

```

        {
            if (!this.SelectedNode.IsMortgaged)
            {

```

```

                UIManagerGlobal.Instance.ShowMessageBox(PanelMessageBoxUI.Type.OK,
                UIManagerMonopolyGame.Instance.MessageCannotUpgradeMaxLevel,
                PanelMessageBoxUI.Icon.Warning);

```

```

            }
            else
            {
                if (this.Balance.Value >= this.SelectedNode.PriceUpgrade)
                {
                    UIManagerMonopolyGame.Instance.HideMonopolyNode();

```

```

                    this.Balance.Value -= this.SelectedNode.PriceUpgrade;

```

```

                    this.HasBuilt = true;
                    this.SelectedNode.Upgrade();

```

```

                }
            }
            else
            {

```

```

                UIManagerGlobal.Instance.ShowMessageBox(PanelMessageBoxUI.Type.OK,
                UIManagerMonopolyGame.Instance.MessageInsufficientFunds,
                PanelMessageBoxUI.Icon.Warning);

```

```

            }
        }
    }

```

```

else if (this.SelectedNode.NodeType == MonopolyNode.Type.Property)
{
    if (!this.HasFullMonopoly(this.SelectedNode, out _) &&
!this.SelectedNode.IsMortgaged)
    {

        UIManagerGlobal.Instance.ShowMessageBox(PanelMessageBoxUI.Type.OK,
        UIManagerMonopolyGame.Instance.MessageCompleteMonopolyRequired,
        PanelMessageBoxUI.Icon.Warning);
    }
    else if (this.HasBuilt)
    {

        UIManagerGlobal.Instance.ShowMessageBox(PanelMessageBoxUI.Type.OK,
        UIManagerMonopolyGame.Instance.MessageAlreadyBuilt,
        PanelMessageBoxUI.Icon.Warning);
    }
    else if (!this.SelectedNode.IsUpgradable)
    {
        if (this.SelectedNode.LocalLevel ==
MonopolyNode.PROPERTY_MAX_LEVEL)
        {

            UIManagerGlobal.Instance.ShowMessageBox(PanelMessageBoxUI.Type.OK,
            UIManagerMonopolyGame.Instance.MessageCannotUpgradeMaxLevel,
            PanelMessageBoxUI.Icon.Warning);
        }
        else
        {

```

```

UIManagerGlobal.Instance.ShowMessageBox(PanelMessageBoxUI.Type.OK,
UIManagerMonopolyGame.Instance.MessageOnlyEvenBuildingAllowed,
PanelMessageBoxUI.Icon.Warning);

```

```

    }
}
else
{
    if (this.Balance.Value >= this.SelectedNode.PriceUpgrade)
    {
        UIManagerMonopolyGame.Instance.HideMonopolyNode();

        this.Balance.Value -= this.SelectedNode.PriceUpgrade;

        this.HasBuilt = true;
        this.SelectedNode.Upgrade();
    }
    else
    {

```

```

UIManagerGlobal.Instance.ShowMessageBox(PanelMessageBoxUI.Type.OK,
UIManagerMonopolyGame.Instance.MessageInsufficientFunds,
PanelMessageBoxUI.Icon.Warning);

```

```

    }
}
}
}

```

```

private void DowngradeProperty()
{

```

```

        if (!this.SelectedNode.IsDowngradable)
        {
            if (this.SelectedNode.LocalLevel ==
MonopolyNode.PROPERTY_MIN_LEVEL)
            {

                UIManagerGlobal.Instance.ShowMessageBox(PanelMessageBoxUI.Type.OK,
                UIManagerMonopolyGame.Instance.MessageCannotDowngradeMinLevel,
                PanelMessageBoxUI.Icon.Warning);
            }
            else
            {

                UIManagerGlobal.Instance.ShowMessageBox(PanelMessageBoxUI.Type.OK,
                UIManagerMonopolyGame.Instance.MessageOnlyEvenBuildingAllowed,
                PanelMessageBoxUI.Icon.Warning);
            }
        }
        else
        {
            UIManagerMonopolyGame.Instance.HideMonopolyNode();

            this.Balance.Value += this.SelectedNode.PriceDowngrade;

            this.SelectedNode.Downgrade();
        }
    }

    public void CallbackMonopolyNode()
    {

```

```

        if
(UIManagerMonopolyGame.Instance.PanelMonopolyNode.MonopolyNodeDialog
Result == PanelMonopolyNodeUI.DialogResult.Upgrade)
    {
        this.UpgradeProperty();
    }
    else
    {
        this.DowngradeProperty();
    }
}

private void HandlePropertyLanding()
{
    if (this.CurrentNode.Owner == null)
    {

UIManagerMonopolyGame.Instance.ShowOffer(this.CurrentNode.NodeSprite,
this.CurrentNode.AffiliatedMonopoly.ColorOfSet,
this.CurrentNode.PricePurchase, this.CallbackPropertyOffer);
    }
    else if (this.CurrentNode.Owner == this || this.CurrentNode.IsMortgaged)
    {
        this.HasCompletedTurn = true;
    }
    else
    {

UIManagerMonopolyGame.Instance.ShowPaymentProperty(this.CurrentNode.No

```

```

deSprite, this.CurrentNode.AffiliatedMonopoly.ColorOfSet,
this.CurrentNode.PriceRent, this.CallbackPayment);
    }
}

private void CallbackPropertyOffer()
{
    if (UIManagerMonopolyGame.Instance.PanelOffer.OfferDialogResult ==
PanelOfferUI.DialogResult.Accepted)
    {
        if (this.Balance.Value >= this.CurrentNode.PricePurchase)
        {
            UIManagerMonopolyGame.Instance.HideOffer();

            this.OwnedNodes.Add(this.CurrentNode);

            this.CurrentNode.UpdateOwnership(NetworkManager.Singleton.LocalClientId);
            this.Balance.Value -= this.CurrentNode.PricePurchase;

            this.HasCompletedTurn = true;
        }
        else
        {
            UIManagerGlobal.Instance.ShowMessageBox(PanelMessageBoxUI.Type.OK,
            UIManagerMonopolyGame.Instance.MessageInsufficientFunds,
            PanelMessageBoxUI.Icon.Warning);
        }
    }
}

```



```

else
{
    UIManagerMonopolyGame.Instance.HideOffer();
    this.HasCompletedTurn = true;
}
}

```

```

[ClientRpc]
private void AddNodeClientRpc(int monopolyNodeIndex, ClientRpcParams
clientRpcParams)
{
    this.OwnedNodes.Add(MonopolyBoard.Instance[monopolyNodeIndex]);
}

```

```

[ClientRpc]
private void RemoveNodeClientRpc(int monopolyNodeIndex, ClientRpcParams
clientRpcParams)
{
    this.OwnedNodes.Remove(MonopolyBoard.Instance[monopolyNodeIndex]);
}

```

```

[ServerRpc(RequireOwnership = false)]
private void AddNodeServerRpc(int monopolyNodeIndex, ulong ownerdId,
ServerRpcParams serverRpcParams)
{
    this.AddNodeClientRpc(monopolyNodeIndex,
GameManager.Instance.GetRedirectionRpc(ownerdId));
}

```

```

[ServerRpc(RequireOwnership = false)]

```

```

private void RemoveNodeServerRpc(int monopolyNodeIndex, ulong ownerdId,
ServerRpcParams serverRpcParams)
{
    this.RemoveNodeClientRpc(monopolyNodeIndex,
GameManager.Instance.GetRedirectionRpc(ownerdId));
}

```

```

#endregion

```

```

#region GUI Callbacks

```

```

private void CallbackChance()
{
    if (UIManagerMonopolyGame.Instance.PanelInfo.InfoDialogResult ==
PanelInfoUI.DialogResult.Confirmed)
    {
        switch (this.CurrentChanceNode.ChanceType)
        {
            case ChanceNodeSO.Type.Reward:
            {
                this.Balance.Value += this.CurrentChanceNode.Reward;
                this.HasCompletedTurn = true;
            }
            break;
            case ChanceNodeSO.Type.SkipTurn:
            {
                this.isSkipTurn = true;
                this.HasCompletedTurn = true;
            }
            break;
        }
    }
}

```

```

case ChanceNodeSO.Type.SendJail:
    this.GoToJail();
    break;
case ChanceNodeSO.Type.MoveForward:
    {
        this.CurrentChanceNode = null;
        GameManager.Instance.RollDice();
        UIManagerMonopolyGame.Instance.ShowDiceAnimation();
        this.Move(GameManager.Instance.TotalRollResult);
    }
    break;
case ChanceNodeSO.Type.MoveBackwards:
    {
        this.CurrentChanceNode = null;
        GameManager.Instance.RollDice();
        UIManagerMonopolyGame.Instance.ShowDiceAnimation();
        this.Move(-GameManager.Instance.TotalRollResult);
    }
    break;
}
}
}

private void CallbackPayment()
{
    if (this.CurrentNode.NodeType == MonopolyNode.Type.Chance ||
this.CurrentNode.NodeType == MonopolyNode.Type.Tax)
    {
        if (this.Balance.Value >= this.CurrentChanceNode.Penalty)
        {

```

```
UIManagerMonopolyGame.Instance.HidePaymentChance();
```

```
this.Balance.Value -= this.CurrentChanceNode.Penalty;
```

```
this.HasCompletedTurn = true;
```

```
}
```

```
else
```

```
{
```

```
    UIManagerGlobal.Instance.ShowMessageBox(PanelMessageBoxUI.Type.OK,
```

```
    UIManagerMonopolyGame.Instance.MessageInsufficientFunds,
```

```
    PanelMessageBoxUI.Icon.Warning);
```

```
}
```

```
}
```

```
else
```

```
{
```

```
    if (this.Balance.Value >= this.CurrentNode.PriceRent)
```

```
    {
```

```
        UIManagerMonopolyGame.Instance.HidePaymentProperty();
```

```
        this.Balance.Value -= this.CurrentNode.PriceRent;
```

```
        this.SendBalanceServerRpc(this.CurrentNode.PriceRent,
```

```
        this.CurrentNode.Owner.OwnerClientId,
```

```
        GameManager.Instance.ServerParamsCurrentClient);
```

```
        this.HasCompletedTurn = true;
```

```
}
```

```
else
```

```
{
```

```

UIManagerGlobal.Instance.ShowMessageBox(PanelMessageBoxUI.Type.OK,
UIManagerMonopolyGame.Instance.MessageInsufficientFunds,
PanelMessageBoxUI.Icon.Warning);

```

```

    }
}

```

```

public void CallbackTradeOffer()
{
    if
(UIManagerMonopolyGame.Instance.PanelTradeOffer.TradeOfferDialogResult ==
PanelTradeOfferUI.DialogResult.Offer)

```

```

    {
        UIManagerMonopolyGame.Instance.SendTradeOffer();
    }
    else
    {
        this.IsTrading = false;

        if (!this.HasRolled)
        {
            UIManagerMonopolyGame.Instance.ShowButtonRollDice();
        }
    }
}

```

```

public void CallbackReceiveTrade()
{
    UIManagerMonopolyGame.Instance.HideReceiveTrade();
}

```

```

        if
        (UIManagerMonopolyGame.Instance.PanelReceiveTrade.ReceiveTradeDialogRes
ult == PanelReceiveTradeUI.DialogResult.Accept)
        {

            this.AcceptTradeServerRpc(UIManagerMonopolyGame.Instance.PanelReceiveTra
de.Credentials, GameManager.Instance.ServerParamsCurrentClient);
        }
        else
        {

            this.DeclineTradeServerRpc(GameManager.Instance.ServerParamsCurrentClient);
        }
    }

    private void HandleButtonRollDiceClicked()
    {
        this.HasRolled = true;

        UIManagerMonopolyGame.Instance.HidePaymentProperty();
        UIManagerMonopolyGame.Instance.HidePaymentChance();
        UIManagerMonopolyGame.Instance.HideMonopolyNode();
        UIManagerMonopolyGame.Instance.HideReceiveTrade();
        UIManagerMonopolyGame.Instance.HideTradeOffer();
        UIManagerMonopolyGame.Instance.HideOffer();

        GameManager.Instance.RollDice();
        UIManagerMonopolyGame.Instance.ShowDiceAnimation();
    }

```

```

    if (this.isInJail)
    {
        if (GameManager.Instance.HasRolledDouble || ++this.turnsInJail >
GameManager.Instance.MaxTurnsInJail)
        {
            this.ReleaseFromJail();
            this.Move(GameManager.Instance.TotalRollResult);
        }
        else
        {
            this.HasCompletedTurn = true;
        }
    }
    else
    {
        this.Move(GameManager.Instance.TotalRollResult);
    }
}

```

```

[ServerRpc(RequireOwnership = false)]
public void DeclineTradeServerRpc(ServerRpcParams serverRpcParams)
{
    this.CallbackTradeResponseClientRpc(false,
GameManager.Instance.ClientParamsCurrentClient);
}

```

```

[ClientRpc]
private void CallbackTradeResponseClientRpc(bool result, ClientRpcParams
clientRpcParams)
{

```

```

        if (GameManager.Instance.CurrentPlayer == null ||
!GameManager.Instance.CurrentPlayer.IsTrading)
        {
            return;
        }

        GameManager.Instance.CurrentPlayer.IsTrading = false;

        if (result)
        {

            UIManagerGlobal.Instance.ShowMessageBox(PanelMessageBoxUI.Type.OK,
            UIManagerMonopolyGame.Instance.MessageTradeAccepted,
            PanelMessageBoxUI.Icon.Warning);
        }
        else
        {

            UIManagerGlobal.Instance.ShowMessageBox(PanelMessageBoxUI.Type.OK,
            UIManagerMonopolyGame.Instance.MessageTradeDeclined,
            PanelMessageBoxUI.Icon.Warning);
        }

        if (!GameManager.Instance.CurrentPlayer.HasRolled)
        {
            UIManagerMonopolyGame.Instance.ShowButtonRollDice();
        }
    }

    [ServerRpc]

```



```

public void AcceptTradeServerRpc(TradeCredentials tradeCredentials,
ServerRpcParams serverRpcParams)
{
    UIManagerMonopolyGame.Instance.HideMonopolyNode();

    MonopolyPlayer sender =
GameManager.Instance.GetPlayerById(tradeCredentials.SenderId);
    MonopolyPlayer receiver =
GameManager.Instance.GetPlayerById(tradeCredentials.ReceiverId);

    if (sender == null || receiver == null)
    {
        return;
    }

    if (tradeCredentials.SenderNodeIndex != -1)
    {
        MonopolyBoard.Instance[tradeCredentials.SenderNodeIndex].UpdateOwnership(tr
adeCredentials.ReceiverId);
        receiver.AddNodeServerRpc(tradeCredentials.SenderNodeIndex,
tradeCredentials.ReceiverId, GameManager.Instance.ServerParamsCurrentClient);
        sender.RemoveNodeServerRpc(tradeCredentials.SenderNodeIndex,
tradeCredentials.SenderId, GameManager.Instance.ServerParamsCurrentClient);
    }

    if (tradeCredentials.ReceiverNodeIndex != -1)
    {

```

```

MonopolyBoard.Instance[tradeCredentials.ReceiverNodeIndex].UpdateOwnership
(tradeCredentials.SenderId);

    sender.AddNodeServerRpc(tradeCredentials.ReceiverNodeIndex,
tradeCredentials.SenderId, GameManager.Instance.ServerParamsCurrentClient);
    receiver.RemoveNodeServerRpc(tradeCredentials.ReceiverNodeIndex,
tradeCredentials.ReceiverId, GameManager.Instance.ServerParamsCurrentClient);
}

if (tradeCredentials.SenderOffer != 0)
{
    sender.SendBalanceServerRpc(tradeCredentials.SenderOffer,
receiver.OwnerClientId, GameManager.Instance.ServerParamsCurrentClient);

GameManager.Instance.GetPlayerById(tradeCredentials.SenderId).PayChargeCli
entRpc(tradeCredentials.SenderOffer,
GameManager.Instance.GetRedirectionRpc(tradeCredentials.SenderId));
}

if (tradeCredentials.ReceiverOffer != 0)
{
    sender.SendBalanceServerRpc(tradeCredentials.ReceiverOffer,
sender.OwnerClientId, GameManager.Instance.ServerParamsCurrentClient);

GameManager.Instance.GetPlayerById(tradeCredentials.ReceiverId).PayChargeCli
entRpc(tradeCredentials.ReceiverOffer,
GameManager.Instance.GetRedirectionRpc(tradeCredentials.ReceiverId));
}

```

```

        this.CallbackTradeResponseClientRpc(true,
GameManager.Instance.ClientParamsCurrentClient);
    }

#endregion

#region Updating Balance

private void HandleBalanceChanged(int previousValue, int newValue)
{
    this.OnBalanceUpdated?.Invoke();
}

[ClientRpc]
private void PayChargeClientRpc(int amount, ClientRpcParams
clientRpcParams)
{
    this.Balance.Value -= amount;
}

[ServerRpc(RequireOwnership = false)]
private void SendBalanceServerRpc(int amount, ulong receiverClientId,
ServerRpcParams serverRpcParams)
{
    this.ReceiveBalanceClientRpc(amount, receiverClientId,
GameManager.Instance.GetRedirectionRpc(receiverClientId));
}

[ClientRpc]

```

```
private void ReceiveBalanceClientRpc(int amount, ulong receiverClientId,
ClientRpcParams clientRpcParams)
{
    GameManager.Instance.GetPlayerById(receiverClientId).Balance.Value +=
amount;
}

#endregion
}
```

Факультет інформатики та обчислювальної техніки

Кафедра інформатики та програмної інженерії

“ЗАТВЕРДЖЕНО”

Керівник роботи

_____ Максим ГОЛОВЧЕНКО

“ ” _____ 2024 р.

ГРА ДЛЯ МОБІЛЬНОГО ПРИСТ«МОНОПОЛІЯ»

Програма та методика тестування

КП.ІП-1402.045490.04.51

“ПОГОДЖЕНО”

Керівник роботи:

_____ Максим ГОЛОВЧЕНКО

Консультант:

_____ Максим ГОЛОВЧЕНКО

Виконавець:

_____ Денис БАБІЧ

Київ – 2024

ЗМІСТ

1 ОБ’ЄКТ ВИПРОБУВАНЬ..... 3

2 МЕТА ТЕСТУВАННЯ..... 4

3 МЕТОДИ ТЕСТУВАННЯ..... 5

4 ЗАСОБИ ТА ПОРЯДОК ТЕСТУВАННЯ..... 6

1 ОБ'ЄКТ ВИПРОБУВАНЬ

Об'єктом випробування є розроблений мобільний ігровий застосунок – реалізація настільної гри Монополія з багатокористувацьким режимом, спеціально адаптована для операційної системи Android.

Для тестування було вирішено використовувати мануальне функціональне тестування. Проведені тести охоплюють різні аспекти функціональності та сумісності з метою забезпечення високоякісного і стабільно працюючого продукту.

Варто зазначити, що для тестування були використані як емулятори, так і справжні пристрої Android для перевірки програми на різних версіях операційної системи та на пристроях з різним розміром екрану. Це дозволяє нам гарантувати стабільну роботу програми на широкому спектрі пристроїв, від різних моделей смартфонів від різних виробників.

Розроблена методи тестування охоплює різноманітні аспекти гри, включаючи багатокористувацький режим та взаємодію з графічним користувацьким інтерфейсом. Під час тестування буде перевірено, що гра працює оптимально і надає належний рівень інтерактивного ігрового досвіду для користувачів. Додатково, буде здійснено тестування на різних типах пристроїв, перевіряючи продуктивність та енергоспоживання програми на пристроях з різними процесорами та роздільними здатностями екранів, версіями операційної системи.

2 МЕТА ТЕСТУВАННЯ

Метою тестування є наступне:

- Перевірка функціональності. Виконання тестів для визначення відповідності програми функціональним вимогам. Перевірка коректності роботи багатокористувацького режиму та забезпечення інтерактивного досвіду для кількох користувачів одночасно.
- Сумісність з різними версіям операційними системами Android. Тестування на різних версіях операційної системи. Виявлення та виправлення можливих проблем, що виникають під час роботи на різних версіях операційної системи.
- Виявлення та усунення проблем. Аналіз помилок, виявлених під час тестування, з метою їх усунення перед випуском продукту. Впровадження заходів для забезпечення стабільності та ефективності додатку.
- Оцінка зручності та естетичності графічного інтерфейсу. Виявлення та виправлення можливих проблем, пов'язаних із візуальною частиною програми для покращення загального користувацького досвіду.

3 МЕТОДИ ТЕСТУВАННЯ

Для тестування програмного забезпечення використовуються такі методи:

- Функціональне тестування. Цей підхід дозволяє самотужки перевіряти функціонал програми, відповідності реальної поведінки програмного забезпечення очікуваній. Зокрема, тут можна провести тестування багатокористувацького режиму, переконатися в коректності взаємодії між користувачами та вірності оновлення даних на різних пристроях;
- Тестування «чорної скриньки». Це тестування зорієнтоване на перевірку зовнішнього вигляду та функціональності програми без необхідності вдаватися в деталі її внутрішньої реалізації. В даному випадку, можливо перевірити правильність валідації вхідних даних та коректності вихідних даних та поведінки;
- Мануальне тестування. Цей тип тестування вдало поєднується з функціональним тестуванням, оскільки дозволяє самостійно перевірити коректність роботи ключових функціональних задач застосунку. Мануальне тестування є ключовим етапом у впровадженні гри, оскільки воно дозволяє нам не лише перевірити функціонал на відповідність очікуванням, але і виявити можливі проблеми, які можуть виникнути в реальних умовах використання. Такий підхід сприяє створенню продукту, який відповідає високим стандартам якості та задовольняє потреби кінцевого користувача.

Також варто зазначити, що оскільки розроблюваним застосунком є ігровий мобільний додаток, то написання різних типів тестів є бажаним та, навіть, необхідним, але через важкість створення та впровадження автоматичних тестів різного рівня саме у сфері розробки відеоігор було вирішено зупинитися саме на вищеписаних типах тестів.

4 ЗАСОБИ ТА ПОРЯДОК ТЕСТУВАННЯ

Тестування виконується мануально з виконання перевірки функціональних вимог за методикою тестування «чорної скриньки». Для того, щоб перевірити працездатність та відмовостійкість застосунку, необхідно провести наступні тестування:

- Динамічне тестування на відповідність функціональним вимогам;
- Тестування на мобільних пристроях з різною роздільною здатністю екрану;
- Тестування на виведення повідомлень про помилку;
- Тестування працездатності програми у випадку відсутності з'єднання до мережі;
- Тестування інтерфейсу користувача.

Факультет інформатики та обчислювальної техніки

Кафедра інформатики та програмної інженерії

“ЗАТВЕРДЖЕНО”

Керівник роботи

_____ Максим ГОЛОВЧЕНКО

“ ” _____ 2024 р.

ГРА ДЛЯ МОБІЛЬНОГО ПРИСТ«МОНОПОЛІЯ»

Керівництво користувача

КП.ІП-1402.045490.05.34

“ПОГОДЖЕНО”

Керівник роботи:

_____ Максим ГОЛОВЧЕНКО

Консультант:

_____ Максим ГОЛОВЧЕНКО

Виконавець:

_____ Денис БАБІЧ

Київ – 2024

ЗМІСТ

1 ПРИЗНАЧЕННЯ ПРОГРАМИ.....	3
2 ПІДГОТОВКА ДО РОБОТИ З ПРОГРАМНИМ ЗАБЕЗПЕЧЕННЯМ.....	4
2.1 Системні вимоги для коректної роботи.....	4
2.2 Завантаження застосунку.....	4
2.3 Перевірка коректної роботи.....	6
3 ВИКОНАННЯ ПРОГРАМИ.....	9

1 ПРИЗНАЧЕННЯ ПРОГРАМИ

Гра для мобільного пристрою «Монополія» – це мобільний застосунок, який використовується для розважальних цілей, розвитку стратегічного мислення та призначена для широко спектру користувачів, людей різного віку, які мають інтерес до настільних ігор, покрокових стратегій та змагального ігрового процесу, де завдяки підвищенню стабільності, продуктивності додатку, покращеному користувацькому графічному інтерфейсу та змінених ігрових правил, які забезпечують більш швидкий темп гри у порівнянні з класичною версією, що у результаті створює покращений ігровий досвід для користувачів.

2 ПІДГОТОВКА ДО РОБОТИ З ПРОГРАМНИМ ЗАБЕЗПЕЧЕННЯМ

2.1 Системні вимоги для коректної роботи

Для успішної роботи даного застосунку необхідна наявність наявність мобільного пристрою з операційною системою на базі Android з версією 5.1 або вище, наявний стабільний доступ до інтернету, і не менше 35 МБ вільної пам'яті.

Мінімальна конфігурація технічних засобів:

- тип процесору: Qualcomm Snapdragon 400;
- об'єм ОЗП: 1 Гб;
- підключення до мережі Інтернет зі швидкість від 5 мегабіт.

Рекомендована конфігурація технічних засобів:

- тип процесору: Qualcomm Snapdragon 8 Gen 2;
- об'єм ОЗП: 2 Гб;
- підключення до мережі Інтернет зі швидкість від 15 мегабіт.

2.2 Завантаження застосунку

На даний момент застосунок можна встановити власноруч, використовуючи відповідний арк-файл, який можна завантажити з GitHub-репозиторію. Для цього спершу необхідно завантажити цей арк-файл на мобільний пристрій, а потім, використовуючи стандартну процедуру інсталяції встановити його на пристрій. У випадку успішного виконання етапі встановлення, користувач отримує новий встановлений застосунок.

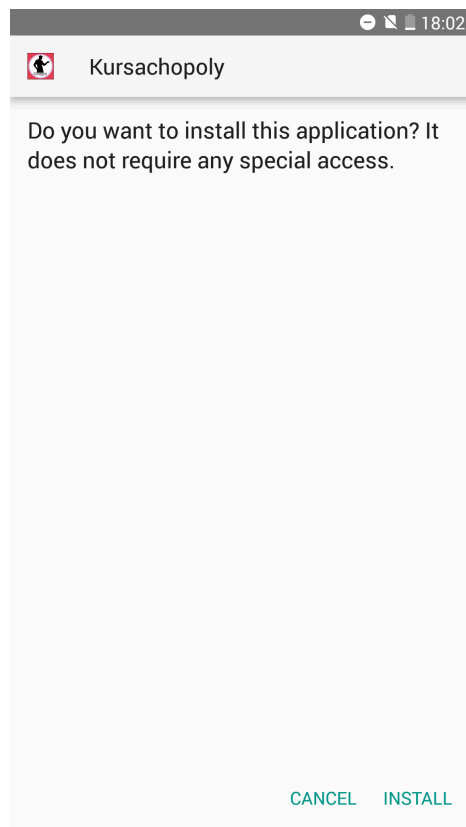


Рисунок 2.1 – Вікно інсталяції ігрового застосунку після відкриття .apk файлу

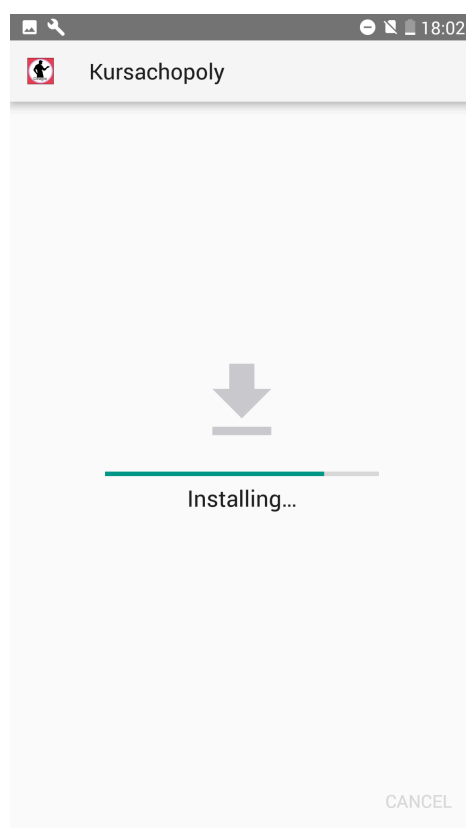


Рисунок 2.2 – Вікно інсталяції під час процесу встановлення гри

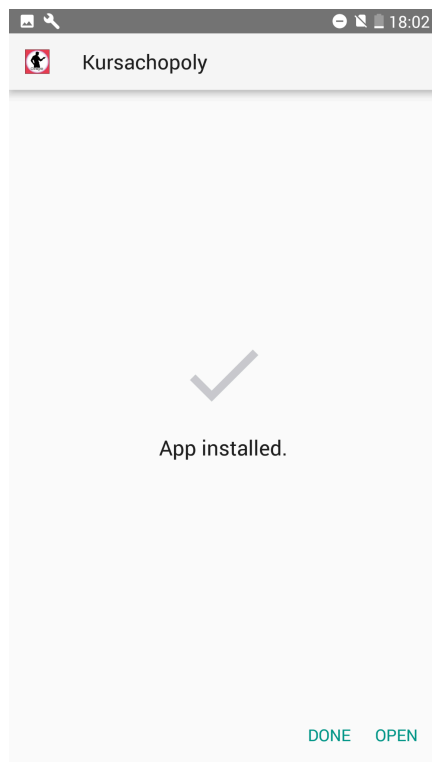


Рисунок 2.3 – Вікно інсталяції після успішного процесу встановлення гри

Після успішного процесу виконання інсталяції користувач отримує новий встановлений застосунок з відповідним зображенням.



Рисунок 2.4 – Встановлена гра

2.3 Перевірка коректної роботи

По завершенню встановлення додатка на робочому столі мобільного пристрою повинна відобразитись іконка даного застосунку. У разі, якщо дана іконка не з'явилась, то встановлення відбулось не успішно. Інакше користувач має змогу запустити додаток, клацнувши на його іконку. Після натискання на неї і відкриття гри починається підключення до ігрового координатора на рисунку 2.5.

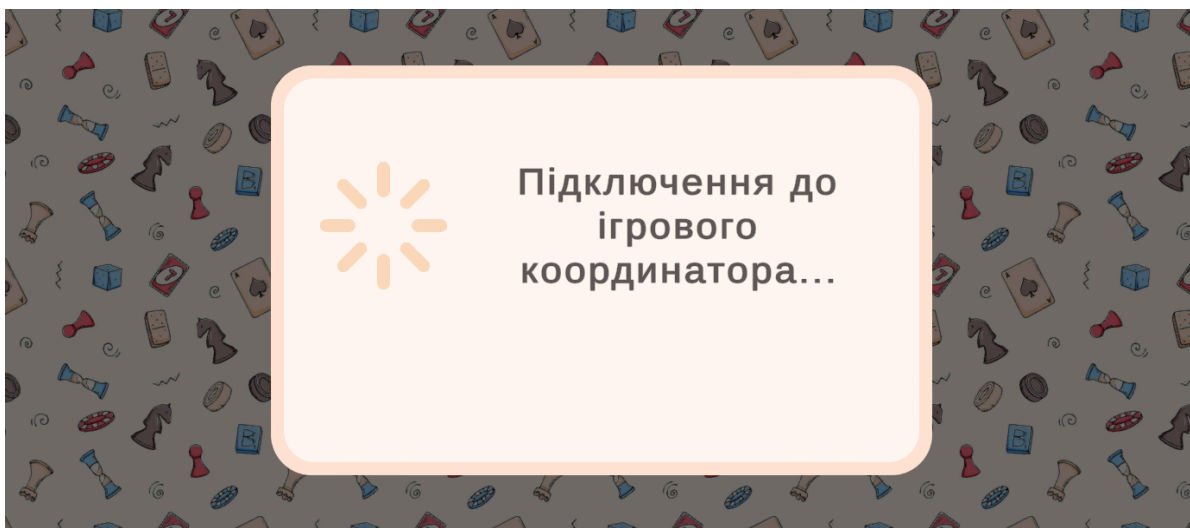


Рисунок 2.5 – Підключення до ігрового координатора

У випадку наявності інтернету на пристрої та доступності ігрових сервісів гравець опиняється у головному меню гри.

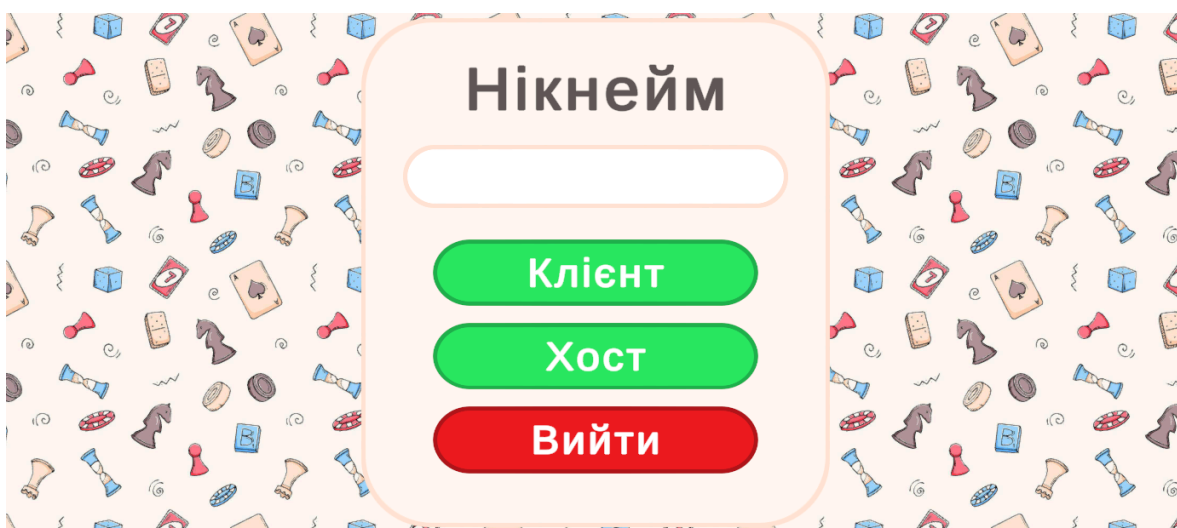


Рисунок 2.6 – Головне меню гри

У випадку невдалого підключення до ігрового координатора виводиться відповідне повідомлення про помилку на рисунок 2.7.

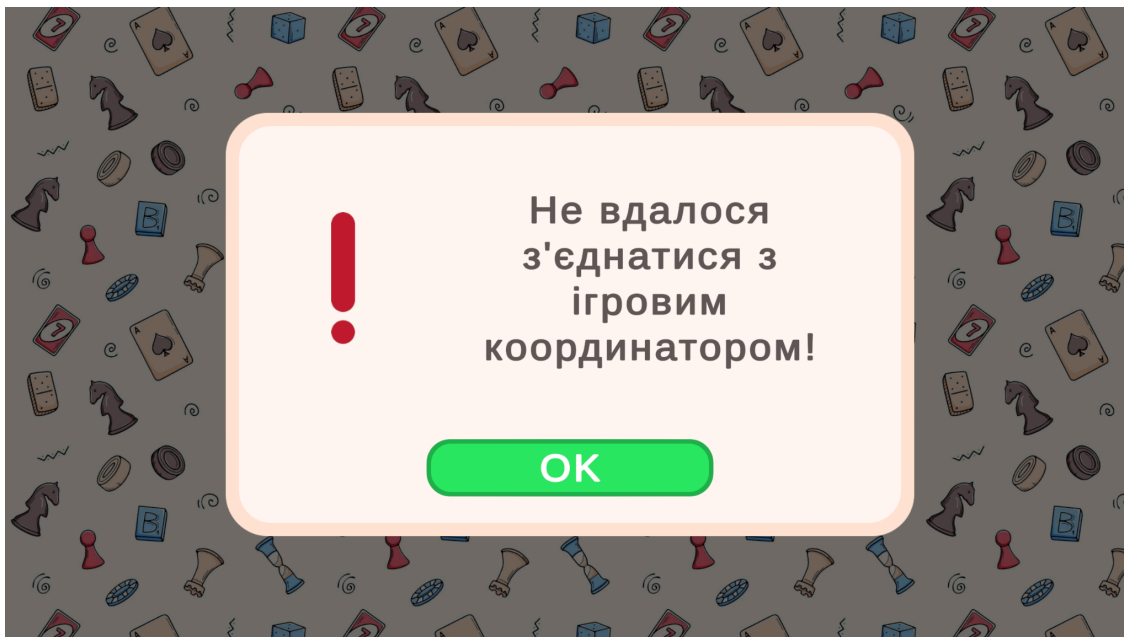


Рисунок 2.7 – Повідомлення у випадку невдалого підключення до ігрового координатора

3 ВИКОНАННЯ ПРОГРАМИ

При запуску програмного застосунку і наявності підключення до мережі інтернет користувач опиниться у головному меню, де буде необхідно ввести своє ім'я для продовження будь-яких дій.

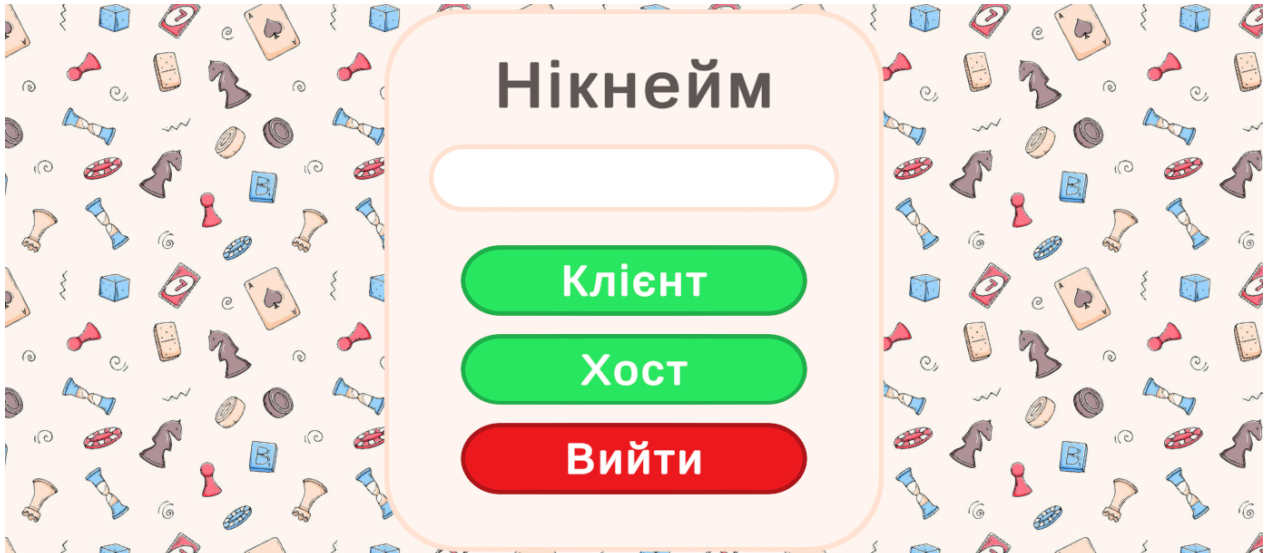


Рисунок 3.1 – Початковий стан головного меню

У відповідне текстове поле користувач має ввести своє ім'я, яке може складатися з літер англійської та української абетки, може містити пробіли, але встановлена межа по кількості символів у 10 знаків.

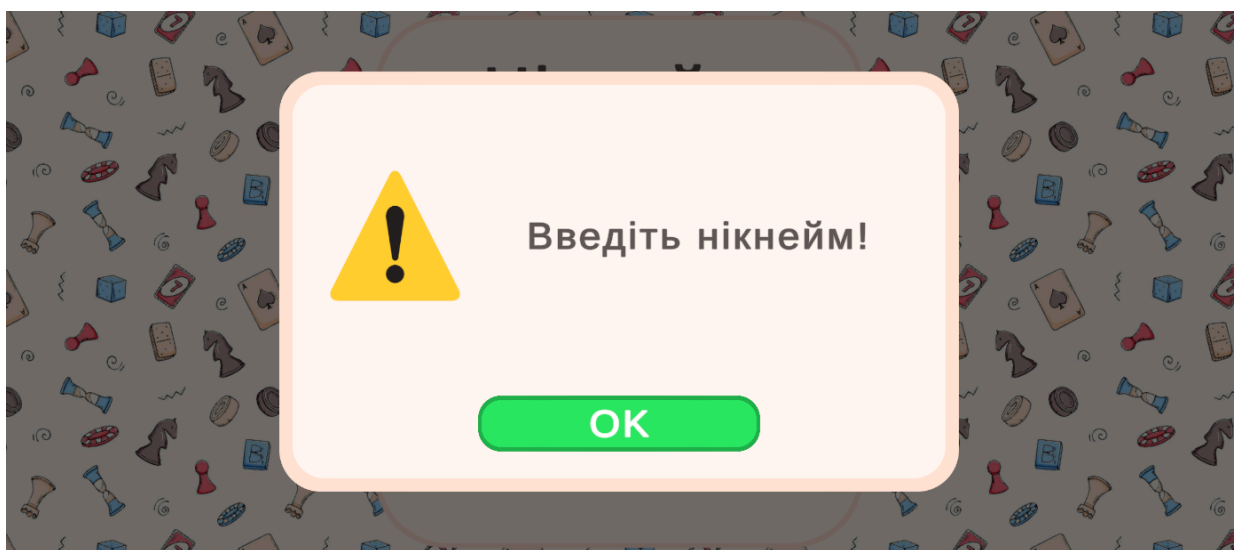


Рисунок 3.2 – У випадку спроби створення, або підключення до ігрової кімнати без введеного імені користувач отримає відповідне попередження

У випадку введення вдалого імені користувачу надається право створити власну кімнату, або підключитися до вже існуючої. У випадку вибору створення власної кімнати користувач буде очікувати завантаження ігрового лобі та переміститься у нього у випадку вдалого завантаження.

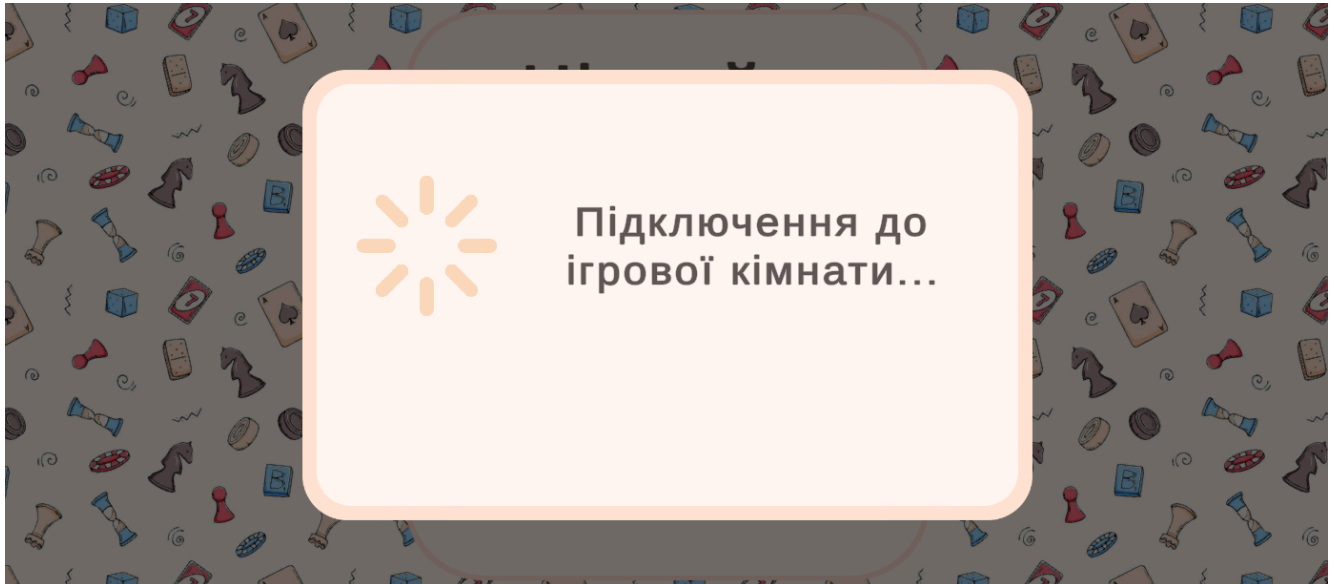


Рисунок 3.3 – Очікування завантаження ігрового лобі

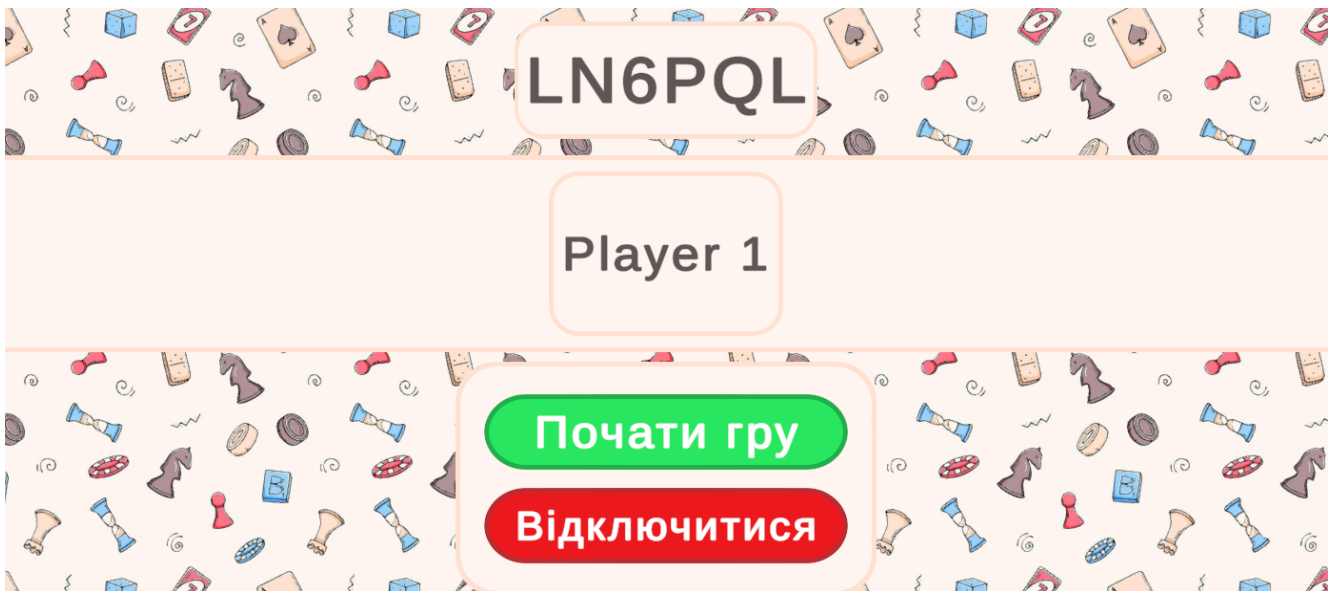


Рисунок 3.4 – Користувач успішно завантажився в ігрове лобі

У випадку, якщо користувач бажає приєднатися до вже створеної ігрової сесії, то необхідно ввести правильний код сесії, попередньо натиснувши на копнку “Клієнт” на рисунку 3.5.

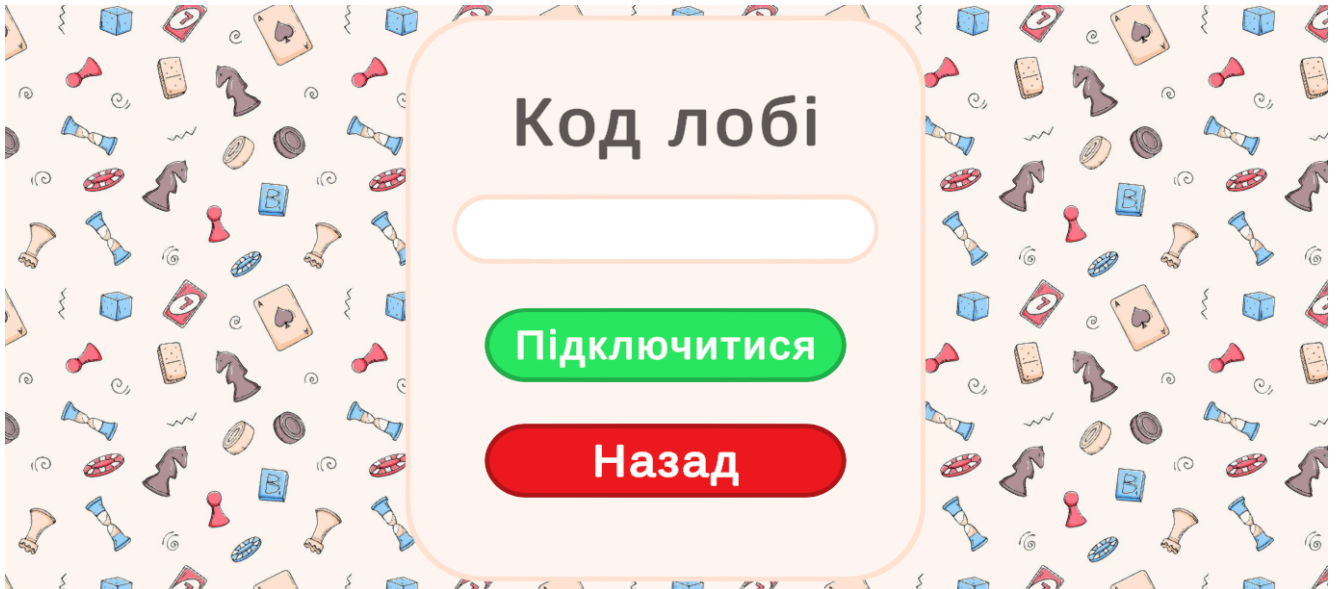


Рисунок 3.5 – Вікно введення коду ігрового лобі

У випадку невдалої спроби підключення до ігрової сесії користувачу буде повідомлено про причину через яку це трапилося.

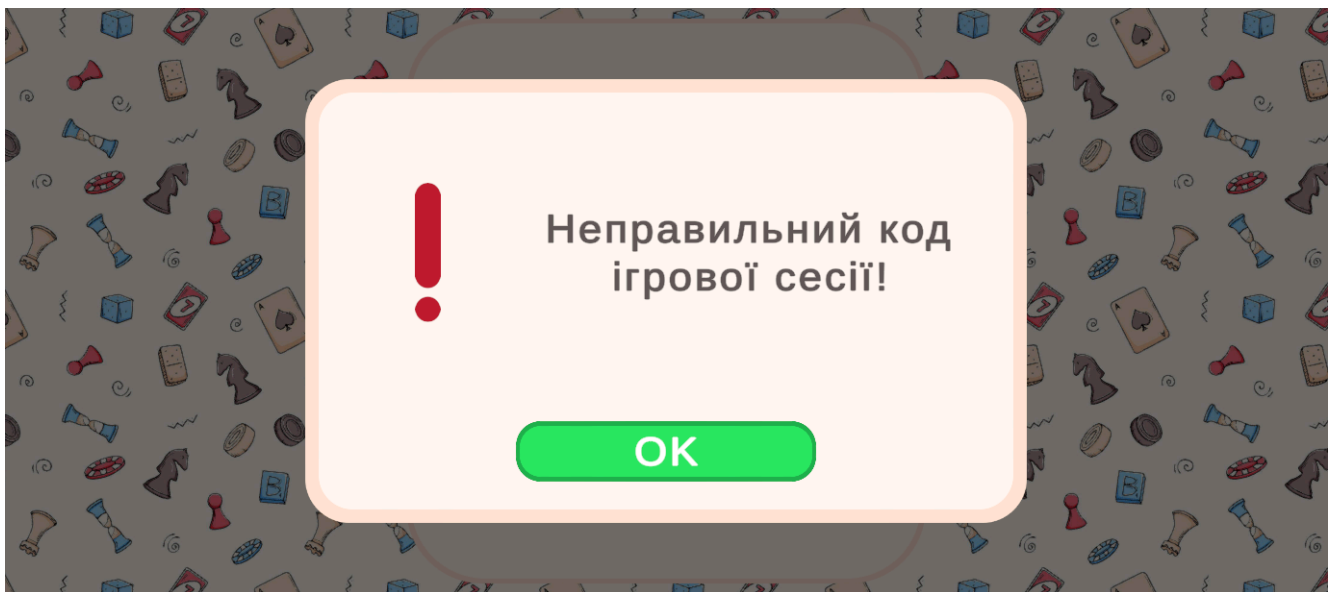


Рисунок 3.6 – Повідомлення у випадку правильно введеного коду ігрової сесії

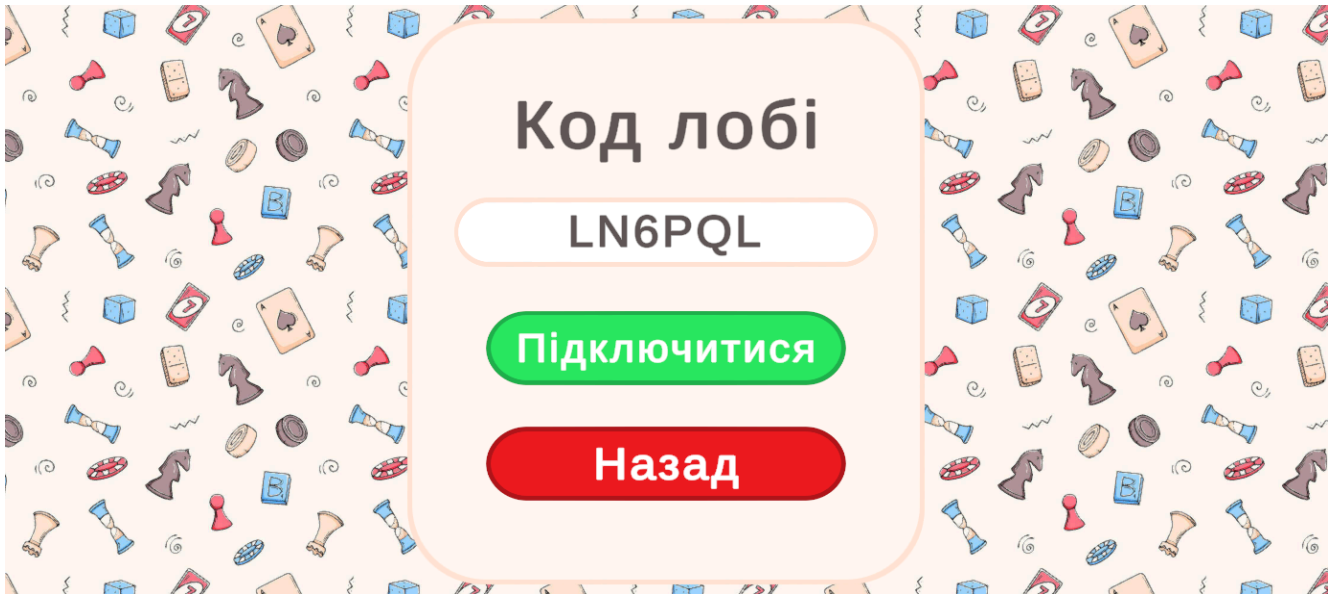


Рисунок 3.7 – Коректно введений код ігрової сесії



Рисунок 3.8 – Обидва гравці успішно завантажилися до ігрового лобі

Лише користувач-хост має право розпочинати гру, натиснувши відповідну кнопку “Почати гру”. Звичайний клієнт має лише право відключитись від ігрової кімнати у випадку необхідності за натисканням відповідної кнопки на рисунку 3.9. У випадку спроби почати матч з замалою кількістю гравців користувача очікує попередження про відповідну неправомірну дію, оскільки мінімальна кількість гравців для початку – 2 на рисунку 3.10.



Рисунок 3.9 – Вид ігрового лобі від лиця клієнта

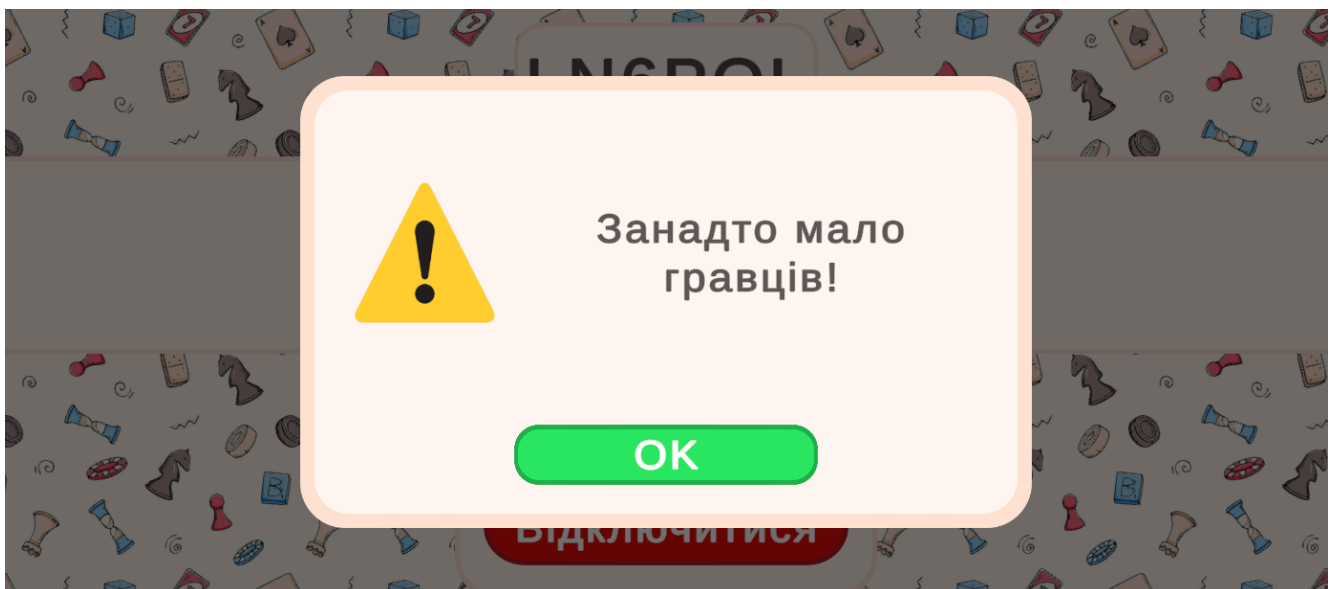


Рисунок 3.10 – Невдала спроба розпочати гру

Після розпочатої гри ігрове поле має вигляд як на рисунку 3.11. Варто зауважити, що кнопка кидання кубика видна лише тому гравцю, чий зараз хід виконується.

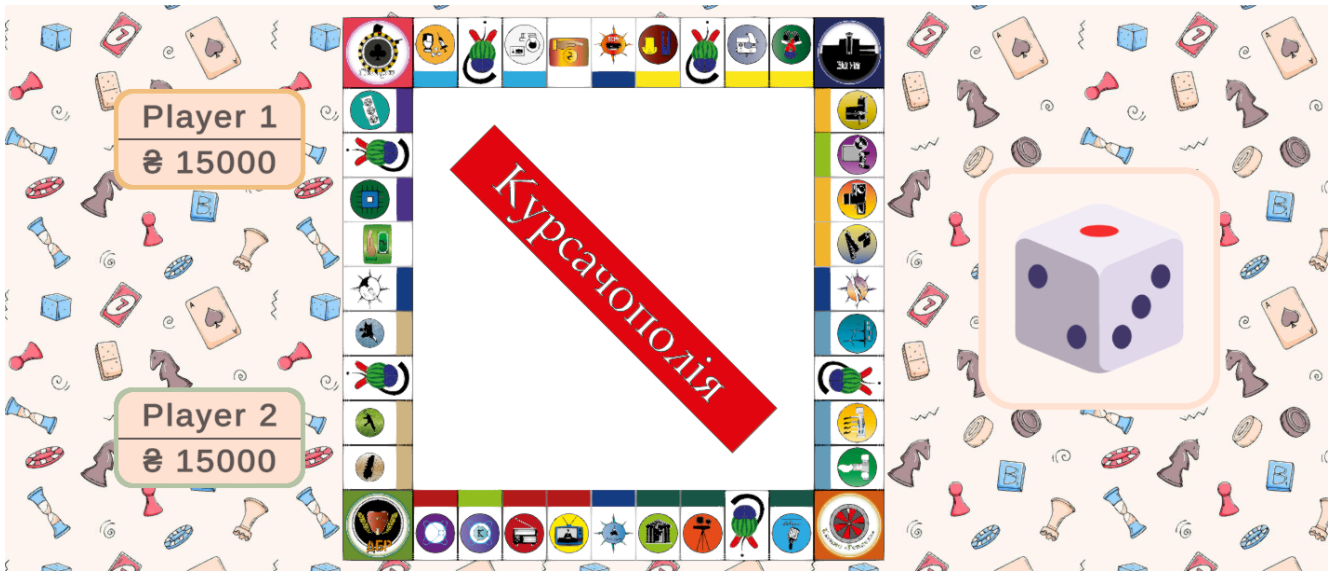


Рисунок 3.11 – Завантажена гра, наявність кнопки кидання кубика свідчить про чергу гравця робити хід

Після натискання на кнопку кидання кубика гравцю демонструється значення, які випали і фішка гравця здійснює пересування ігровим полем.

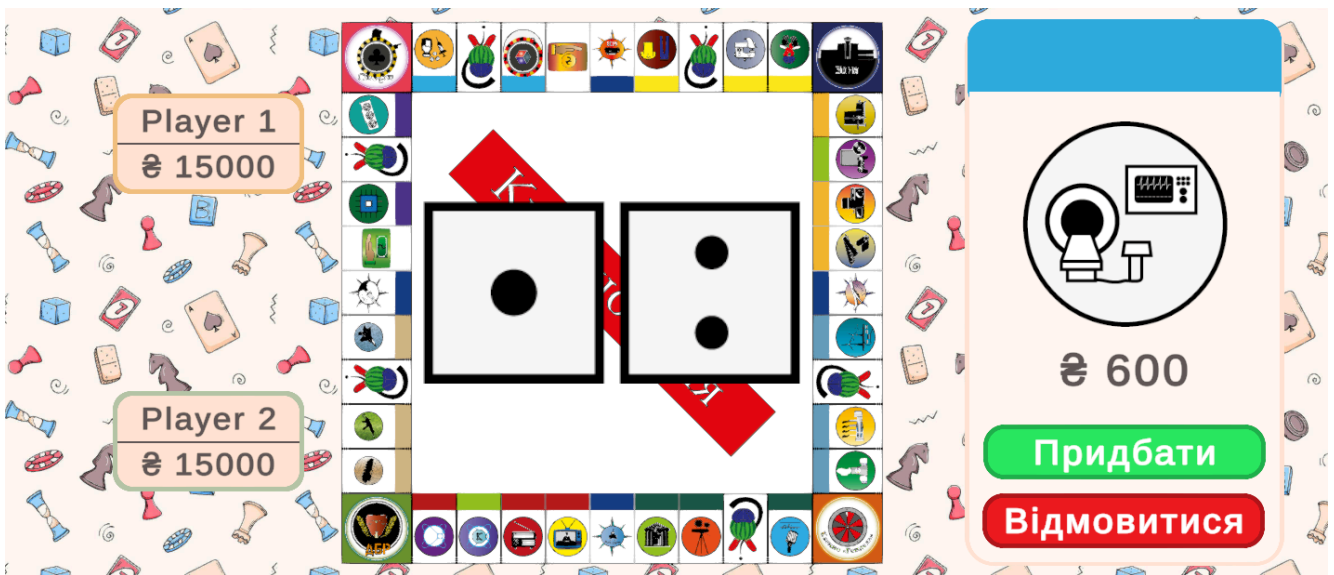


Рисунок 3.12 – Гравець здійснив хід та приземлився на поле власності, яке йому було запропоновано придбати

У випадку, якщо гравець має достатньо коштів та бажає викупити поле, то користувач може скористатися кнопкою “Придбати”. Інакше – гравець може відмовитися від покупки натиснувши відповідну кнопку “Відмовитися”. У обох випадках хід буде передано наступному гравцю, якщо тільки не гравець не

отримав парне значення з кубиків, у цьому випадку поточному гравцю надається право кидати кубик ще раз.



Рисунок 3.13 – У випадку придбання поля власності поле на дошці змінює колір на колір рамки гравця, який його придбав



Рисунок 3.14 – Стан поля після того, як другий гравець виконав свій хід та придбав поле у свою власність

Гравець, якому випало право ходити має можливість запропонувати іншому гравцю здійснити обмін полями власності. Для цього потрібно натиснути на панель іншого гравця, яка знаходиться з лівого боку екрану.



Рисунок 3.15 – Відкрита пропозиція обміну

Натискаючи на поля на дошці, які можна обмінювати гравець може запропонувати здійснити обмін на своїх умовах.



Рисунок 3.16 – Приклад заповненої пропозиції обміну

У випадку натискання на кнопку скасувати обміну буде скасовано і гравець отримує право кинути кубик, у випадку, якщо пропозицію буде надіслано, то гравець буде очікувати відповіді від іншого користувача.



Рисунок 3.17 – Вигляд екрану гравця у випадку отримання пропозиції обміну

Отримувачу пропозиції надається можливість прийняти, або відхилити пропозицію натискаючи на відповідні кнопки. У випадку згоди буде виконано обміну власністю та будуть оновлення балансу гравців та власників полів.

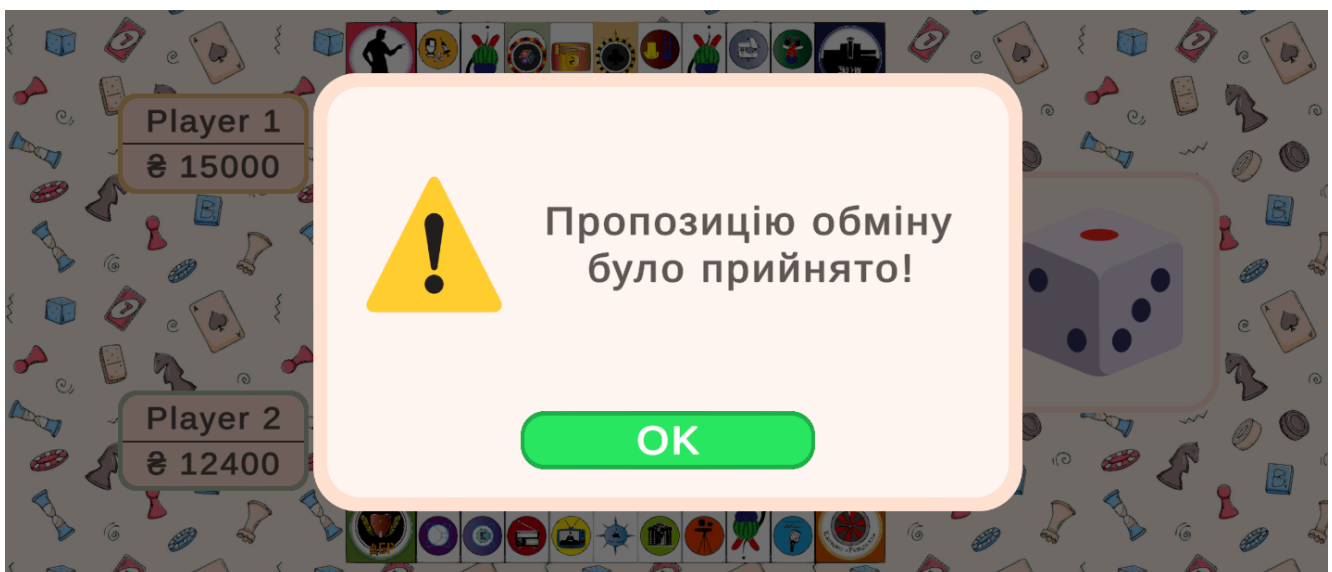


Рисунок 3.18 – Повідомлення про прийняту пропозицію обміну



Рисунок 3.19 – Оновлене поле та баланс гравців

Користувач має можливість закладати поля з метою отримання коштів тут і зараз. Також гравець має можливість покращувати поля у випадку наявності всієї зібраної власності монополії (кожна монополія позначається одним кольором на дошці).

Для закладання відповідного поля необхідно натиснути на нього на дошці та у відкритій панелі натиснути на кнопку “Продати”.



Рисунок 3.20 – Панель, яка надає можливість продати поле власності



Рисунок 3.21 – Закладене поле позначається темним кольором власника

У грі присутні поля шанс, наступаючи на які гравець має шанс отримати певні позитивні, або негативні ефекти.



Рисунок 3.22 – Приклад потрапляння на поле шанс з відповідними наслідками

У випадку, коли гравець потрапляє на поле власності, яке належить іншому гравцю йому відкривається панель сплати оренди за перебування на ньому на рисунку 3.23.



Рисунок 3.23 – Сплата оренди за перебування на полі

У випадку, якщо гравець не має можливості, або бажання продовжувати гру, то у будь-який момент користувач може натиснути на панель зі своїм кольором та іменем і ініціалізувати процес добровільної здачі.

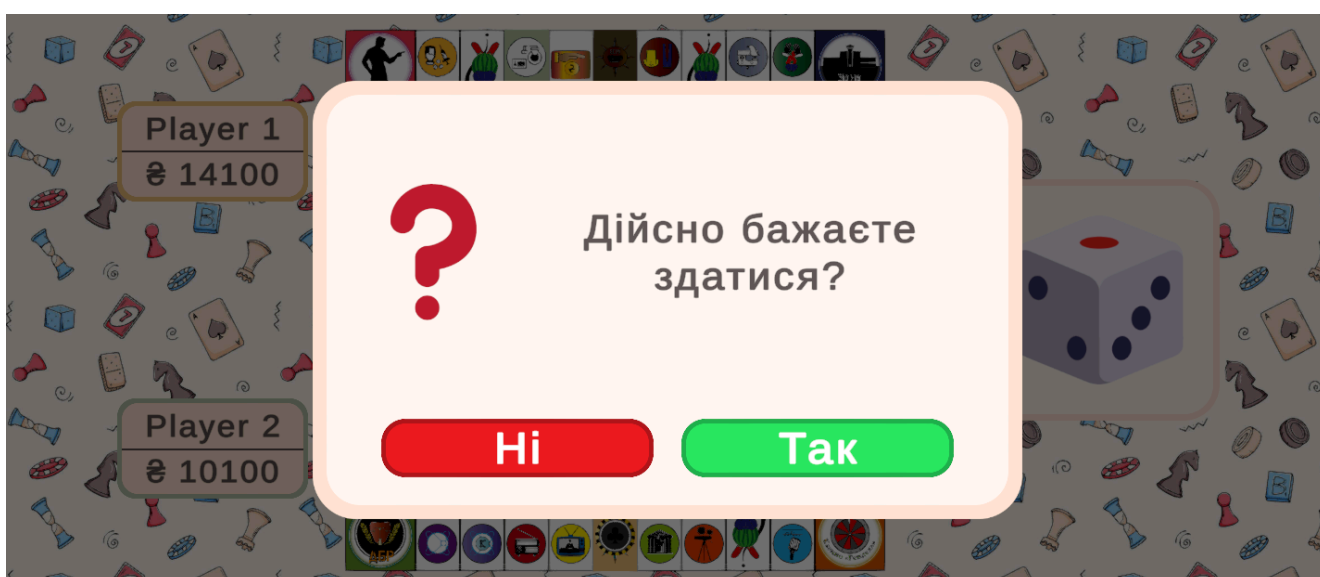


Рисунок 3.24 – Діалогове вікно підтвердження рішення гравця

У випадку підтвердження здачі, гравець видаляється з гри, статки та власність користувача анулюється і право ходу передається наступному гравцю.



Рисунок 3.25 – Екран гравця після добровільної здачі

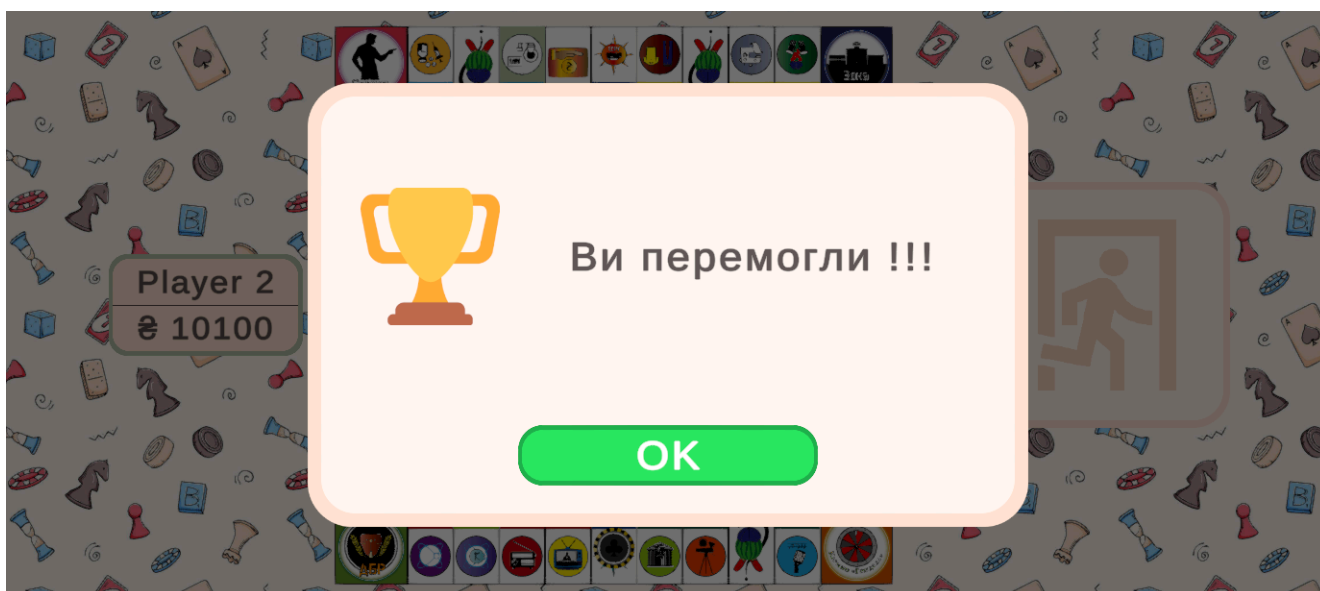


Рисунок 3.26 – Екран іншого гравця, який переміг у матчі

Після натискання на кнопки покинути гру користувач переноситься до головного меню.

На цьому моменті важливо зробити зауваження, що якщо хост ігрового лобі покидає гру, то всі гравців автоматично відключається через особливість архітектури однорангової централізованої мережі.